



Applications web et bases de données documentaires

Interactions entre NodeJS, MongoDB et ElasticSearch

Résumé

Ce rapport répond à trois objectifs :

- Expliquer ce qui m'a amené à réaliser ce projet,
- Détailler les étapes nécessaires et les difficultés rencontrées,
- Partager les conclusions et impressions qui se sont dégagées de cette expérience.

NFE204 – B3D

BASES DE DONNEES DOCUMENTAIRES ET DISTRIBUEES

2^e semestre 2014-2015

Frédéric DEBOS

frederic@debos.fr

Table des matières

Introduction	1
Contexte	1
Choix du thème	1
Réalisation	2
Installation de MongoDB.....	2
Utilisation de base	9
MongoDB et NodeJS.....	12
MapReduce.....	22
Application web	29
Installation d'ElasticSearch	41
ElasticSearch dans NodeJS	49
Conclusion.....	58
Impressions	58
SGBD relationnel VS NoSQL	58
Webographie	59
Les manuels de référence	59
Les tutoriaux :	59
Autres sources :	60
Outils :	60
Annexes	62
Code dans le fichier <code>/app.js</code>	62
Code dans le fichier <code>/routes/index.js</code>	63
Code dans le fichier <code>/public/index.html</code>	66

Introduction

Contexte

Militaire en cours de reconversion, je cumule aujourd'hui 10 années d'expérience de développement à l'Armée dans le monde Java. Ma nouvelle carrière m'engage dans un environnement concurrent car je développe maintenant avec le framework .Net. Spécialisé dans les applications web, je suis familiarisé avec les systèmes de gestion de bases de données tels que MySQL, Oracle et SQL Server.

Etudiant au CNAM, je suis sur le point de finaliser une L3. J'amorce cette année le cycle ingénieur CYC12 car mon objectif à moyen terme sera d'obtenir un diplôme d'ingénieur en informatique.

Le choix du parcours Base de données avancé fut presque une évidence. Malgré mon expérience et les diverses mises à niveaux dispensées par l'Armée, le terme de base de données documentaire m'était encore complètement étranger et m'intriguait au plus haut point. A l'aise avec le langage JavaScript, les structures Json et XML, je sentais que le programme était à ma portée. Et ce qui m'a définitivement convaincu, c'est que la notion de BigData sera abordée. Les compétences dans ce domaine sont un énorme atout sur le marché de l'emploi et j'ai hâte de savoir de quoi il en retourne.

Choix du thème

En suivant les premiers chapitres du cours, j'ai réalisé que JavaScript était omniprésent dans le monde des bases de données documentaires. Certains gestionnaires de bases tels que MongoDB l'utilisent, pour stocker leurs documents, le format Json qui est une syntaxe développée à l'origine pour JavaScript. De plus, dès le chapitre sur MapReduce, on remarque que certains bouts de code en JavaScript permettent de décrire les comportements des différentes étapes du processus.

En tant que développeur spécialisé dans les applications web, j'ai un véritable penchant pour JavaScript. Ce langage est simple à déployer ; avec le temps, il a su abstraire les problèmes de compatibilités avec les différents navigateurs ; et il est de plus en plus riche avec les quantités d'API et frameworks que l'on peut utiliser gratuitement sur Internet.

Un aspect de JavaScript que je souhaitais aborder depuis longtemps était celui du développement coté serveur. C'est ce que NodeJS propose. J'ai eu l'heureuse surprise de découvrir que des plugins de NodeJS permettaient de connecter des applications serveurs NodeJS sur des bases telles que MongoDB.

L'environnement de travail était alors fixé. Il restait à définir quel serait le thème du projet. Un autre souhait pour ce projet était de réaliser une application cartographique. J'ai donc cherché une base documentaire en Json portant sur des données géo-localisées. J'ai trouvé sur le site www.data.gouv.fr une base listant des arbres particuliers référencés dans l'agglomération de Metz.

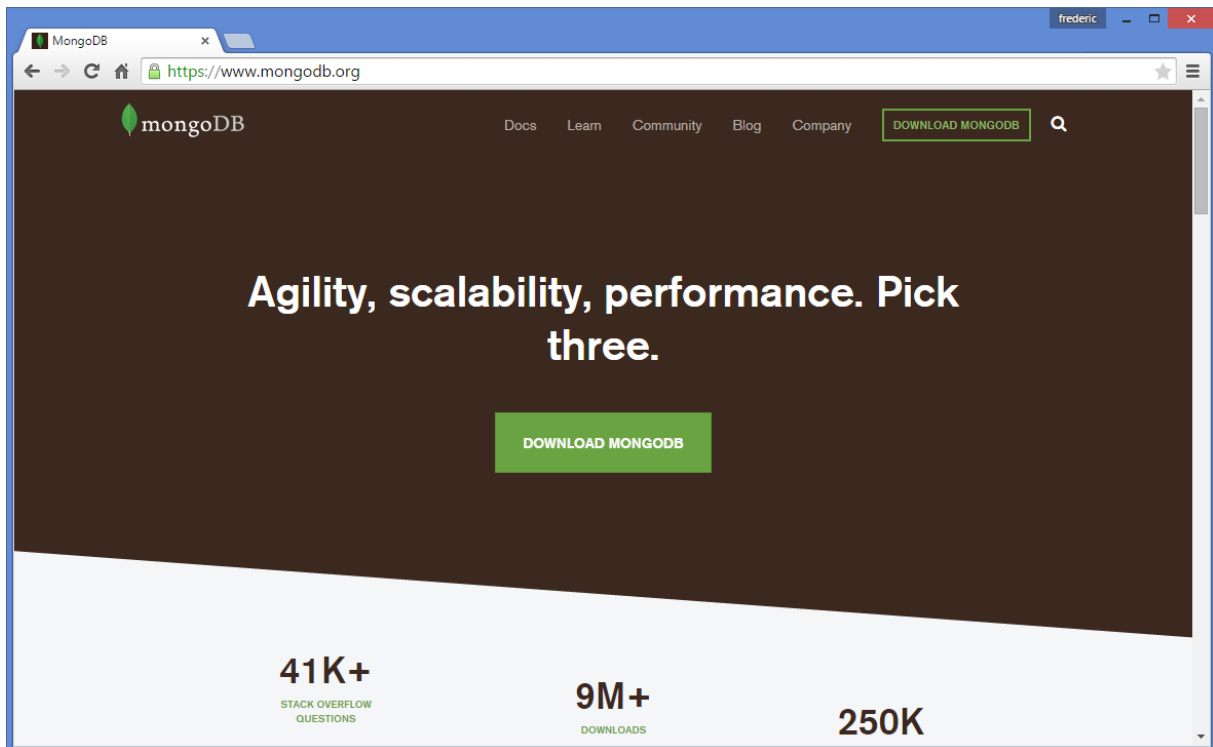
Ainsi, j'avais en tête mes objectifs : réaliser une application web servie par NodeJS et MongoDB. Cette application permettra de localiser des arbres, d'en modifier leur paramétrage et de mettre à disposition une panoplie d'outils de recherche à l'image de ce que proposent les outils liés aux bases de données documentaires.

Réalisation

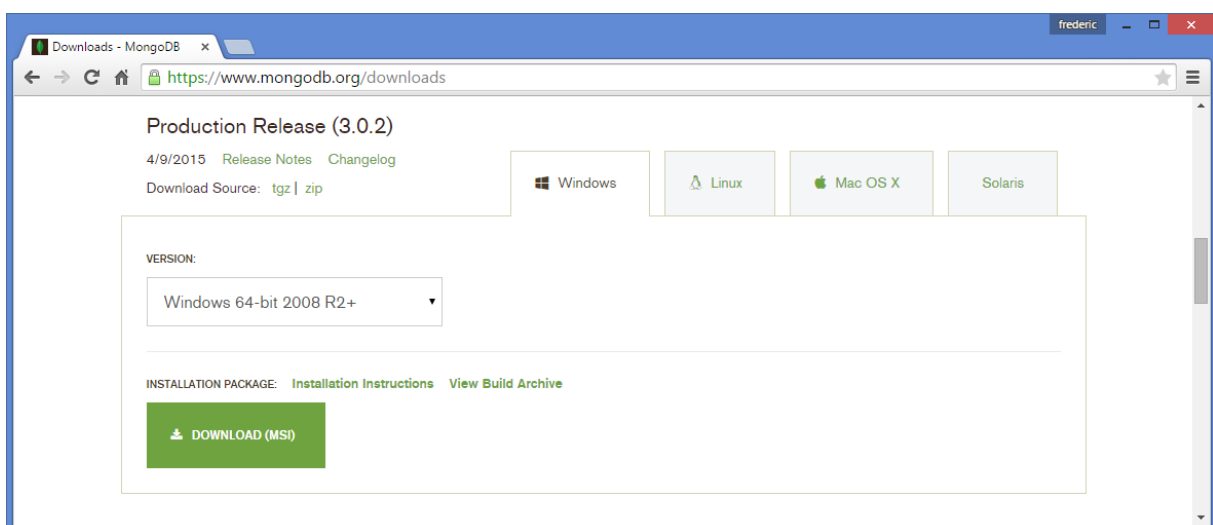
Cette partie du rapport détaille les étapes nécessaires à la réalisation du projet sur Windows 8.

Installation de MongoDB

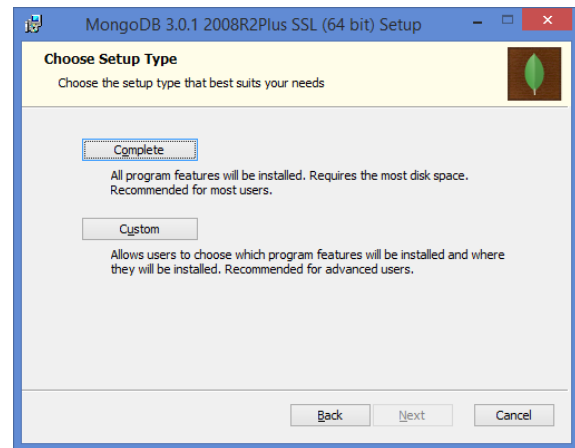
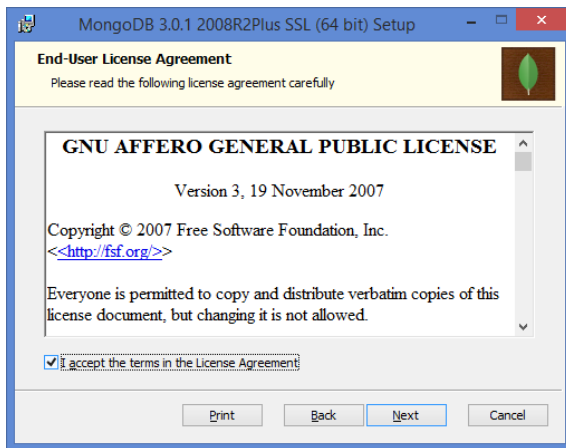
Les quelques étapes suivantes sont grandement inspirées du guide d'installation rapide de MongoDB.



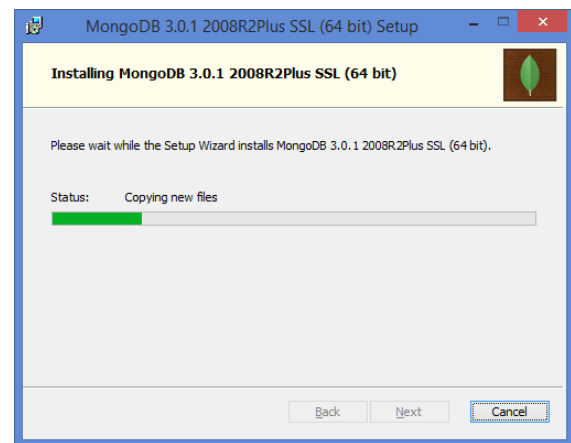
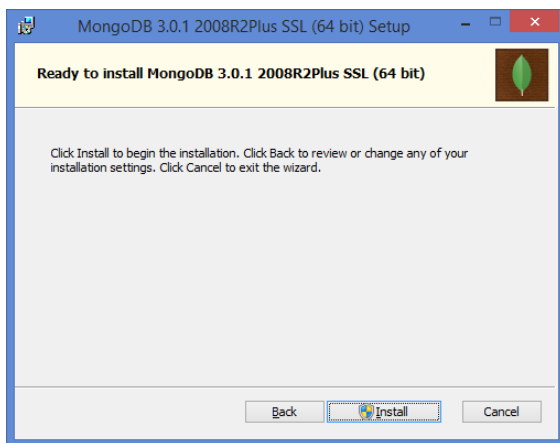
La première chose à faire est de se rendre sur le site de MongoDB : www.mongodb.org



On cherche ensuite l'installer correspondant à l'OS utilisé.



L'installation complète par défaut est adaptée à la suite du projet.



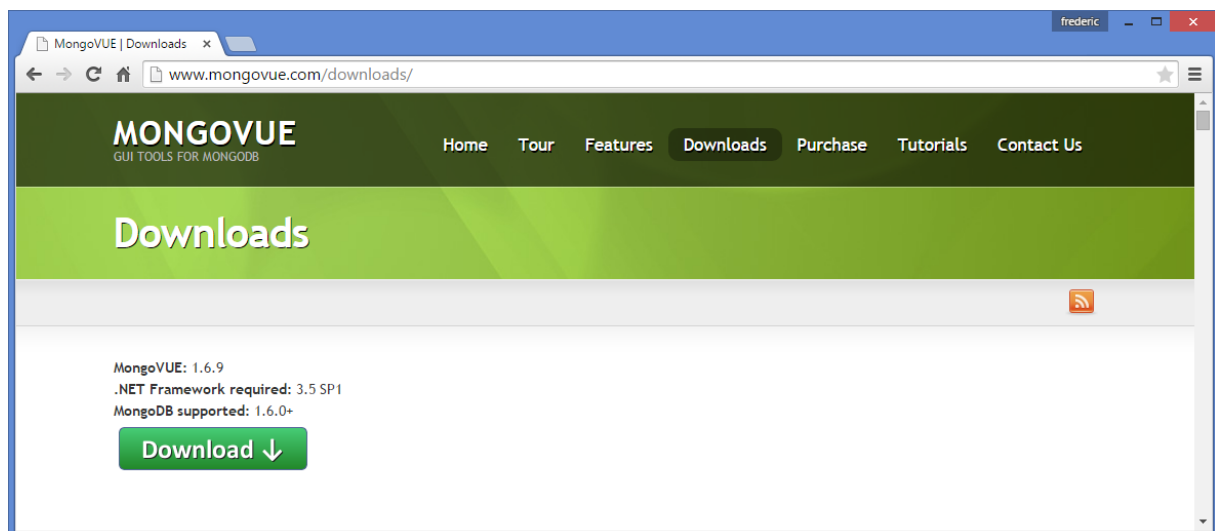
On ne cherche donc pas à modifier les paramètres et on termine l'installation.



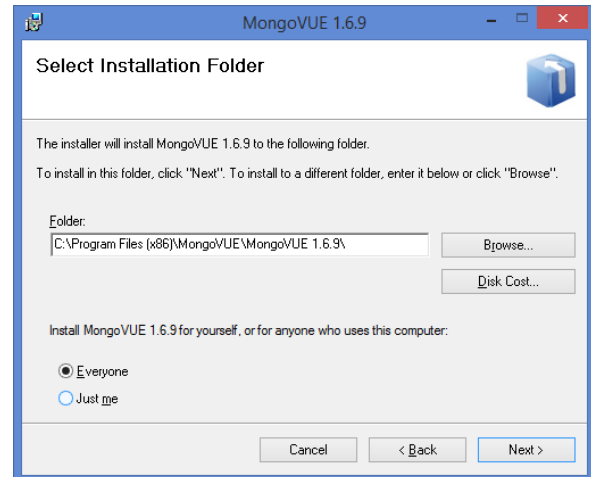
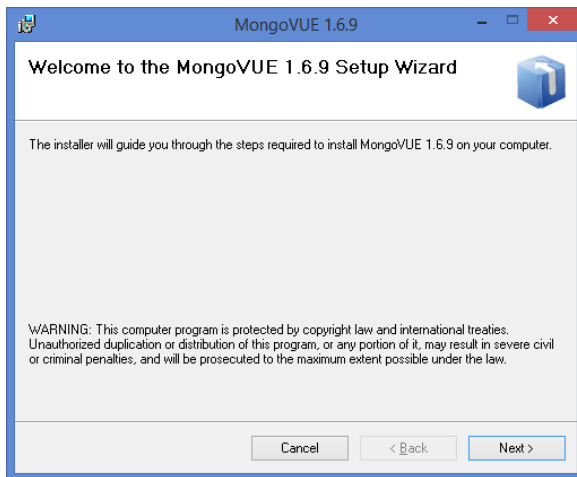
Une fois l'installation terminée, on passe à l'installation d'un utilitaire permettant l'administration et la manipulation des données dans MongoDB.



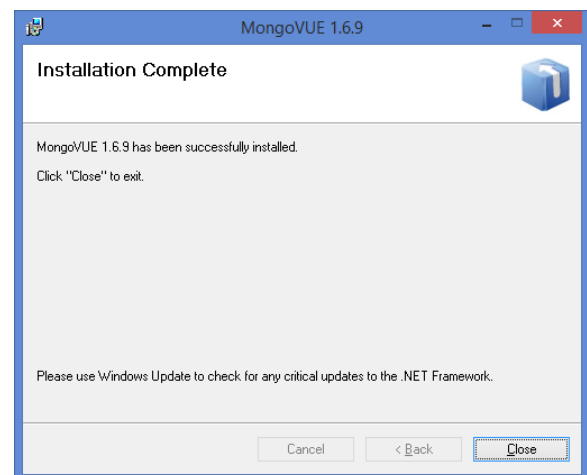
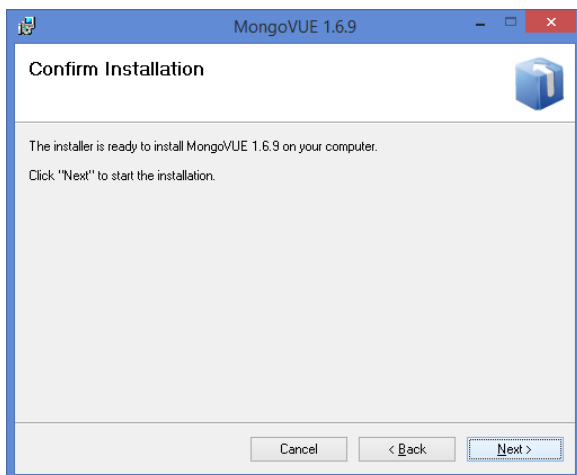
J'ai opté pour MongoVue qui est disponible en shareware. Ce logiciel est intéressant pour apprendre les commandes MongoDB car il les affiche dans une console au moment où elles sont exécutées à travers l'interface utilisateur. Cette approche ludique est donc idéale pour ces premiers pas dans le monde des bases de données documentaires. La durée réduite d'utilisation n'est pas un obstacle car je compte rapidement commander ma base à travers l'application web.



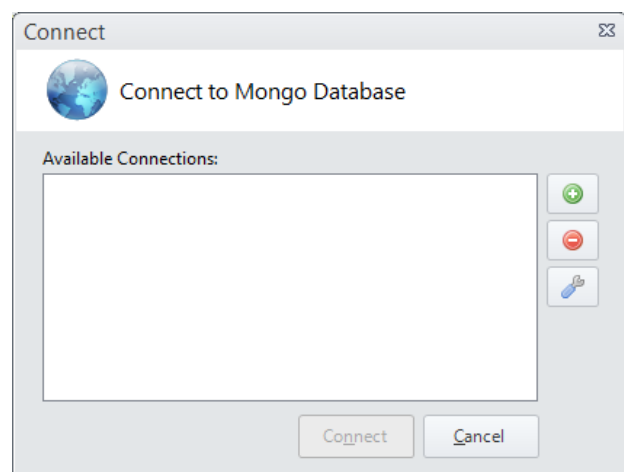
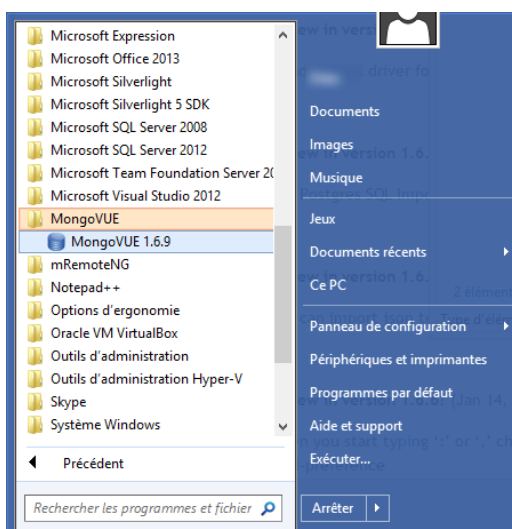
Cette version de MongoVue est compatible avec la base que je viens d'installer car elle peut contrôler MongoDB à partir de sa version 1.6. La version installée est une release 3.0.1.



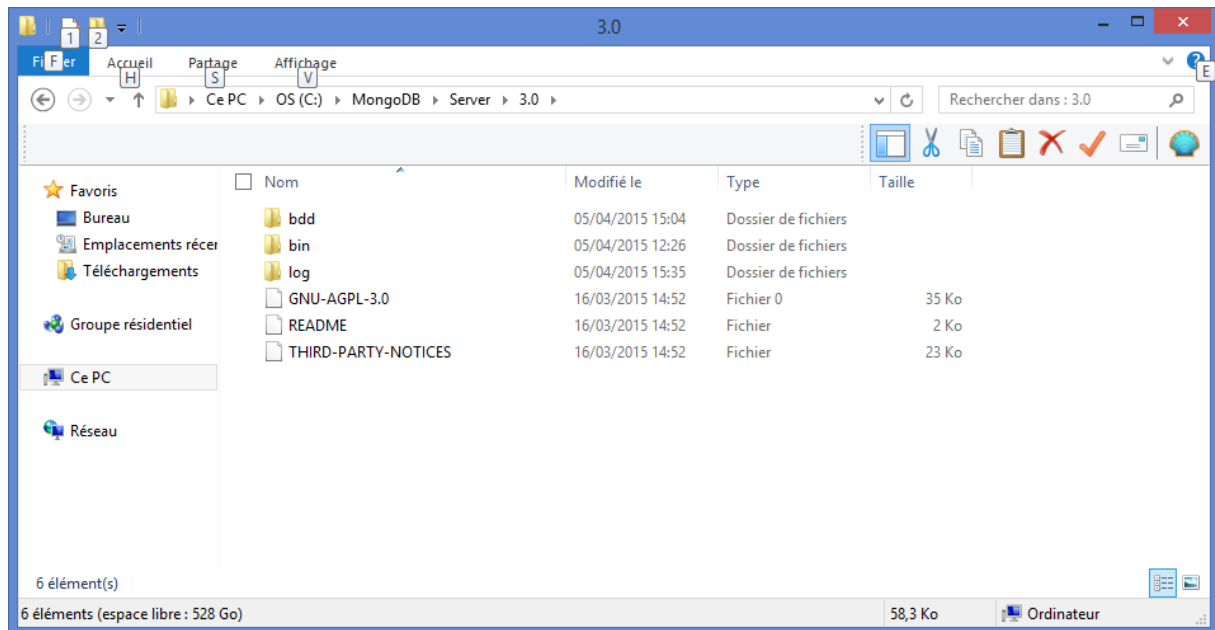
Encore une fois, l'installation par défaut est adaptée à la suite du projet.



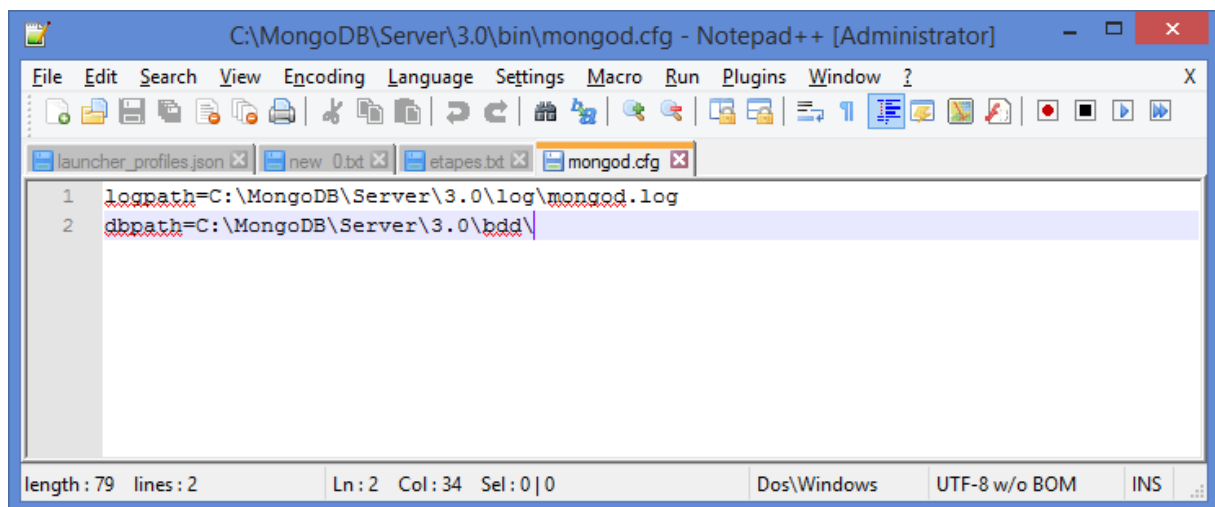
Une fois l'installation terminée, on lance l'application pour commencer à administrer la base nouvellement mise en route.



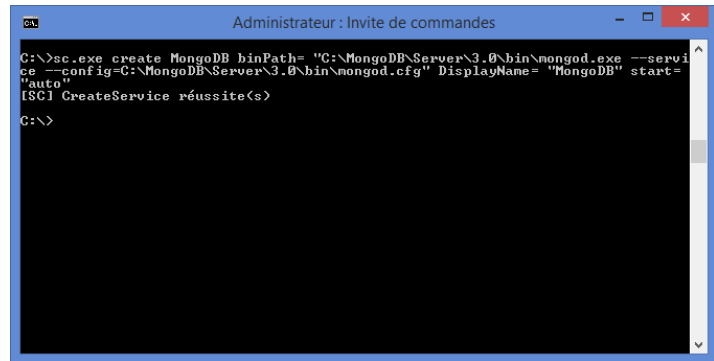
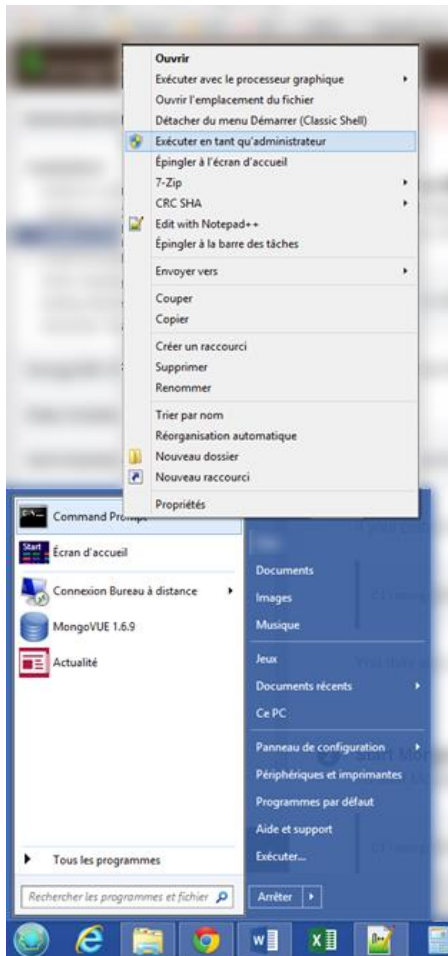
La première chose à faire est de se connecter sur MongoDB.



Le problème c'est que la base n'est pas lancée automatiquement. On va donc créer et configurer l'environnement de travail de MongoDB. On se rend sur le répertoire d'installation et on crée un dossier `bdd` et un autre `log` ...



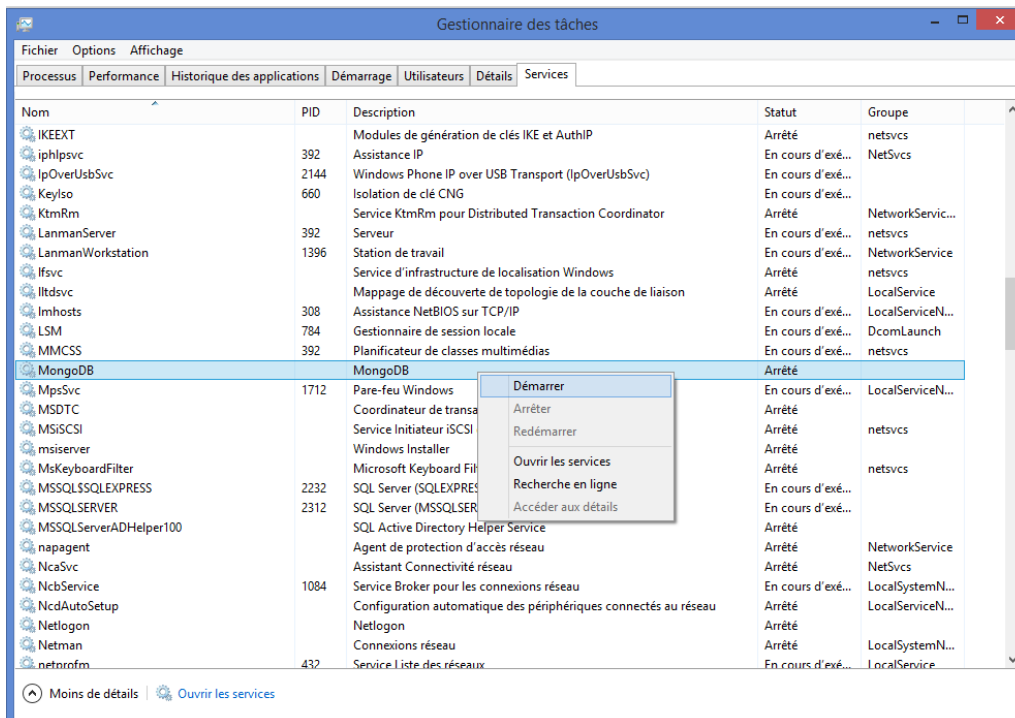
On va ensuite éditer le fichier de configuration dans le dossier bin. Les deux paramètres intéressant à modifier sont `logpath` et `dbpath` qui redirigent respectivement le chemin du fichier de log et le chemin du répertoire stockant les bases.



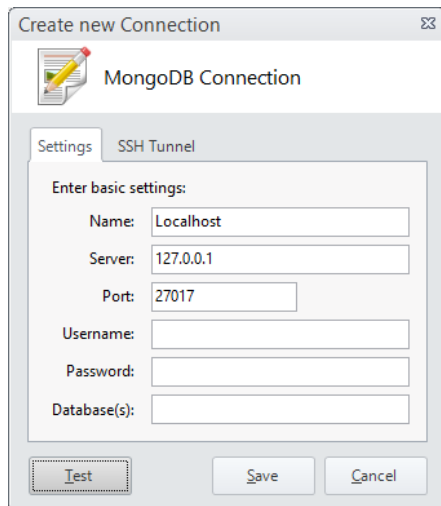
On lance l'invite de commande en tant qu'administrateur pour créer un nouveau service.

Avec la commande suivante, on peut réaliser un service qui démarrera MongoDB automatiquement :

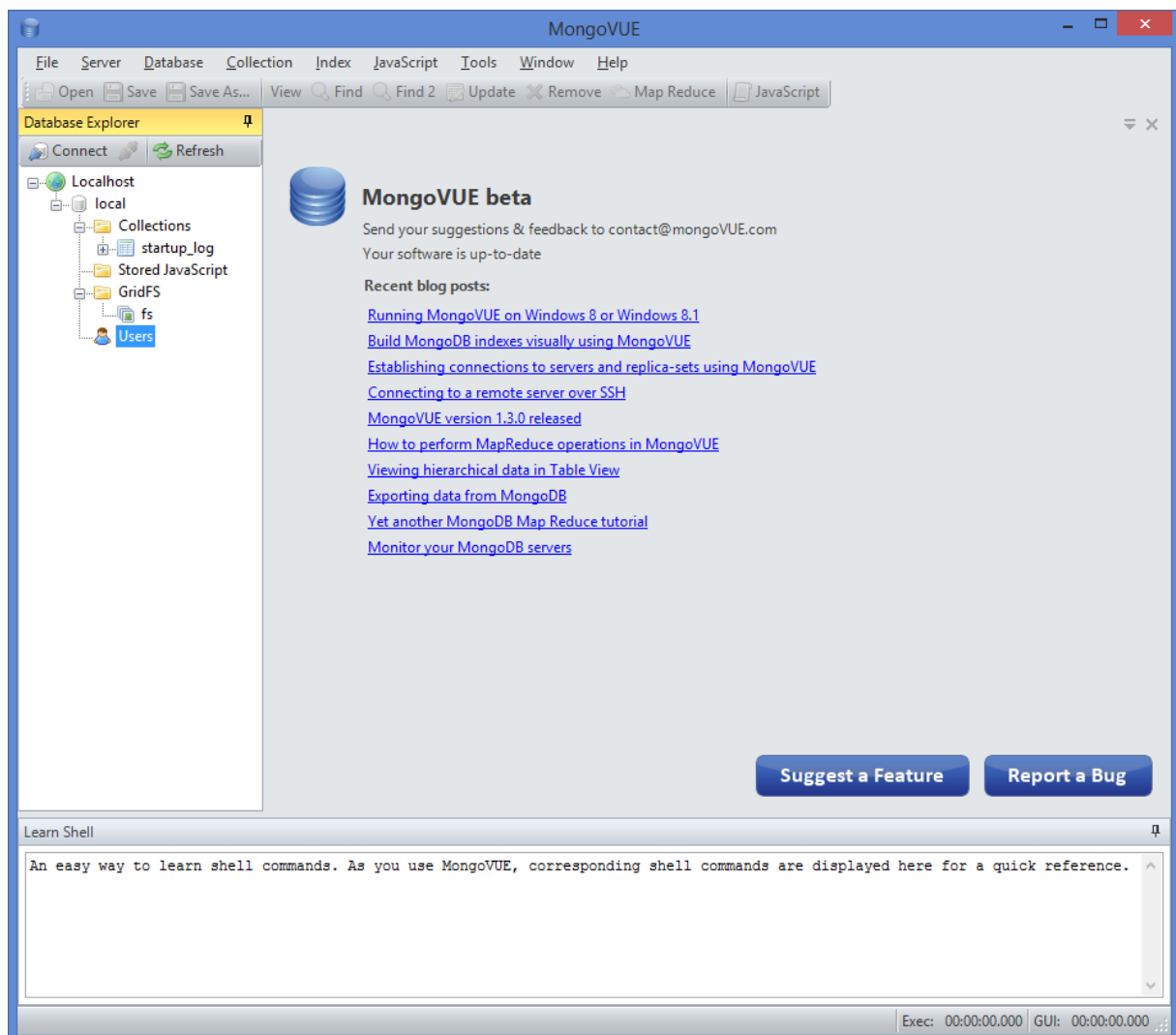
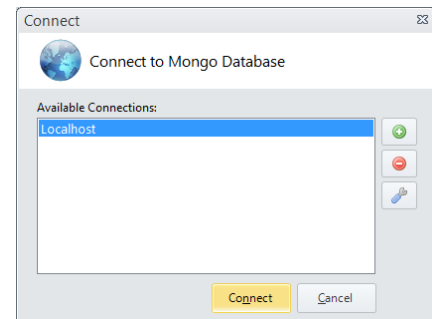
```
sc.exe create MongoDB binPath= "C:\MongoDB\Server\3.0\bin\mongod.exe --service --config=C:\MongoDB\Server\3.0\bin\mongod.cfg" DisplayName= "MongoDB" start= "auto"
```



Le service apparaît dans la liste. Le premier démarrage n'est pas automatique.



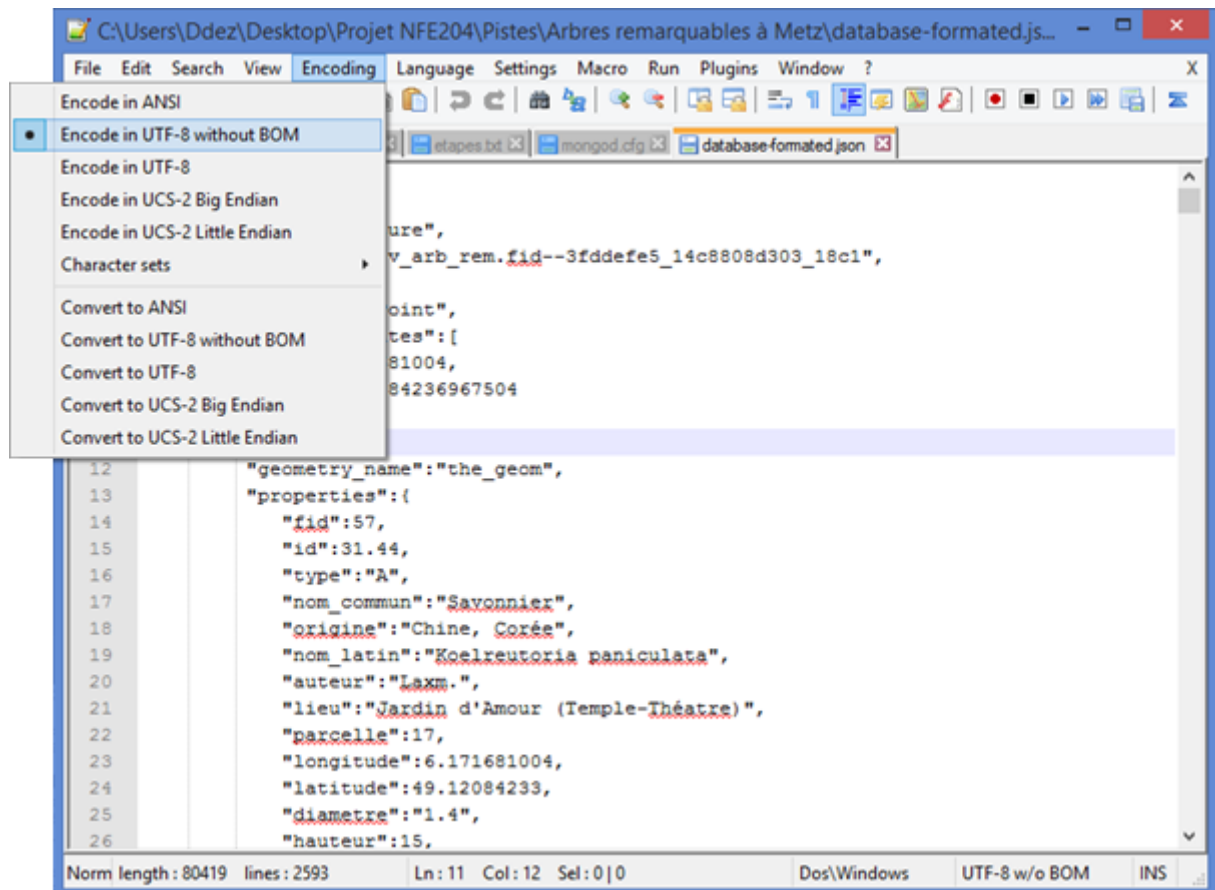
On peut ensuite revenir sur MongoVUE et créer une nouvelle connexion. Aucun paramètre n'a été modifié. Le port par défaut est 27017.



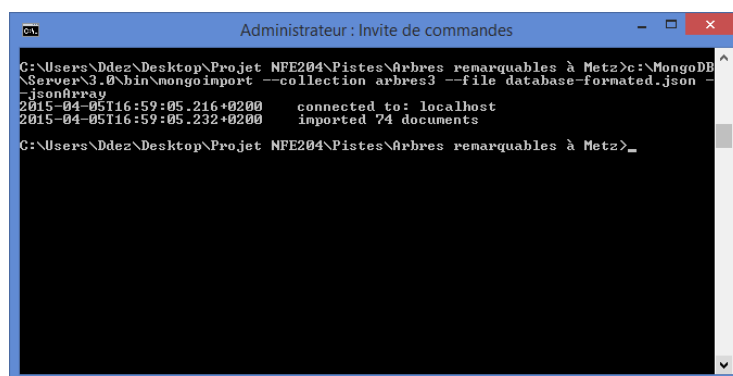
On visualise la liste des collections de la base. Toutes les requêtes réalisées à travers l'interface sont loguées dans la fenêtre « Learn Shell ». Cet outil est très pratique pour découvrir l'utilisation de MongoDB.

Utilisation de base

La source de la base documentaire a été trouvée sur le site www.data.gouv.fr à l'adresse <https://www.data.gouv.fr/fr/datasets/arbres-remarquables-1/>. J'y ai téléchargé la version Json. J'ai réalisé quelques modifications sur la mise en page (sauts de ligne et indentation) avec le site <http://jsonformatter.curiousconcept.com/>. Le problème est que cette technique a apporté des problèmes d'encodage.

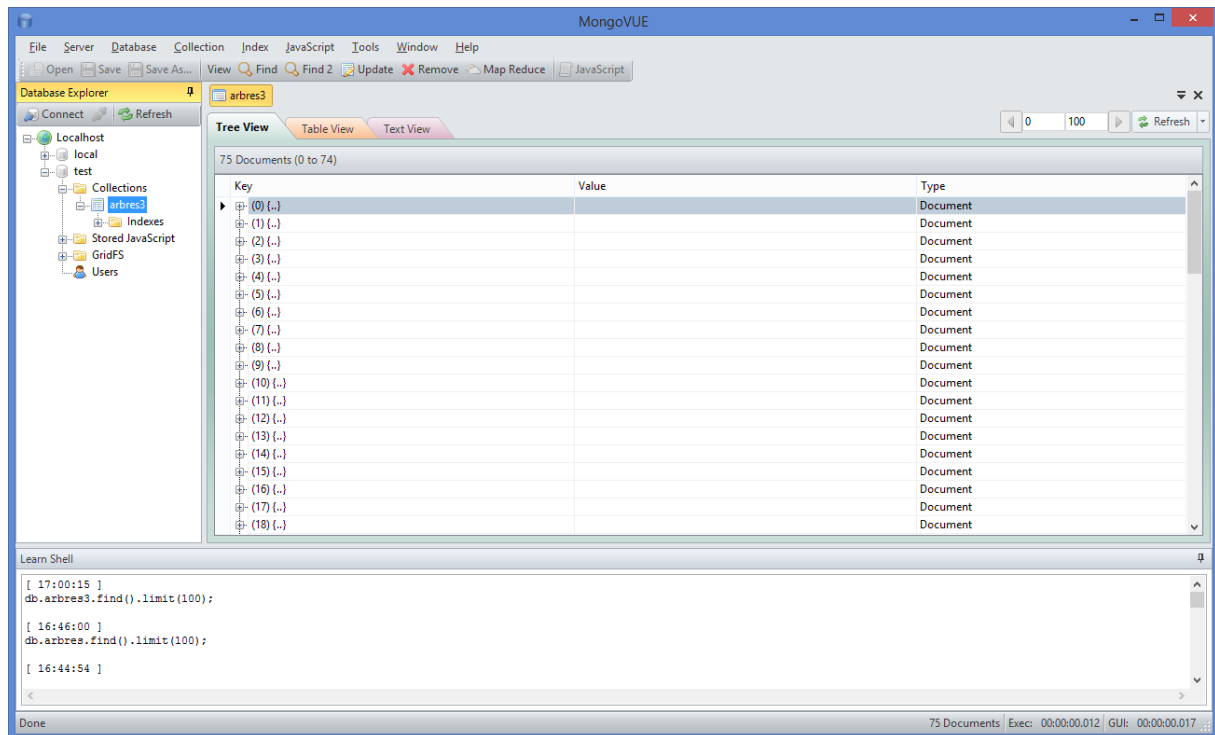


Notepad++ m'a permis de rétablir l'encodage correct. Le fichier résultant se trouve dans l'archive fournie en pièce jointe sous le nom `database-formated.json`.



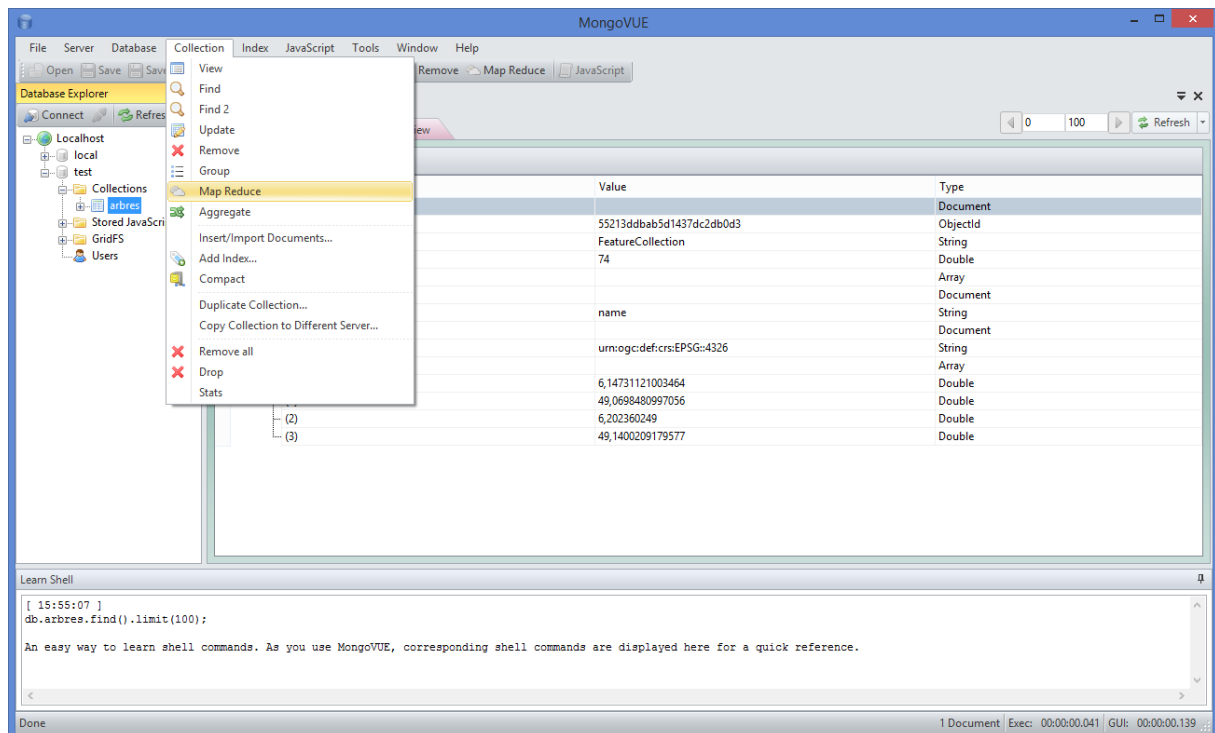
J'ai ensuite injecté ce fichier dans la base MongoDB avec la commande suivante :

```
mongoimport -collection arbres3
-file database-formated.json --
jsonArray
```



En regardant dans MongoVue, on retrouve les 74 documents présents dans le fichier Json. On peut noter au passage que l'affichage de cette collection s'est traduite dans la console par la commande

```
db.arbres3.find().limit(100)
```



Par l'interface utilisateur, on peut accéder à un formulaire permettant l'exécution d'une opération de MapReduce.

Une fois les étapes `map`, `reduce`, `finalize` et `out` renseignées avec des fonctions JavaScript, on peut lancer l'action. Voici la commande qui apparaît alors dans la console « Learn Shell » :

```
db.runCommand(
{
  mapreduce: "arbres3",
  map : function Map () {
    emit(
      this.properties.nom_commun, // valeur de regroupement
      this // donnée associée
    );
  },
  reduce : function Reduce (key, values) { // ne passent ici que les groupes a
vec plusieurs données
    var reduced = {key:key, values:values};
    return reduced;
  },
  finalize : function Finalize (key, reduced) { // passent ici toutes les donn
ées, réduites ou non
    var finalized = reduced;
    if (reduced.id) { // passage pour les données non groupées
      finalized = [reduced];
    } else { // passage pour les données traitées par Reduce
      finalized = reduced.values;
    }
    return finalized;
  },
  out : { replace: "arbres4" } // enregistrement du résultat dans une nouvelle
collection
}
);
```

Les arbres n'avaient aucun classement dans la collection `arbres3`. Ils apparaissent maintenant dans la collection `arbres4` avec un premier critère de regroupement : leur nom commun.

_id	value
Arbre au caramel	Array[1]
Arbre au liège de l'Amour	Array[1]
Arbre aux quarantes écus	Array[1]
Arbre de fer	Array[2]
Chicot du canada	Array[1]
Chêne des marais	Array[1]
Chêne pédonculé	Array[7]
Chêne rouge d'Amérique	Array[1]
Cyprés chauve	Array[1]
Cèdre blanc ou Cèdre à encens	Array[1]
Cèdre de l'atlas	Array[1]
Erable à écorce de bouleau	Array[1]
Faux cyprès de Lawson	Array[1]
Faux orme de Sibérie	Array[1]

Voici le résultat dans MongoVue.

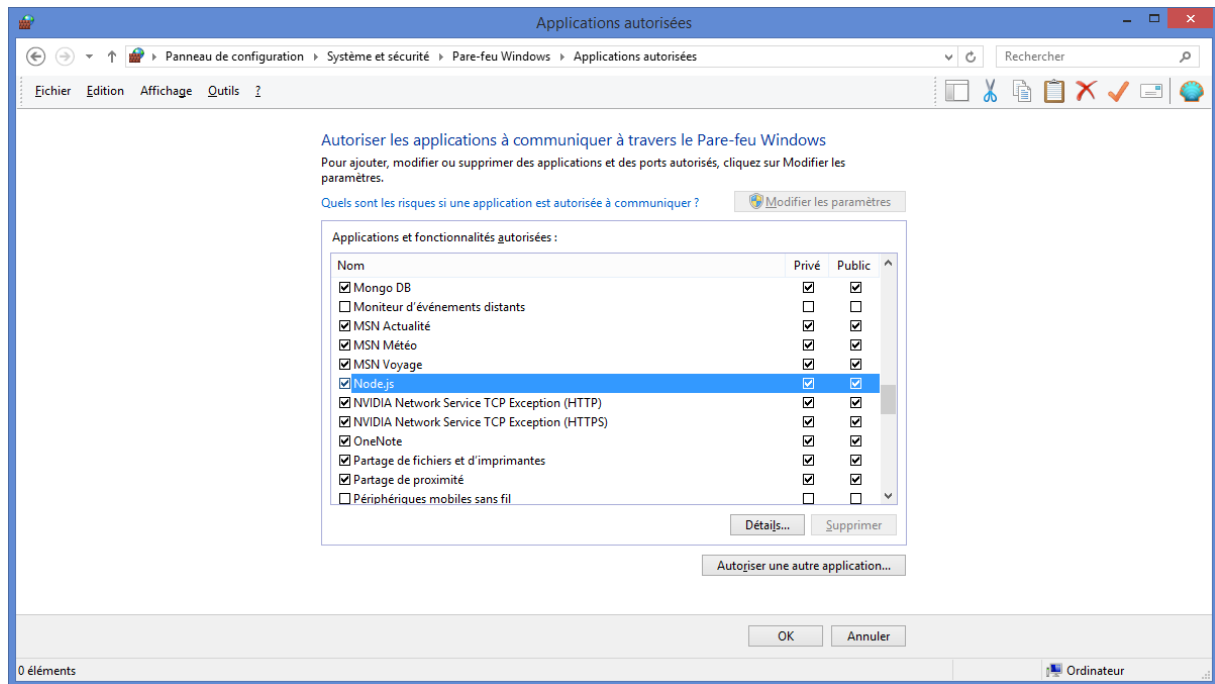
MongoDB et NodeJS

On vient de voir que l'utilisation de MongoDB n'est pas très complexe. On peut simplement injecter des collections à partir de fichiers Json. On peut interroger les bases grâce à des requêtes ou à travers des interfaces utilisateur riches. De plus, la procédure de MapReduce offre un moyen de persister des regroupements qui permettront des accès simplifiés pour les futures requêtes de l'application.

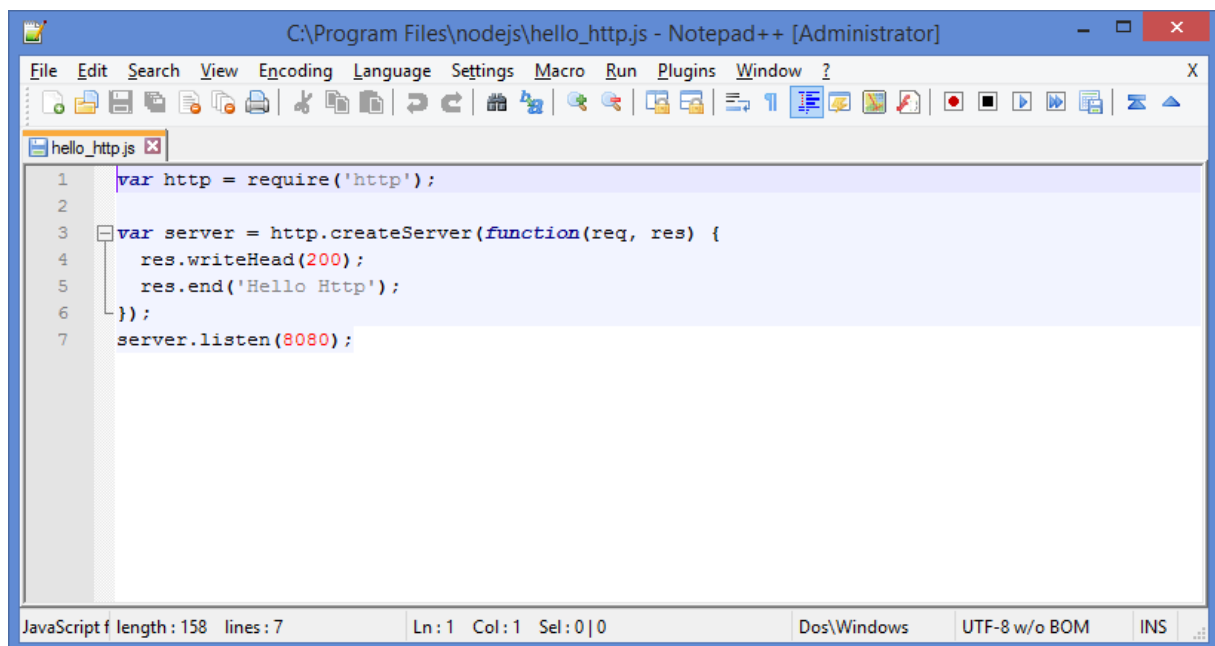
Avant de se lancer dans le développement de l'application en elle-même, voyons comment utiliser NodeJS et surtout, comment requêter MongoDB à travers ce serveur web.



La première étape consiste à installer NodeJS. Pour cela, il faut se rendre sur le site [www.nodejs.org](https://nodejs.org). Le bouton « INSTALL » lance directement le téléchargement. On installe, comme toujours, avec les paramètres par défaut.



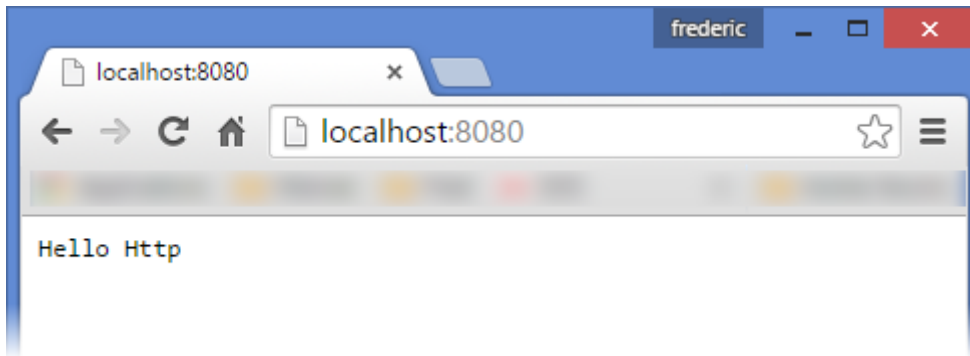
Une fois NodeJS installé, on crée une exception dans le pare-feu de Windows pour cette application ainsi que pour MongoDB.



Etant novice dans l'utilisation de ce serveur, je commence par créer une simple page du type Hello World en suivant les lignes de codes données par un tutorial sur Internet.



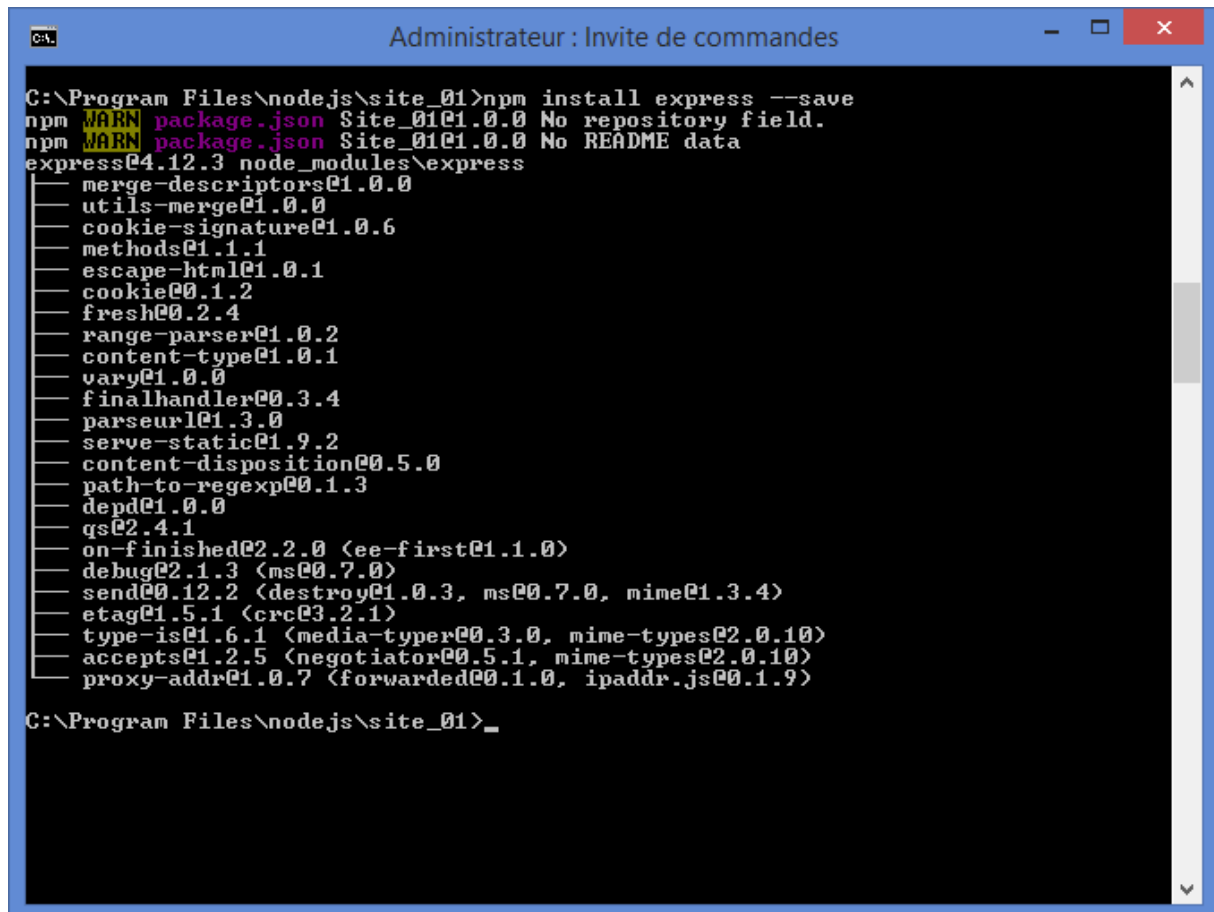
Je lance ensuite le serveur NodeJS pour qu'il déploie la page que je viens de créer.



La page est accessible sur le port 8080 tel qu'il a été défini dans le script du fichier hello_http.js.

L'utilisation semble basique. Pour réaliser la suite, j'ai eu recours à un tutorial que l'on peut trouver sur le site <http://cwbuecheler.com/web/tutorials/2013/node-express-mongo/>

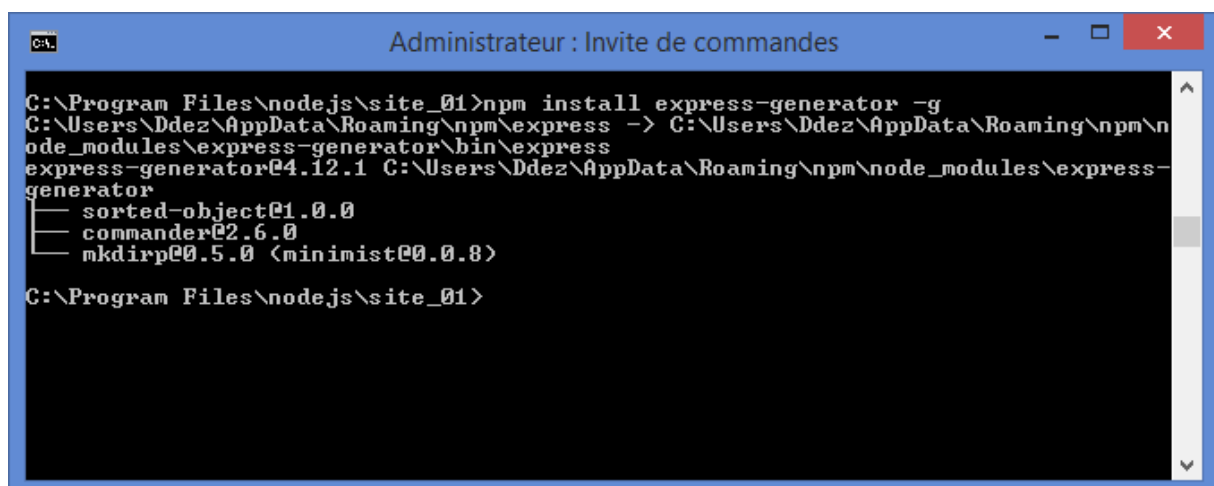
Ce tuto décrit les étapes nécessaires pour le déploiement d'un site avec ExpressJS (un module de NodeJS) et pour brancher ce site sur MongoDB.



```
C:\Program Files\nodejs\site_01>npm install express --save
npm WARN package.json Site_01@1.0.0 No repository field.
npm WARN package.json Site_01@1.0.0 No README data
express@4.12.3 node_modules\express
├── merge-descriptors@1.0.0
├── utils-merge@1.0.0
├── cookie-signature@1.0.6
├── methods@1.1.1
├── escape-html@1.0.1
├── cookie@0.1.2
├── fresh@0.2.4
├── range-parser@1.0.2
├── content-type@1.0.1
├── vary@1.0.0
├── finalhandler@0.3.4
├── parseurl@1.3.0
├── serve-static@1.9.2
├── content-disposition@0.5.0
├── path-to-regexp@0.1.3
├── depd@1.0.0
├── qs@2.4.1
├── on-finished@2.2.0 (ee-first@1.1.0)
├── debug@2.1.3 (ms@0.7.0)
├── send@0.12.2 (destroy@1.0.3, ms@0.7.0, mime@1.3.4)
├── etag@1.5.1 (crc@3.2.1)
├── type-is@1.6.1 (media-typer@0.3.0, mime-types@2.0.10)
├── accepts@1.2.5 (negotiator@0.5.1, mime-types@2.0.10)
└── proxy-addr@1.0.7 (forwarded@0.1.0, ipaddr.js@0.1.9)

C:\Program Files\nodejs\site_01>
```

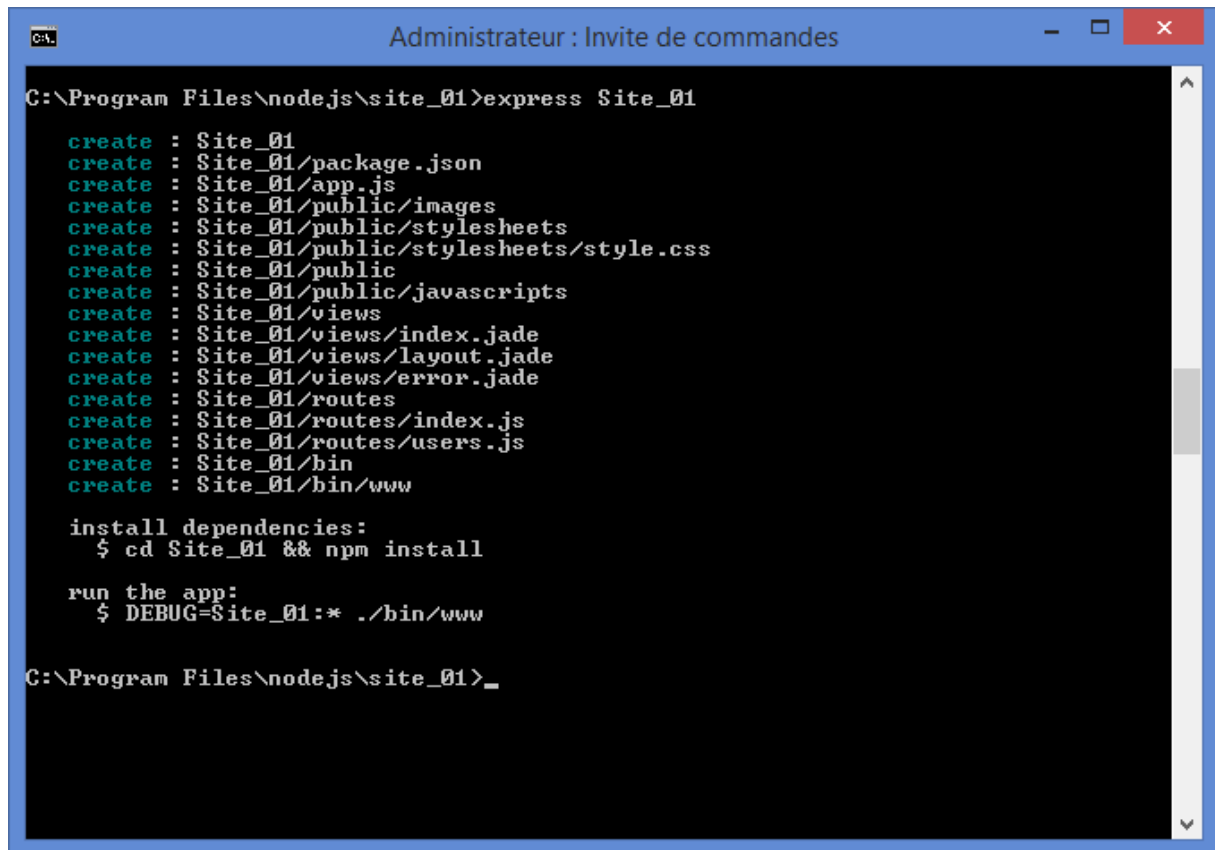
Je crée un répertoire `Site_01` qui contiendra l'ensemble des éléments nécessaire au site web. La commande `npm` lance des opérations de NodeJS. Dans ce répertoire, je lance donc une installation du plugin ExpressJS.



```
C:\Program Files\nodejs\site_01>npm install express-generator -g
C:\Users\Ddez\AppData\Roaming\npm\express -> C:\Users\Ddez\AppData\Roaming\npm\node_modules\express-generator\bin\express
express-generator@4.12.1 C:\Users\Ddez\AppData\Roaming\npm\node_modules\express-generator
├── sorted-object@1.0.0
├── commander@2.6.0
└── mkdirp@0.5.0 (minimist@0.0.8)

C:\Program Files\nodejs\site_01>
```

J'y installe également `express-generator` qui permettra de générer le squelette de mon site.



```
C:\Program Files\nodejs\site_01>express Site_01

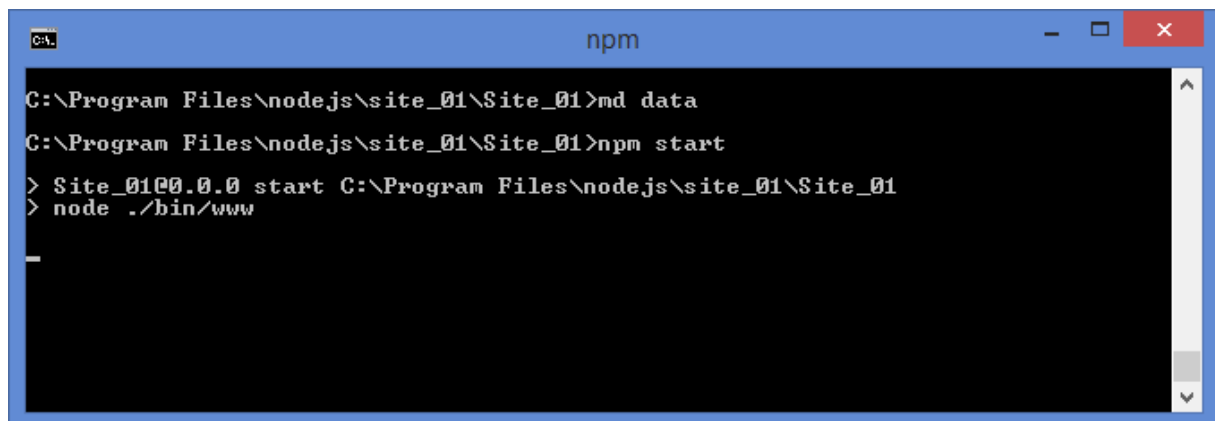
create : Site_01
create : Site_01/package.json
create : Site_01/app.js
create : Site_01/public/images
create : Site_01/public/stylesheets
create : Site_01/public/stylesheets/style.css
create : Site_01/public
create : Site_01/public/javascripts
create : Site_01/views
create : Site_01/views/index.jade
create : Site_01/views/layout.jade
create : Site_01/views/error.jade
create : Site_01/routes
create : Site_01/routes/index.js
create : Site_01/routes/users.js
create : Site_01/bin
create : Site_01/bin/www

install dependencies:
$ cd Site_01 && npm install

run the app:
$ DEBUG=Site_01:* ./bin/www

C:\Program Files\nodejs\site_01>_
```

Je peux maintenant utiliser ExpressJS pour déployer mon site. Il est représenté dans NodeJS comme un nouveau nœud.

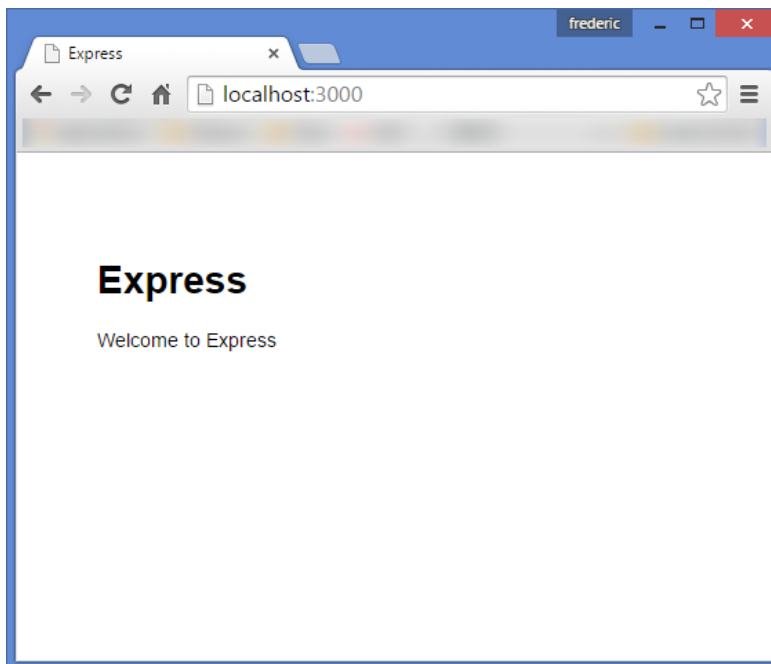


```
C:\Program Files\nodejs\site_01\Site_01>md data
C:\Program Files\nodejs\site_01\Site_01>npm start

> Site_01@0.0.0 start C:\Program Files\nodejs\site_01\Site_01
> node ./bin/www

_
```

Je lance ensuite le serveur pour déployer le site.



Et on peut découvrir la page par défaut générée par ExpressJS sur le port 3000.

A screenshot of a Notepad++ window titled 'C:\Program Files\nodejs\site_01\Site_01\package.json - Notepad++ [Administrator]'. The window shows the content of a 'package.json' file. The code is as follows:

```
1 {
2   "name": "Site_01",
3   "version": "0.0.0",
4   "private": true,
5   "scripts": {
6     "start": "node ./bin/www"
7   },
8   "dependencies": {
9     "body-parser": "~1.12.0",
10    "cookie-parser": "~1.3.4",
11    "debug": "~2.1.1",
12    "express": "~4.12.2",
13    "jade": "~1.9.2",
14    "morgan": "~1.5.1",
15    "serve-favicon": "~2.2.0",
16    "mongodb": "*",
17    "monk": "*"
18  }
19 }
```

The status bar at the bottom indicates 'Normal text file', 'length: 362 lines: 19', 'Ln: 15 Col: 30 Sel: 37|3', 'UNIX', 'UTF-8 w/o BOM', and 'INS'.

Maintenant il faut connecter ce site sur MongoDB. Pour cela, j'ajoute deux dépendances dans le fichier `package.json` du site : `mongodb` et `monk`.

```

C:\Program Files\nodejs\site_01>cd Site_01
C:\Program Files\nodejs\site_01\Site_01>npm install
npm WARN peerDependencies The peer dependency mongodb@~1.4 included from mongoskin
in will no
npm WARN peerDependencies longer be automatically installed to fulfill the peerD
ependency
npm WARN peerDependencies in npm 3+. Your application will need to depend on it
explicitly.
> kerberos@0.0.10 install C:\Program Files\nodejs\site_01\Site_01\node_modules\m
ongodb\node_modules\mongodb-core\node_modules\kerberos
> (node-gyp rebuild 2) builderror.log) !! (exit 0)
!
C:\Program Files\nodejs\site_01\Site_01\node_modules\mongodb\node_modules\mongod
b-core\node_modules\kerberos>if not defined npm_config_node_gyp (node "C:\Progra
m Files\nodejs\node_modules\npm\bin\node-gyp-bin\..\..\node_modules\node-gyp\bi
n\node-gyp.js" rebuild) > else (rebuild)

```

En lançant la commande `install` ...

```

qs@2.4.1
etag@1.5.1 (crc@3.2.1)
on-finished@2.2.0 (ee-first@1.1.0)
send@0.12.2 (ms@0.7.0, destroy@1.0.3, mime@1.3.4)
proxy-addr@1.0.7 (forwarded@0.1.0, ipaddr.js@0.1.9)
accepts@1.2.5 (negotiator@0.5.1, mime-types@2.0.10)
type-is@1.6.1 (media-typer@0.3.0, mime-types@2.0.10)
body-parser@1.12.2 node_modules\body-parser
  content-type@1.0.1
  bytes@1.0.0
  raw-body@1.3.3
  depd@1.0.0
  on-finished@2.2.0 (ee-first@1.1.0)
  qs@2.4.1
  type-is@1.6.1 (media-typer@0.3.0, mime-types@2.0.10)
  iconv-lite@0.4.7
mongodb@2.0.25 node_modules\mongodb
  readable-stream@1.0.31 (string_decoder@0.10.31, isarray@0.0.1, core-util-is@
1.0.1, inherits@2.0.1)
  mongodb-core@1.1.21 (rimraf@2.2.6, mkdirp@0.5.0, kerberos@0.0.10, bson@0.3.1
)
jade@1.9.2 node_modules\jade
  commander@2.6.0
  character-parser@1.2.1
  void-elements@2.0.1
  mkdirp@0.5.0 (minimist@0.0.8)
  with@4.0.2 (acorn-globals@1.0.3, acorn@1.0.1)
  constantinople@3.0.1 (acorn-globals@1.0.3)
  transformers@2.1.0 (promise@2.0.0, css@1.0.8, uglify-js@2.2.5)
monk@1.0.1 node_modules\monk
  mpromise@0.5.1
  mongodb@1.4.35 (readable-stream@1.0.33, bson@0.2.21, kerberos@0.0.9)
  mongoskin@1.4.13
C:\Program Files\nodejs\site_01\Site_01>_

```

... les nouvelles dépendances sont automatiquement installées.

```

1  var express = require('express');
2  var path = require('path');
3  var favicon = require('serve-favicon');
4  var logger = require('morgan');
5  var cookieParser = require('cookie-parser');
6  var bodyParser = require('body-parser');
7
8      // Mon code
9      var mongo = require('mongodb');
10     var monk = require('monk');
11     var db = monk('localhost:27017/test');
12     // Mon code
13
14     var routes = require('./routes/index');
15     var users = require('./routes/users');
16
17     var app = express();
18
19     // view engine setup
20     app.set('views', path.join(__dirname, 'views'));
21     app.set('view engine', 'jade');
22
23     // uncomment after placing your favicon in /public
24     //app.use(favicon(path.join(__dirname, 'public', 'favicon.ico')));
25     app.use(logger('dev'));
26     app.use(bodyParser.json());
27     app.use(bodyParser.urlencoded({ extended: false }));
28     app.use(cookieParser());
29     app.use(express.static(path.join(__dirname, 'public')));
30
31     // Mon code
32     // Rend la variable db accessible dans le router
33     app.use(function(req,res,next){
34         req.db = db;
35         next();
36     });
37     // Mon code
38
39     app.use('/', routes);
40     app.use('/users', users);

```

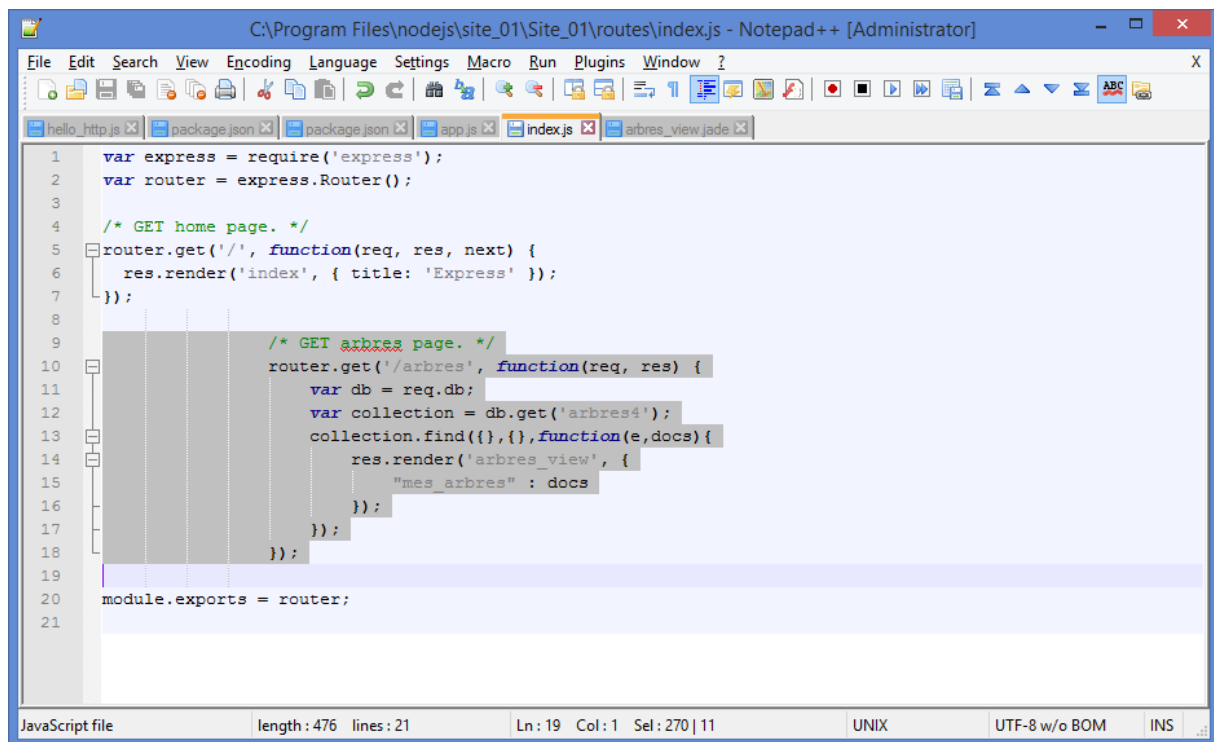
JavaScript file length: 1734 lines: 74 Ln: 32 Col: 58 Sel: 0 | 0 UNIX UTF-8 w/o BOM INS

Le fichier d'entrée d'un nœud NodeJS est `app.js`. Il se trouve dans le répertoire principal. Ce fichier contient le code qui permettra l'initialisation du site. Il ne sera donc exécuté qu'au démarrage. On peut y instancier toutes les dépendances et les connexions.

C'est dans ce fichier que je crée les instances des plugins `mongodb` et `monk`. Ils me permettent d'établir une connexion à la base de données MongoDB avec l'URL suivante :

```
localhost:27017/test
```

Cette connexion est ensuite transmise à l'ensemble des requêtes par l'ajout d'une propriété nommée `db`.



```
1 var express = require('express');
2 var router = express.Router();
3
4 /* GET home page. */
5 router.get('/', function(req, res, next) {
6   res.render('index', { title: 'Express' });
7 });
8
9 /* GET arbres page. */
10 router.get('/arbres', function(req, res) {
11   var db = req.db;
12   var collection = db.get('arbres4');
13   collection.find({}, {}, function(e, docs) {
14     res.render('arbres_view', {
15       "mes_arbres" : docs
16     });
17   });
18 });
19
20 module.exports = router;
21
```

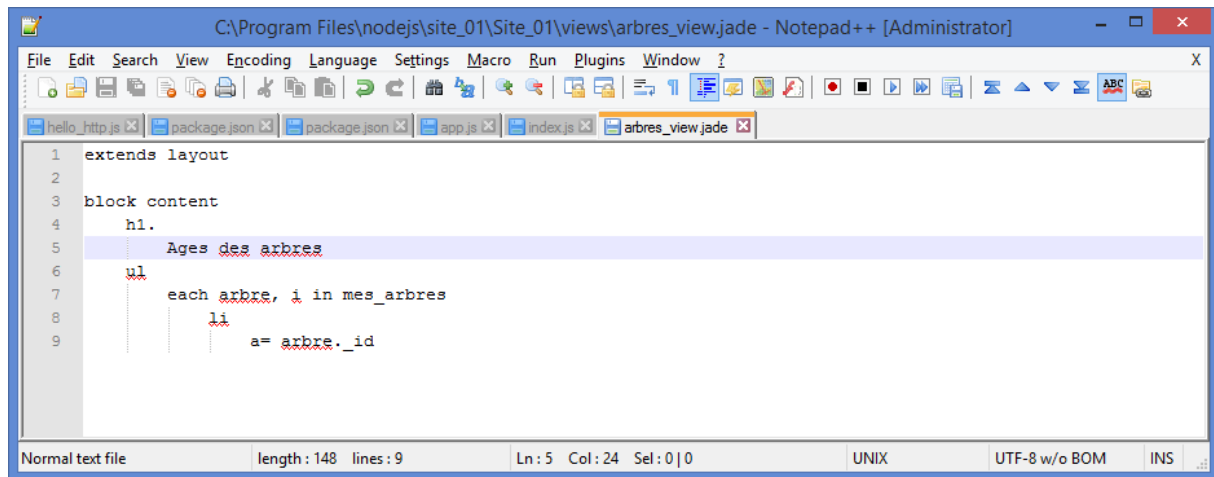
Le second fichier important est le fichier `index.js` présent dans le dossier `routes`. Dans ce fichier, nous pouvons coder toutes les logiques de routage dépendantes de l'URL demandée et de la méthode utilisée (GET ou POST). Dans notre cas, le projet sera simple et nous n'avons pas besoin de mettre en place une architecture logicielle élaborée. Nous allons donc nous contenter de coder les accès en base et les réponses au client directement dans les fonctions de routage.

Le code ci-dessus cherche à traiter les requêtes HTTP envoyées en GET à l'URL `/arbres`. On retrouve la variable `db` dans le paramètre `req` qui nous permettra d'utiliser la connexion à MongoDB instanciée au démarrage du serveur NodeJS.

Avec cette variable `db`, on appelle la collection `arbres4` qui avait été générée dans MongoVue avec un MapReduce regroupant les arbres d'après leurs noms communs.

Dans cette collection, on applique un `find` à l'instar de la commande native de MongoDB. Comme aucun filtre n'a été spécifié, tous les documents seront récupérés.

Ces documents sont alors présents dans le paramètre `docs` de la fonction de callback de `find`. On passe la main au renderer de NodeJS en lui indiquant qu'il faudra créer la vue `arbres_view` avec les paramètres `{ "mes_arbres" : docs }`.



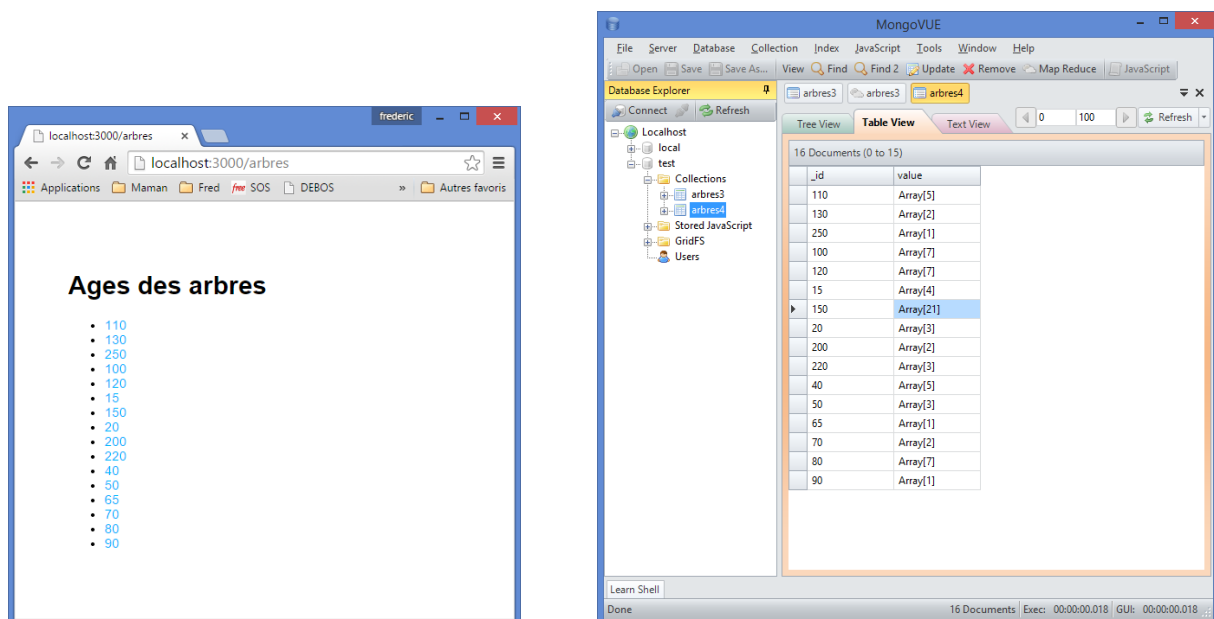
```

1 extends layout
2
3 block content
4   h1.
5     Ages des arbres
6   ul
7     each arbre, i in mes_arbres
8       li
9         a= arbre._id

```

La vue `arbres_view` est décrite dans le fichier `/views/arbres_view.jade`. Jade est un langage proche du HTML. Son écriture est simplifiée car les balises ne sont pas échappées par les caractères `<` et `>` comme en XML. De plus, l'utilisation de balises fermantes est remplacée par un usage strict de l'indentation.

Sur cette page, on remarque que l'on retrouve la variable `mes_arbres` que l'on a passé en paramètre précédemment. On itère sur cette liste pour afficher les clés de regroupement.



Entre temps, j'avais écrasé la collection `arbres4` pour faire un nouveau MapReduce avec un regroupement sur les âges. C'est pour cela que l'on ne retrouve pas la liste des noms communs comme on aurait pu le prévoir.

On peut voir sur l'URL de la page que l'on appelle bien `/arbres` en méthode GET. La structure de la page est conforme à la description du fichier Jade et le contenu est représentatif des documents persistés. Cela prouve que l'on est capable d'interroger MongoDB à travers NodeJS.

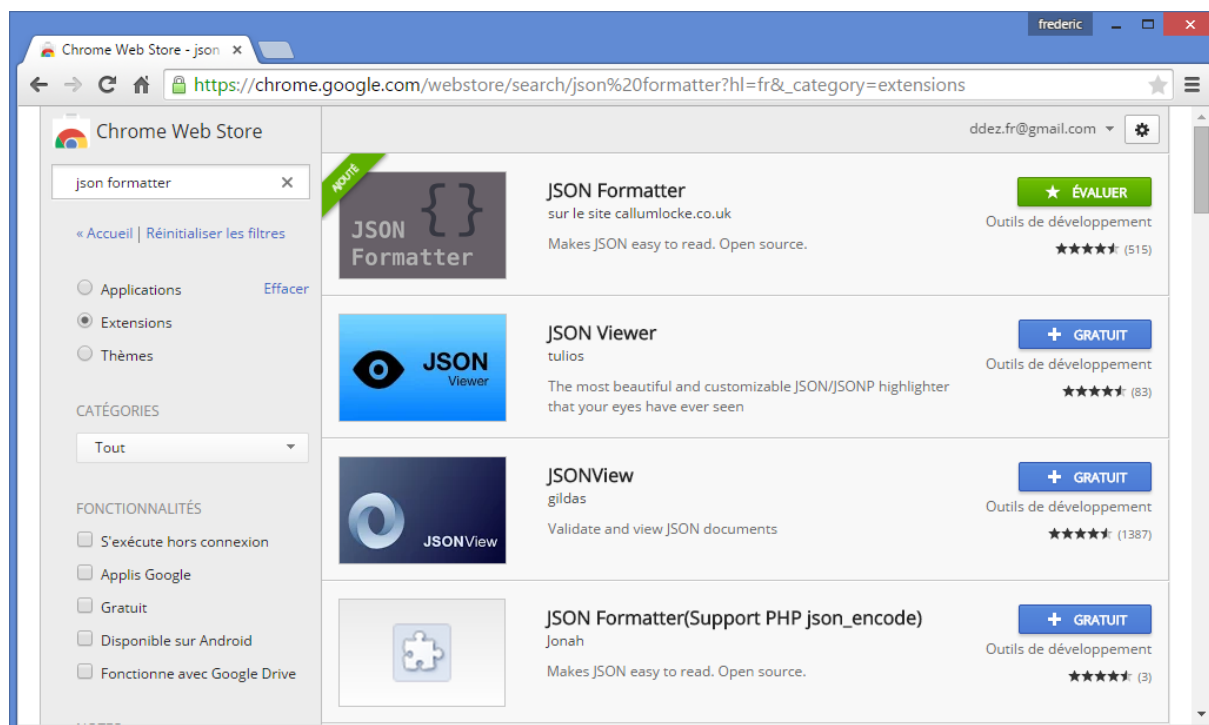
MapReduce

Dans l'exercice précédent, on a utilisé une collection provenant d'un MapReduce. Cette utilisation est séduisante car les données sont prêtes à consulter si tant est que le critère d'entrée est une recherche des arbres par âge. Cependant, il est facile de pressentir les difficultés que l'on aura lorsque l'on réalisera des modifications sur ces arbres. Ce ne sera pas une collection qu'il faudra mettre à jour, mais deux ; sans compter les difficultés que l'on pourra avoir à retrouver un document dans une collection qui n'est pas destinées à réaliser des recherches par identifiant mais par âge.

Malgré tout, MapReduce est très efficace pour regrouper des données selon un critère. Nous n'allons donc pas abandonner cette méthode : nous allons la rendre dynamique en exécutant cette action dans le serveur NodeJS.

J'ajoute à tout cela un autre objectif. Le Web 2.0 n'étant pas né d'hier, une application web se doit d'être réactive et sans rechargement complet de page. L'appel de documents via Ajax est un incontournable pour remplir cet objectif. Pour ce projet, j'opte pour la distribution de documents au format Json. On évitera ainsi le déploiement de nouveaux plugins (comme il aurait pu être nécessaire pour générer des réponses XML) tout en gardant la flexibilité dans la structure de nos données.

Nous allons donc développer de nouvelles ressources. Les URL demandées répondront retourneront des flux Json dans lequel nous transmettrons les documents correspondant à la requête.

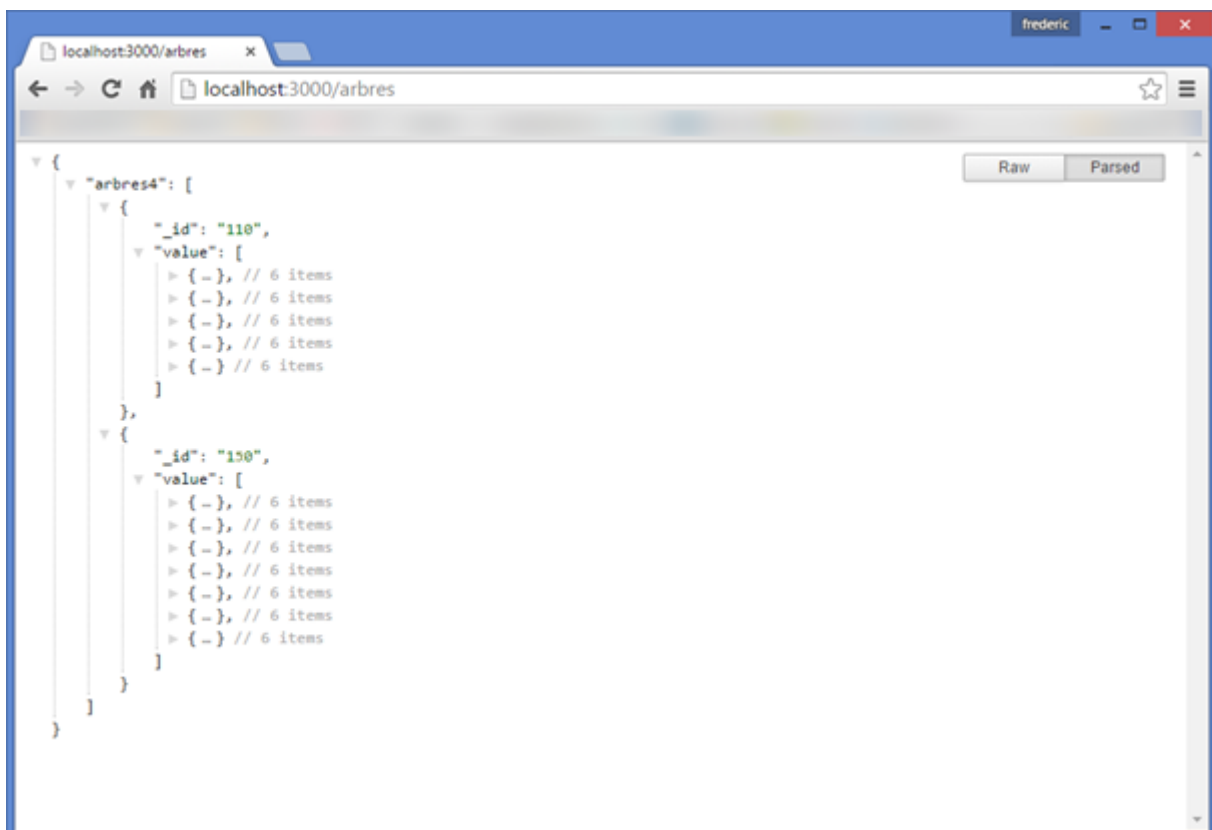


Utilisant Chrome pour la qualité de ses outils de débogage, je passe par ce navigateur pour la suite de mes développements. J'ajoute une extension bien pratique permettant de formater les flux Json dans le rendu de la page.

```

1  var express = require('express');
2  var router = express.Router();
3
4  /* GET home page. */
5  router.get('/', function(req, res, next) {
6    res.render('index', { title: 'Express' });
7  });
8
9  /* GET arbres page. */
10 router.get('/arbres', function(req, res) {
11   var db = req.db;
12   var collection = db.get('arbres4');
13   collection.find({ _id: { $in: [ '110', '150' ] } }, {}, function(e, docs) {
14     // le résultat est retourné au format json
15     res.writeHead(200, {"Content-Type": "application/json"});
16     var json = JSON.stringify({
17       arbres4: docs
18     });
19     res.end(json);
20   });
21 });
22
23 module.exports = router;
24

```



Je fais un premier test avec une requête qui retourne l'ensemble des arbres ayant soit 110 ans, soit 150 ans. La réponse est traitée sans passer par le renderer. On précise dans l'entête HTTP que le flux est au format Json et on écrit la liste sérialisée de document dans la réponse.

La mise en page de Json Formatter permet une lecture aisée du flux.

```

5 router.get('/', function(req, res, next) {
6   res.render('index', { title: 'Express' });
7 });
8
9
10 /* GET arbres page. */
11 router.get('/arbres', function(req, res) {
12   var db = req.db;
13   var collection = db.get('arbres4');
14   collection.find({}, function(e, docs) {
15     // le résultat est retourné au format json
16     res.writeHead(200, {"Content-Type": "application/json; charset=utf-8"});
17
18     var myItems = [];
19     for (var idx in docs) {
20       var item = docs[idx];
21       if(parseInt(item._id) > 110) {
22         myItems.push(item);
23       }
24     }
25
26     var json = JSON.stringify({ arbres4: myItems });
27     res.end(json);
28   });
29 });
30 module.exports = router;

```

JavaScript file length: 789 lines: 31 Ln: 13 Col: 39 Sel: 0|0 UNIX UTF-8 w/o BOM INS

```

{
  "arbres4": [
    {
      "_id": "130",
      "value": [ ] // 2 items
    },
    {
      "_id": "250",
      "value": [ ] // 1 item
    },
    {
      "_id": "120",
      "value": [ ] // 7 items
    },
    {
      "_id": "150",
      "value": [ ] // 21 items
    },
    {
      "_id": "200",
      "value": [ ] // 2 items
    },
    {
      "_id": "220",
      "value": [ ] // 3 items
    }
  ]
}

```

Le problème que je rencontre, c'est que l'âge est stocké en chaîne de caractère. On ne peut pas réaliser de comparaison numérique dessus sauf en récupérant tous les documents et en passant la valeur de l'identifiant en Integer. Il faudra éviter le classement de valeurs numérique en chaînes à l'avenir.

```
C:\Program Files\nodejs\site_01\Site_01>npm install mongoose --save
> kerberos@0.0.10 install C:\Program Files\nodejs\site_01\Site_01\node_modules\mongoose\node_modules\mongodb\node_modules\mongodb-core\node_modules\kerberos
> <node-gyp rebuild 2> builderror.log> !! <exit 0>

C:\Program Files\nodejs\site_01\Site_01\node_modules\mongoose\node_modules\mongodb\node_modules\mongodb-core\node_modules\kerberos>if not defined npm_config_node_gyp (node "C:\Program Files\nodejs\node_modules\npm\bin\node-gyp-bin\..\..\node_modules\node-gyp\bin\node-gyp.js" rebuild ) else <rebuild>

> bson@0.2.21 install C:\Program Files\nodejs\site_01\Site_01\node_modules\mongoose\node_modules\mongodb\node_modules\mongodb-core\node_modules\bson
> <node-gyp rebuild 2> builderror.log> !! <exit 0>

C:\Program Files\nodejs\site_01\Site_01\node_modules\mongoose\node_modules\mongodb\node_modules\mongodb-core\node_modules\bson>if not defined npm_config_node_gyp (node "C:\Program Files\nodejs\node_modules\npm\bin\node-gyp-bin\..\..\node_modules\node-gyp\bin\node-gyp.js" rebuild ) else <rebuild>
mongoose@4.0.1 node_modules\mongoose
├── regexp-clone@0.0.1
├── mpromise@0.5.4
├── hooks-fixed@1.0.1
├── muri@1.0.0
├── sliced@0.0.5
├── mpath@0.1.1
├── kareem@1.0.0
├── async@0.9.0
├── ms@0.1.0
├── mquery@1.4.0 <debug@0.7.4, bluebird@2.3.2>
├── mongodb@2.0.24 <readable-stream@1.0.31, mongodb-core@1.1.20>
C:\Program Files\nodejs\site_01\Site_01>
```

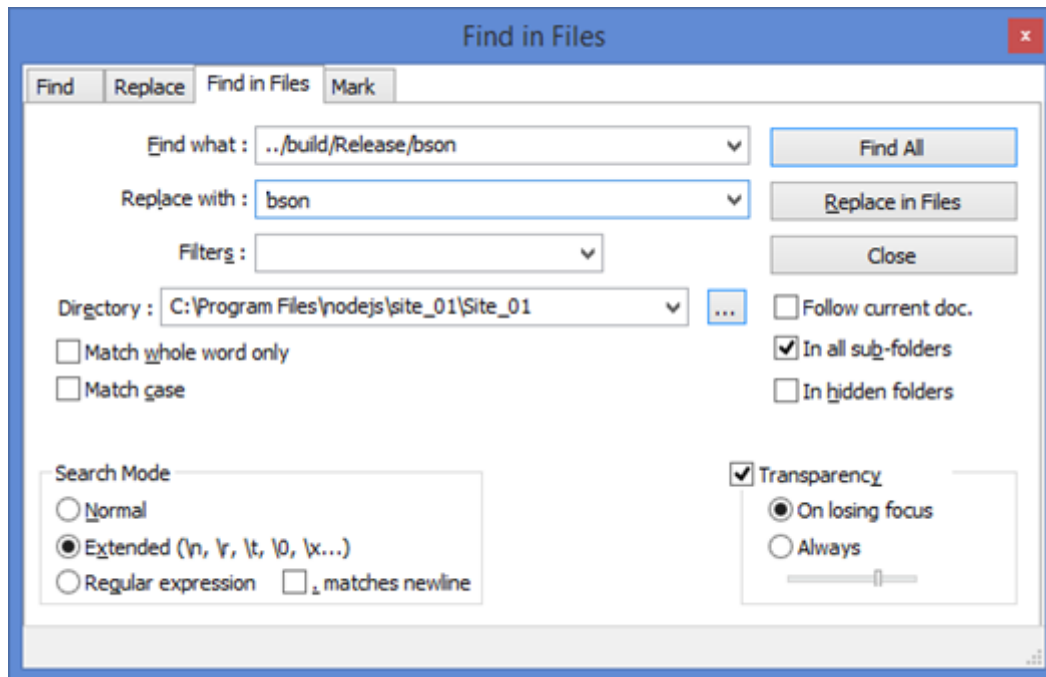
Avec Monk, on ne peut pas réaliser de requêtes complexes sur MongoDB. On passe donc à une autre librairie qui s'appelle Mongoose. Celle-ci est bien plus évoluée et permet notamment de réaliser des opérations de MapReduce.

```
C:\Program Files\nodejs\site_01\Site_01>npm start

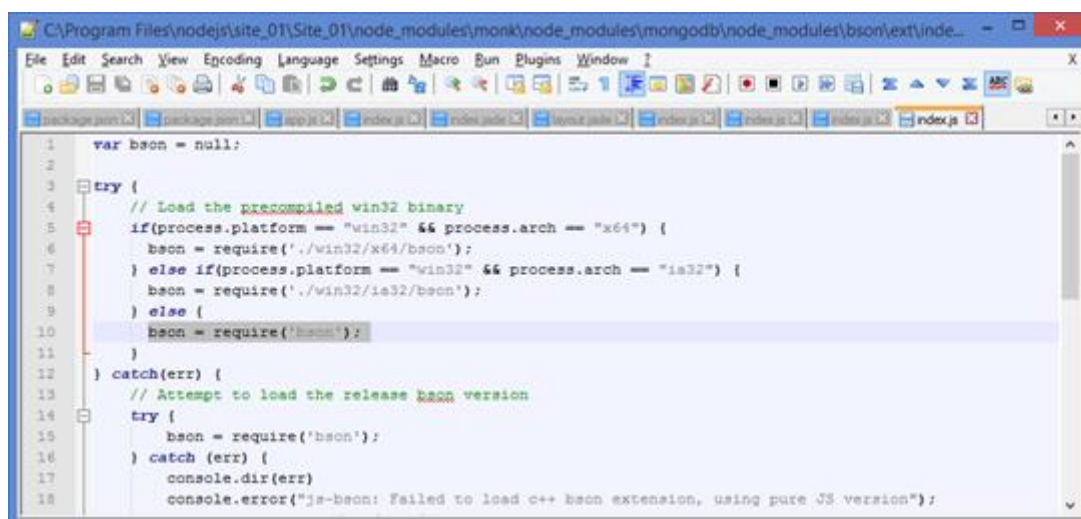
> Site_01@0.0.0 start C:\Program Files\nodejs\site_01\Site_01
> node ./bin/www

{ [Error: Cannot find module '../build/Release/bson'] code: 'MODULE_NOT_FOUND' }
js-bson: Failed to load c++ bson extension, using pure JS version
{ [Error: Cannot find module '../build/Release/bson'] code: 'MODULE_NOT_FOUND' }
js-bson: Failed to load c++ bson extension, using pure JS version
{ [Error: Cannot find module '../build/Release/bson'] code: 'MODULE_NOT_FOUND' }
js-bson: Failed to load c++ bson extension, using pure JS version
{ [Error: Cannot find module '../build/Release/bson'] code: 'MODULE_NOT_FOUND' }
js-bson: Failed to load c++ bson extension, using pure JS version
{ [Error: Cannot find module '../build/Release/bson'] code: 'MODULE_NOT_FOUND' }
js-bson: Failed to load c++ bson extension, using pure JS version
```

Malheureusement, cette nouvelle installation fait planter le site au démarrage.



Après une recherche sur internet, j'ai trouvé une solution à ce problème. Les librairies nouvellement installées utilisent un plugin appelé Bson. Il s'agit d'une librairie permettant de manipuler des objets Json en binaire, ce qui rend les traitements plus efficaces. Dans sa version distribuée, Mongoose cherche la librairie Bson dans un dossier `build`. La librairie étant disponible dans le dossier courant, on change les liens tel que décrit sur cette image.



De plus, pour éviter un message d'erreur (sans impact) au déploiement du site, on ajoute cette autre modification : dans le fichier

```
\node_modules\mongoose\node_modules\mongodb\node_modules\mongodb-core\node_modules\bson\ext\index.js,
```

```
Utiliser bson = require('../lib/bson/bson');
```

```

C:\Program Files\nodejs\site_01\Site_01>npm start
> Site_01@0.0.0 start C:\Program Files\nodejs\site_01\Site_01
> node ./bin/www

```

On peut ainsi redémarrer le site sans erreur.

Par la suite, j'ai eu la regrettable surprise de découvrir que ce champ aurait dû être déclaré ainsi :

```
_id : mongoose.Schema.ObjectId
```

En base, j'avais des ObjectId qui étaient transformés en String dans l'application à cause de cette description. Lorsque je souhaitais faire des recherches par `_id`, il m'était impossible de récupérer les documents correspondants car la valeur recherchée était un String.

```

1  var express = require('express');
2  var path = require('path');
3  var favicon = require('serve-favicon');
4  var logger = require('morgan');
5  var cookieParser = require('cookie-parser');
6  var bodyParser = require('body-parser');
7
8  // Mon code
9  var mongoose = require('mongoose');
10 mongoose.connect('mongodb://localhost/test');
11
12 var ArbreSchema = new mongoose.Schema({
13   _id: String, type: String, id: String,
14   geometry: {coordinates: Array, type: String},
15   geometry_name: String,
16   properties: {
17     longitude: Number, diametre: String, age_estimation: String,
18     bbox: Array, fid: Number, lieu: String, hauteur: Number,
19     auteur: String, parcelle: Number, type: String, nom_commun: String,
20     nom_latin: String, latitude: Number, commentaire: String, id: Number,
21     origine: String
22   }
23 }, { collection: 'arbres3' });
24
25 mongoose.schemas = {
26   arbre: ArbreSchema
27 };
28 // Mon code
29
30 var routes = require('./routes/index');
31 var users = require('./routes/users');
32
33 var app = express();
34
35 // Mon code
36 // Rend la variable db accessible dans le router
37 app.use(function(req, res, next) {
38   req.mongoose = mongoose;
39   next();
40 });
41 // Mon code
42
43 // view engine setup
44 app.set('views', path.join(__dirname, 'views'));
45 app.set('view engine', 'jade');

```

Maintenant que Mongoose est disponible, on peut l'instancier avec la chaîne de connexion suivante : `mongodb://localhost/test`.

La grande particularité de Mongoose est sa déclaration obligatoire de schémas. Chaque document que l'on voudra manipuler devra avoir une description exhaustive de l'ensemble de ses champs et sous-champs. Dans notre exemple, on peut voir le schéma d'un arbre.

```

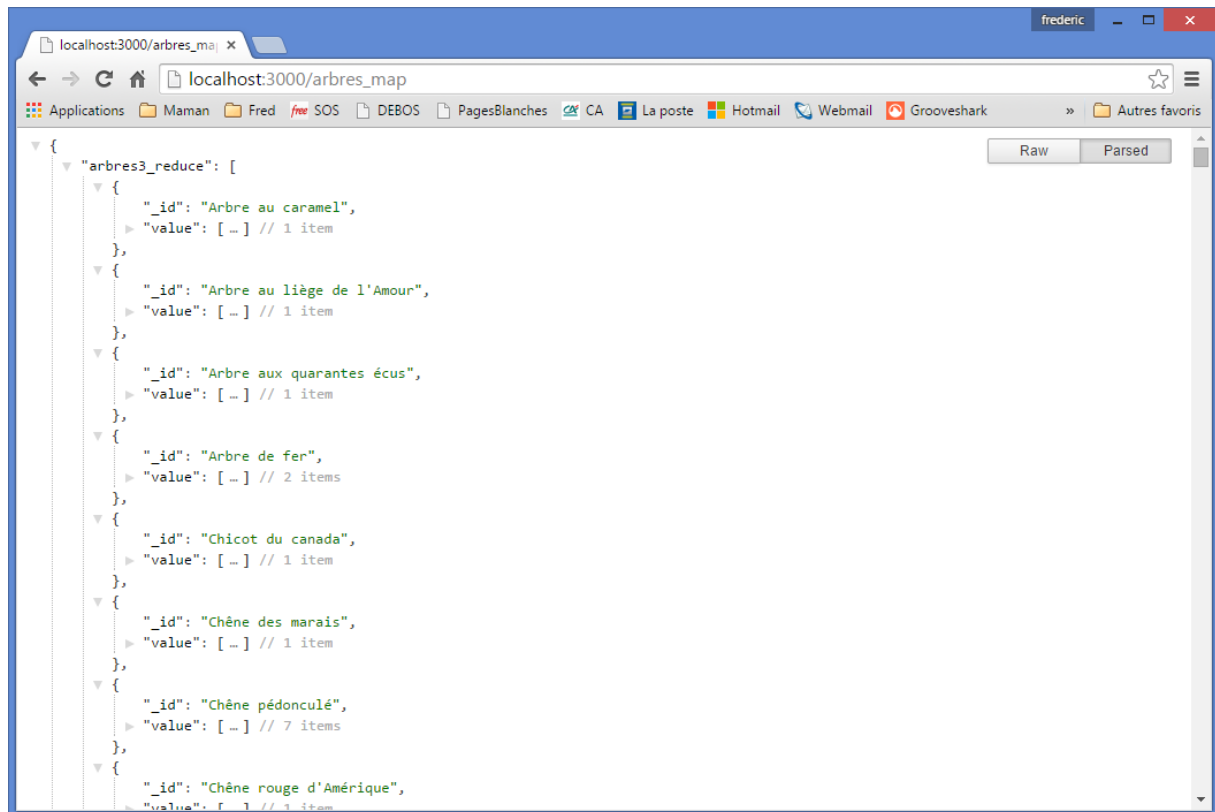
1  var express = require('express');
2  var router = express.Router();
3
4  /* GET home page. */
5  router.get('/', function(req, res, next) {
6    res.render('index', { title: 'Express' });
7  });
8
9  /* GET arbres page. */
10 router.get('/arbres_map', function(req, res) {
11
12     var mongoose = req.mongoose;
13     var db = mongoose.connection;
14
15     var arbre = db.model('Arbre', mongoose.schemas.arbre)
16
17     var o = {
18       map : function () {
19         emit(this.properties.nom_commun, this)
20       },
21       reduce : function (key, values) {
22         var reduced = {key:key, values:values};
23         return reduced;
24       },
25       finalize : function(key, reduced){
26         var finalized = reduced;
27         if (reduced._id) { // passage pour les données non groupées
28           finalized = [reduced];
29         } else { // passage pour les données traitées par Reduce
30           finalized = reduced.values;
31         }
32         return finalized;
33       }
34     };
35     arbre.mapReduce(o, function (e, docs) {
36       // le résultat est retourné au format json
37       res.writeHead(200, {"Content-Type": "application/json; charset=utf-8"});
38
39       var json = JSON.stringify({ arbres3_reduce: docs });
40       res.end(json);
41     });
42
43
44     module.exports = router;
45

```

Dans la page de routage, on récupère le schéma décrit précédemment. On l'utilise pour instancier un objet métier que l'on nomme `arbre` et qui nous permettra de réaliser toutes les manipulations sur la base.

On commence par décrire l'ensemble des opérations contenues dans la procédure de MapReduce. On retrouve les méthodes `map`, `reduce` et `finalize` tel qu'attendues dans MongoDB. On renseigne ces fonctions comme on l'avait fait dans MongoVue.

L'étape suivante consiste à exécuter ces opérations. Pour cela, l'objet model `arbre` de Mongoose est doté notamment de la fonction `mapReduce()`. Elle attend l'objet contenant les opérations de MapReduce et retourne, en paramètres du callback, la liste des documents retournés par MongoDB. On sérialise ces documents et on les écrit dans la réponse sous forme de flux Json.



```
{
  "arbres3_reduce": [
    {
      "_id": "Arbre au caramel",
      "value": [...] // 1 item
    },
    {
      "_id": "Arbre au liège de l'Amour",
      "value": [...] // 1 item
    },
    {
      "_id": "Arbre aux quarantes écus",
      "value": [...] // 1 item
    },
    {
      "_id": "Arbre de fer",
      "value": [...] // 2 items
    },
    {
      "_id": "Chicot du canada",
      "value": [...] // 1 item
    },
    {
      "_id": "Chêne des marais",
      "value": [...] // 1 item
    },
    {
      "_id": "Chêne pédonculé",
      "value": [...] // 7 items
    },
    {
      "_id": "Chêne rouge d'Amérique",
      "value": [...] // 1 item
    }
  ]
}
```

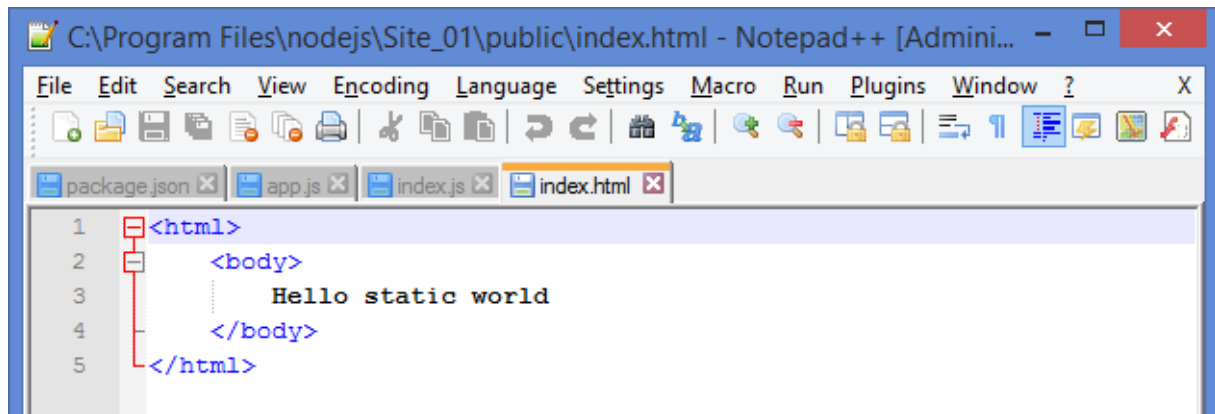
Nous avons cette fois-ci fait un mapping sur les noms communs des arbres. Nous les retrouvons donc groupés selon une clé qui correspond à leur nom.

Application web

Nous avons maintenant les capacités de réaliser tous les outils nécessaires à notre application du côté du serveur. Nous allons nous occuper de la partie client.

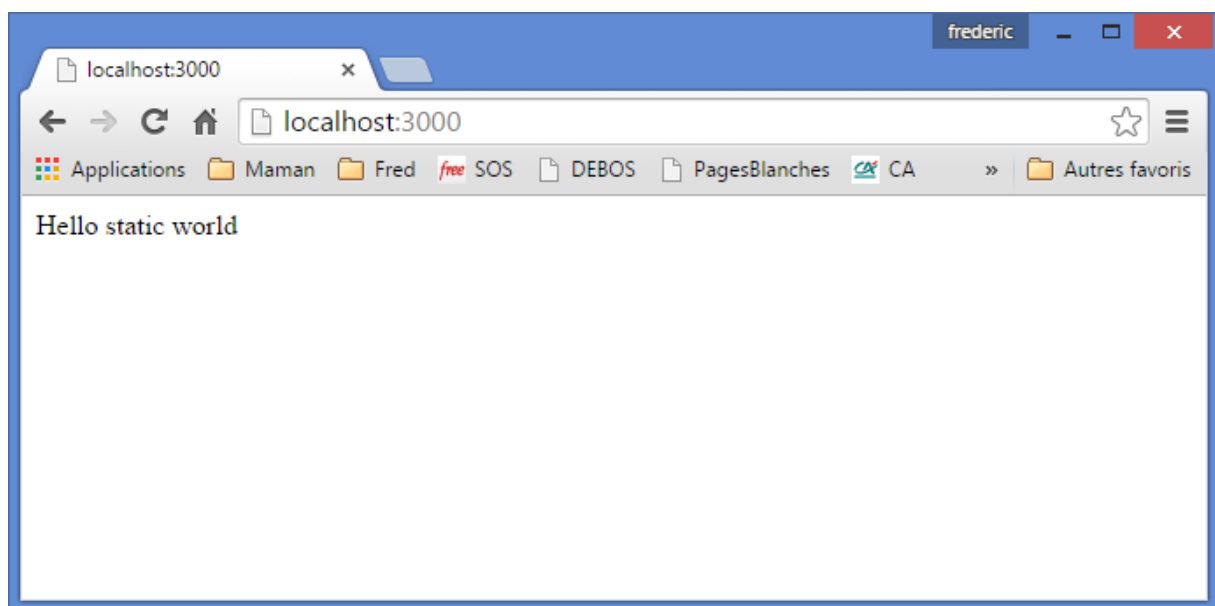
Le développement avec le langage Jade n'étant pas vraiment mon fort, je m'oriente vers un langage plus traditionnel : HTML.

NodeJS offre la possibilité de déployer des pages qui seront servies sans analyse de l'interpréteur. On saisit cette opportunité pour créer une page html que l'on dépose dans le dossier `public` du site.

A screenshot of the Notepad++ application window. The title bar shows the file path: C:\Program Files\nodejs\Site_01\public\index.html - Notepad++ [Admini...]. The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Macro, Run, Plugins, Window, and ?. The toolbar contains various icons for file operations. The tab bar shows four open files: package.json, app.js, index.js, and index.html. The main text area displays the following HTML code:

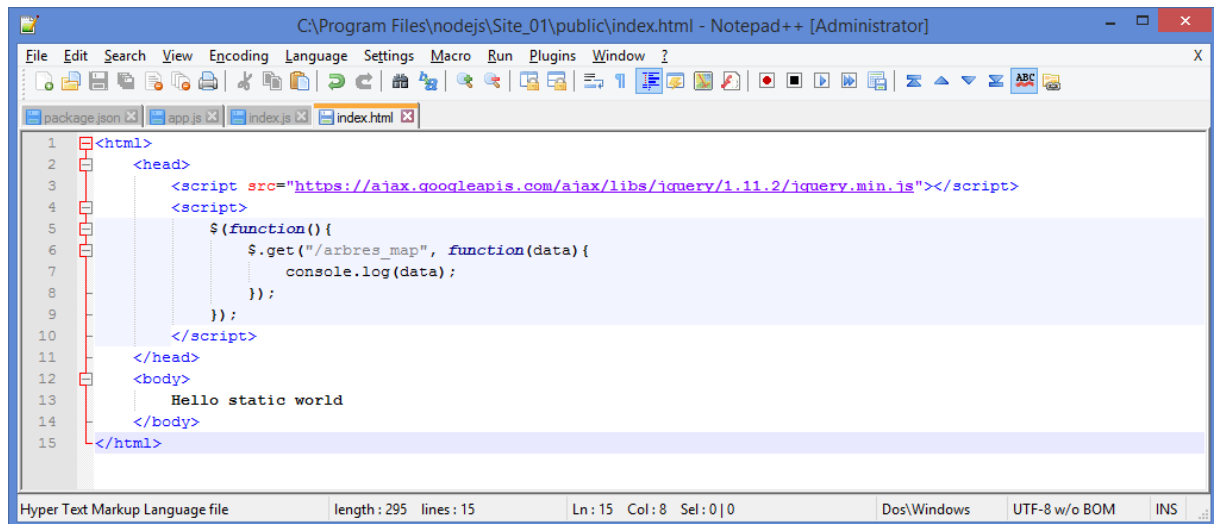
```
1 <html>
2   <body>
3     Hello static world
4   </body>
5 </html>
```

Je réalise une première page extrêmement simple pour vérifier le fonctionnement de ce répertoire.



Et tout fonctionne comme prévu. L'URL du domaine retourne la page index.html qui a été définie comme page par défaut dans les configurations d'ExpressJS.

Pour la suite, on va faire appel à des bibliothèques JavaScript distribuées gratuitement. Nous n'aurons qu'à les référencer dans notre page pour les rendre disponible dans notre code.



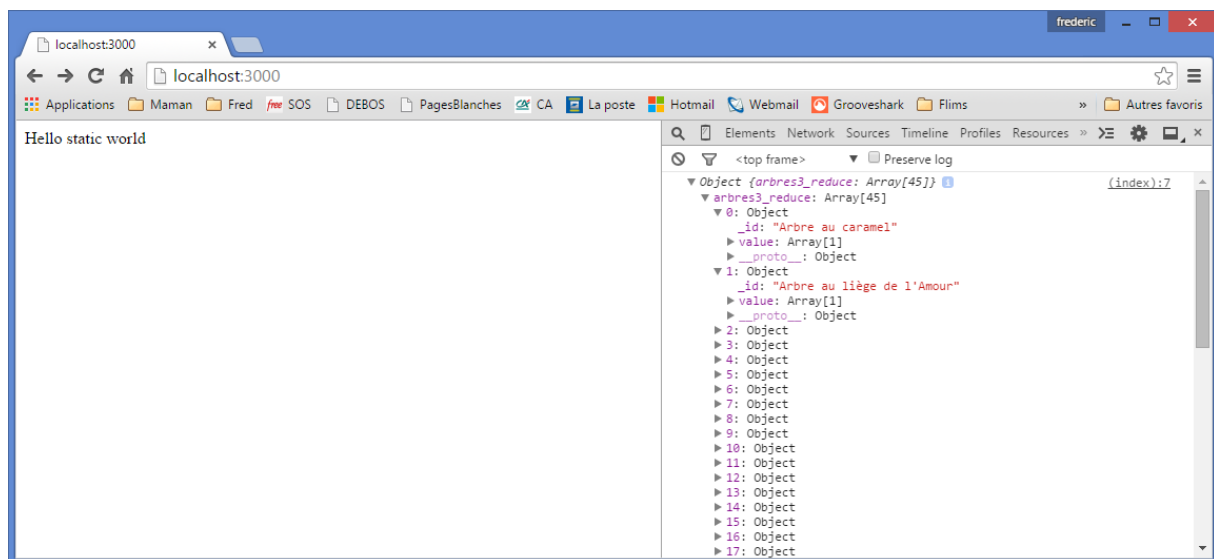
```

1 <html>
2 <head>
3   <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js"></script>
4   <script>
5     $(function(){
6       $.get("/arbres_map", function(data){
7         console.log(data);
8       });
9     });
10  </script>
11 </head>
12 <body>
13   Hello static world
14 </body>
15 </html>

```

Pour simplifier les appels Ajax et la manipulation du DOM de notre application, je décide d'utiliser JQuery. Google propose un lien permanent par lequel nous pouvons accéder à cette API sans avoir à l'héberger.

La première chose que je tente est un appel à la ressource Json que j'ai développé. Pour rappel, elle est disponible par méthode GET à l'URL `/arbres_map`.



En actualisant la page et en affichant la console du navigateur, on retrouve la collection au format Json telle qu'elle a été retournée par MongoDB.

L'étape suivante consiste à afficher la cartographie car, notre objectif est de géolocaliser les arbres présent dans notre base. Plusieurs API peuvent convenir à notre projet : Google Map, Bing Map, OpenStreetMap... Sans réelle critère départageant, je choisis de continuer avec Google Map.

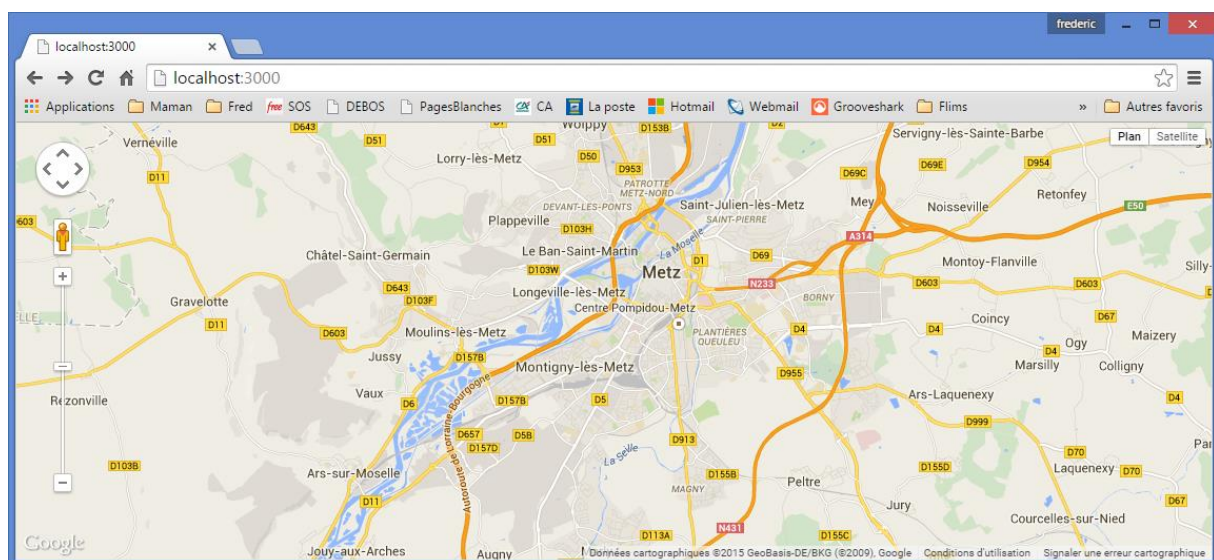
Sur mon compte Google, je commence par activer l'utilisation de l'API. J'obtiens alors une clé m'offrant le droit d'utiliser le service.

```

1 <html>
2 <head>
3   <script type="text/javascript" src="https://maps.googleapis.com/maps/api/js?key=AIzaSyADaXC1tIHdpI
4   <script type="text/javascript">
5     function initialize() {
6       var mapOptions = {
7         center: { lat: 49.10462564968479, lng: 6.160435854000001 },
8         zoom: 12
9       };
10      var map = new google.maps.Map(document.getElementById('map-canvas'),
11        mapOptions);
12    }
13    google.maps.event.addDomListener(window, 'load', initialize);
14  </script>
15  <style type="text/css">
16    html, body, #map-canvas { height: 100%; margin: 0; padding: 0; }
17  </style>
18 </head>
19 <body>
20   <div id="map-canvas"></div>
21 </body>
22 </html>

```

Ensuite, j'intègre l'API dans la page en indiquant la clé délivrée. Dans mon `body`, je n'inclue qu'une balise `div` qui contiendra la carte. J'ajoute quelques lignes en JavaScript pour attraper l'évènement de chargement de page et lancer la création de la carte. Pour créer cette carte, je me contente d'indiquer la position et le zoom par défaut. A l'instanciation, j'indique la `div` qui servira de conteneur. Pour finir, quelques lignes de CSS permettent d'avoir la carte en plein page dans le navigateur.



Ce code, aussi simple soit-il, permet un affichage de la carte. On va réaliser à présent un menu à gauche listant les noms d'arbres. Ces noms seront des liens qui permettront l'affichage des arbres concernés sur la carte.

```
<html>
  <head>
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js"></script>
    <script type="text/javascript" src="https://maps.googleapis.com/maps/api/js?key=AIzaSyADaXCltIHdpUYTcYs33SNCvp0IpYE17F4">
  </script>
    <script type="text/javascript">
      // Déclaration de variables globales
      var types_arbres = null;
      var map = null;
      var cercles = new Array();

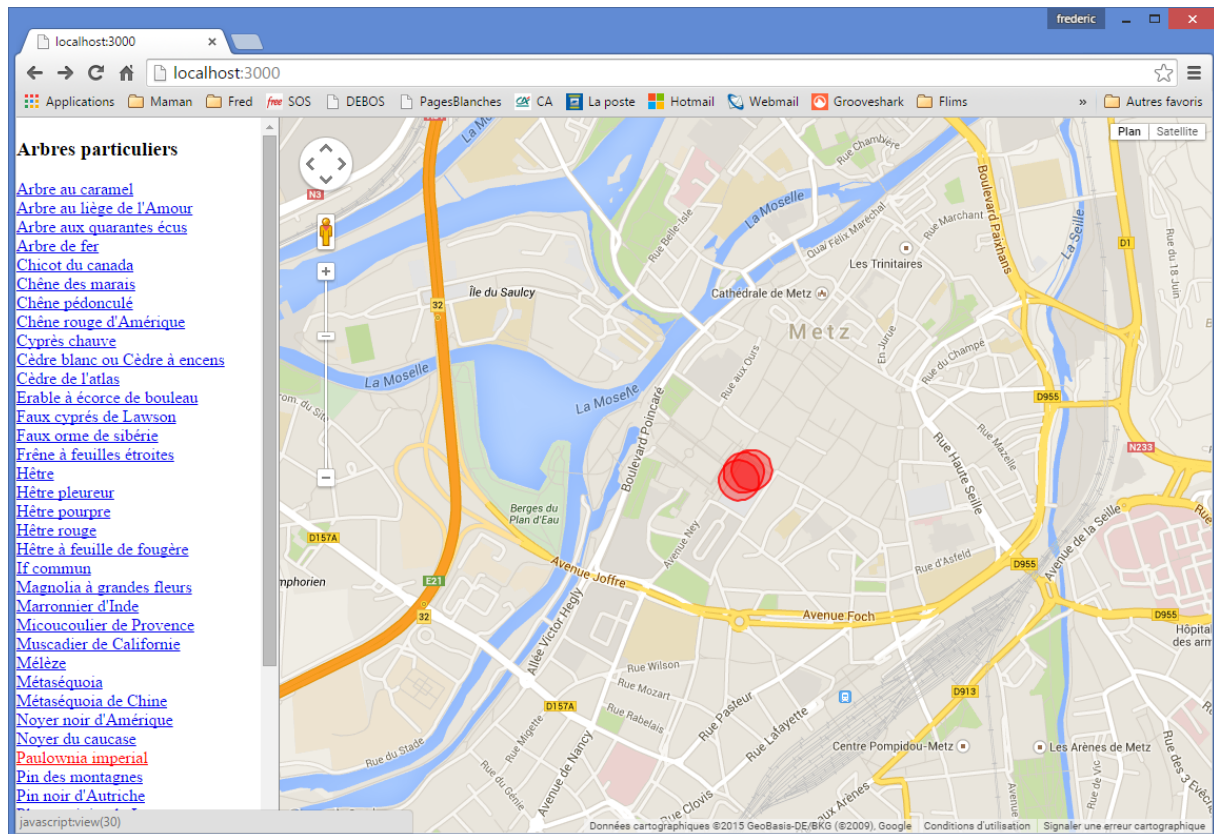
      // fonction d'initialisation de la page
      function initialize() {
        // Instanciation de la carte
        map = new google.maps.Map(document.getElementById('map-canvas'), {
          center : { lat: 49.10462564968479, lng: 6.160435854000001 },
          zoom : 13
        });
        // fin de l'initialisation de la carte
        $(function(){
          // requête pour récupérer la liste des arbres
          $.get("/arbres_map",function(data) {
            // On affiche cette liste dans la console pour le débogage
            console.log(data);
            types_arbres = data.arbres3_reduce;
            // pour chaque élément, on fait un nouveau lien dans le menu
            for (var idx in types_arbres) {
              var item = types_arbres[idx];
              // pour chaque élément, on crée une balise A qui appellera la fonction view
              $("#menu").append(
                "<a href='javascript:view(" + idx + ")'>" + item._id + "</a><br/>"
              );
            }
          });
        });
        // Récupère l'évènement onload de la page pour lancer l'initialisation
        google.maps.event.addDomListener(window, 'load', initialize);

        // Fonction lancée lorsque l'on choisit une option dans le menu
        function view(idx) {
          // On commence par supprimer les cercles présents sur la map
          for (var i in cercles) {
            cercles[i].setMap(null);
```

```

    }
    cercles = new Array();
    // On recherche les coordonnées min et max pour centrer la carte à la fin
    var maxLat=0, minLat=99999;
    var maxLong=0, minLong=99999;
    for (i in types_arbres[idx].value) {
        var arbre = types_arbres[idx].value[i];
        var latitude = arbre.properties.latitude;
        var longitude = arbre.properties.longitude;
        // Pour chaque arbre, on va créer un nouveau cercle rouge sur la carte
        cercles.push(new google.maps.Circle( {
            strokeColor : '#FF0000', strokeOpacity: 0.8, strokeWeight: 2,
            fillColor : '#FF0000', fillOpacity: 0.35, radius: 60,
            map : map,
            center : new google.maps.LatLng(latitude, longitude)
        } ));
        // Avant de passer à l'élément suivant, on met à jour les valeurs min et max des coordonnées
        if (maxLat < latitude) maxLat = latitude;
        if (minLat > latitude) minLat = latitude;
        if (maxLong < longitude) maxLong = longitude;
        if (minLong > longitude) minLong = longitude;
    }
    // Finalement, on zoom sur les coordonnées récupérées.
    var bounds = new google.maps.LatLngBounds(
        new google.maps.LatLng(minLat-0.005, minLong-0.005),
        new google.maps.LatLng(maxLat+0.005, maxLong+0.005)
    );
    map.fitBounds(bounds);
    map.panToBounds(bounds);
}
</script>
<style type="text/css">
    html, body, body>div { height: 100%; margin: 0; padding: 0; }
    #menu { position: absolute; width:249px; border-right:1px solid #888; overflow-y:scroll; }
    #map-canvas { width:calc(100% - 250px); left:250px; }
</style>
</head>
<body>
    <div id="menu">
        <h3>Arbres particuliers</h3>
    </div>
    <div id="map-canvas"></div>
</body>
</html>

```



La liste des arbres est récupérée à l'initialisation de la page. A la réception de cette liste, on crée le menu de gauche en créant un lien sur avec chaque nom. Ce lien appelle une fonction à laquelle on passe l'identifiant de liste correspondant. Comme la liste est conservée dans les variables globales, on peut récupérer l'ensemble des documents correspondant et on affiche un cercle sur chaque position que l'on trouve. Pour un confort d'utilisation, on zoom sur la zone correspondant à la position des cercles.

Je parlais d'Ajax tout à l'heure. Il est vrai que la récupération des données provenant de la base utilise Ajax. Cependant, il serait plus propre de ne pas charger la totalité de la base et d'exécuter une requête uniquement sur les informations qui nous intéressent. Aussi, pour réaliser ce fractionnement d'information, il va falloir modifier le code coté serveur.

On profite de cette prochaine étape pour changer le regroupement des données. On trie les documents par hauteur cette fois-ci.

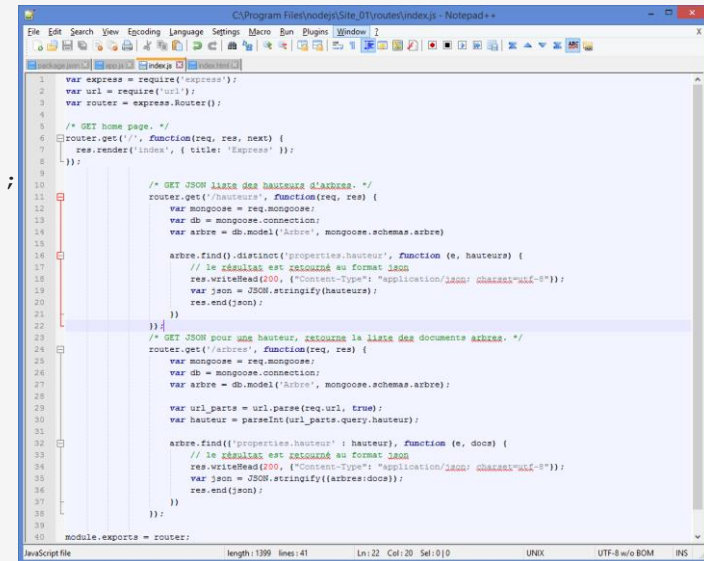
Code coté serveur dans le fichier `/routes/index.js`

```

/* GET JSON liste des hauteurs d'arbres. */
router.get('/hauteurs', function(req, res) {
  var mongoose = req.mongoose;
  var db = mongoose.connection;
  var arbre = db.model('Arbre', mongoose.schemas.arbre)
  // recherche des hauteurs sans doublon
  arbre.find().distinct('properties.hauteur', function (e, hauteurs) {
    // le résultat est retourné au format json
    res.writeHead(200, {"Content-Type": "application/json; charset=utf-8"});
    var json = JSON.stringify(hauteurs);
    res.end(json);
  })
});

/* GET JSON pour une hauteur, retourne la liste des documents arbres. */
router.get('/arbres', function(req, res) {
  var mongoose = req.mongoose;
  var db = mongoose.connection;
  var arbre = db.model('Arbre', mongoose.schemas.arbre);
  // récupération de la hauteur transmise dans l'URL
  var url_parts = url.parse(req.url, true);
  var hauteur = parseInt(url_parts.query.hauteur);
  // recherche des documents correspondant à la hauteur
  arbre.find({'properties.hauteur' : hauteur}, function (e, docs) {
    // le résultat est retourné au format json
    res.writeHead(200, {"Content-Type": "application/json; charset=utf-8"});
    var json = JSON.stringify({arbres:docs});
    res.end(json);
  })
});

```



Code coté client dans le fichier `/public/index.html`

```

<html>
  <head>
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js"></script>
    <script type="text/javascript" src="https://maps.googleapis.com/maps/api/js?key=AIzaSyADaXC1tIHdpUYTcYs33SNCvp0IpYE17F4">
  </script>
    <script type="text/javascript">
      var map = null;
      var cercles = new Array();

      function initialize() {

```

```
map = new google.maps.Map(document.getElementById('map-canvas'), {
  center: { lat: 49.10462564968479, lng: 6.160435854000001 },
  zoom: 13
});

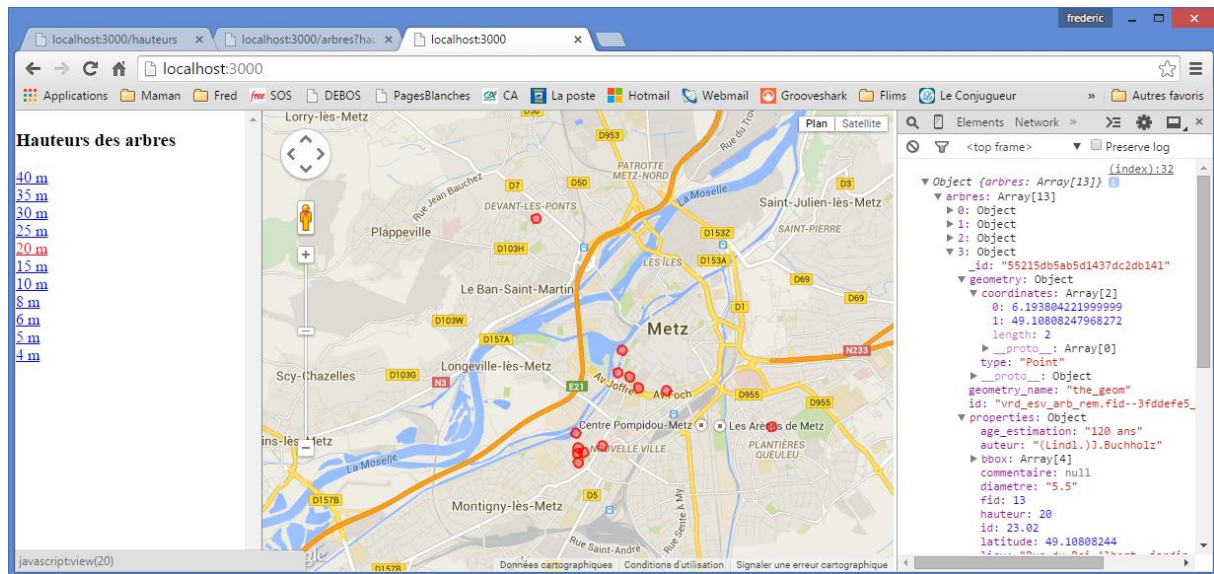
$(function() {
  // l'URL d'initialisation du menu est maintenant /hauteurs
  $.get("/hauteurs", function(data) {
    // le distinct de mongoose n'a pas permis de trier les données : on le fait ici
    data = data.sort(function(a, b) { return b - a });
    for(var idx in data) {
      // on affiche maintenant des hauteurs dans les liens
      var hauteur = data[idx];
      $("#menu").append(
        "<a href='javascript:view(" + hauteur + ")'>" + hauteur + " m</a><br/>"
      );
    }
  });
});

google.maps.event.addDomListener(window, 'load', initialize);

function view(hauteur) {
  // Au lieu d'afficher directement les données,
  // on fait une requête pour récupérer les documents qui nous intéressent.
  // L'URL permettant d'obtenir les documents demande une hauteur en paramètre.
  $.get("/arbres?hauteur=" + hauteur, function(docs) {
    // la suite n'a pas changé
    console.log(docs);
    for(var i in cercles) {
      cercles[i].setMap(null);
    }
    cercles = new Array();
    var maxLat=0, minLat=99999;
    var maxLong=0, minLong=99999;
    for(var idx in docs.arbres) {
      var doc_arbre = docs.arbres[idx];
      var latitude = doc_arbre.geometry.coordinates[1];
      var longitude = doc_arbre.geometry.coordinates[0];
      cercles.push(new google.maps.Circle({
        strokeColor: '#FF0000', strokeOpacity: 0.8, strokeWeight: 2,
        fillColor: '#FF0000', fillOpacity: 0.35, radius: 60,
        map: map,
        center: new google.maps.LatLng(latitude, longitude)
      }));
    }
  });
}
```



```
        ));  
        if (maxLat < latitude) maxLat = latitude;  
        if (minLat > latitude) minLat = latitude;  
        if (maxLong < longitude) maxLong = longitude;  
        if (minLong > longitude) minLong = longitude;  
    }  
    var bounds = new google.maps.LatLngBounds(  
        new google.maps.LatLng(minLat-0.005, minLong-0.005),  
        new google.maps.LatLng(maxLat+0.005, maxLong+0.005)  
    );  
    map.fitBounds(bounds);  
    map.panToBounds(bounds);  
});  
}  
</script>  
<style type="text/css">  
    html, body, body>div { height: 100%; margin: 0; padding: 0; }  
    #menu { position: absolute; width:249px; border-right:1px solid #888; overflow-y:scroll; }  
    #map-canvas { width:calc(100% - 250px); left:250px; }  
</style>  
</head>  
<body>  
    <div id="menu">  
        <h3>Hauteurs des arbres</h3>  
    </div>  
    <div id="map-canvas"></div>  
</body>  
</html>
```



Voici le résultat que l'on obtient en exécutant ce code. On a à gauche la liste des hauteurs comme menu. En choisissant une hauteur, on interroge le serveur qui nous retourne la liste des documents correspondant à la hauteur sélectionnée. A la réception de ces documents, on les affiche dans la carte.

Pour aller encore plus loin, on va enrichir l'interface pour que l'on puisse :

- éditer ou ajouter un document à travers un formulaire
- supprimer un document sélectionné.

Cette partie étant plus chargée en développement coté client, je m'aide de l'API JQuery UI pour une mise en forme plus élaborée du menu et l'implémentation d'un pop-up contenant le formulaire de saisie. Je ne m'attarde pas trop sur cet aspect du projet qui est quelque peu en dehors du domaine d'étude mais sachez que le code est disponible en annexe.

Le plus important est de décrire les actions coté serveur car c'est ici que se trouve les interactions avec la base MongoDB. Pour réaliser ces nouvelles fonctionnalités, il nous faut :

- une URL permettant de supprimer un document à partir de son identifiant
- une URL sur laquelle on pourra transmettre un document que l'on veut persister.

Code coté serveur dans le fichier `/routes/index.js`

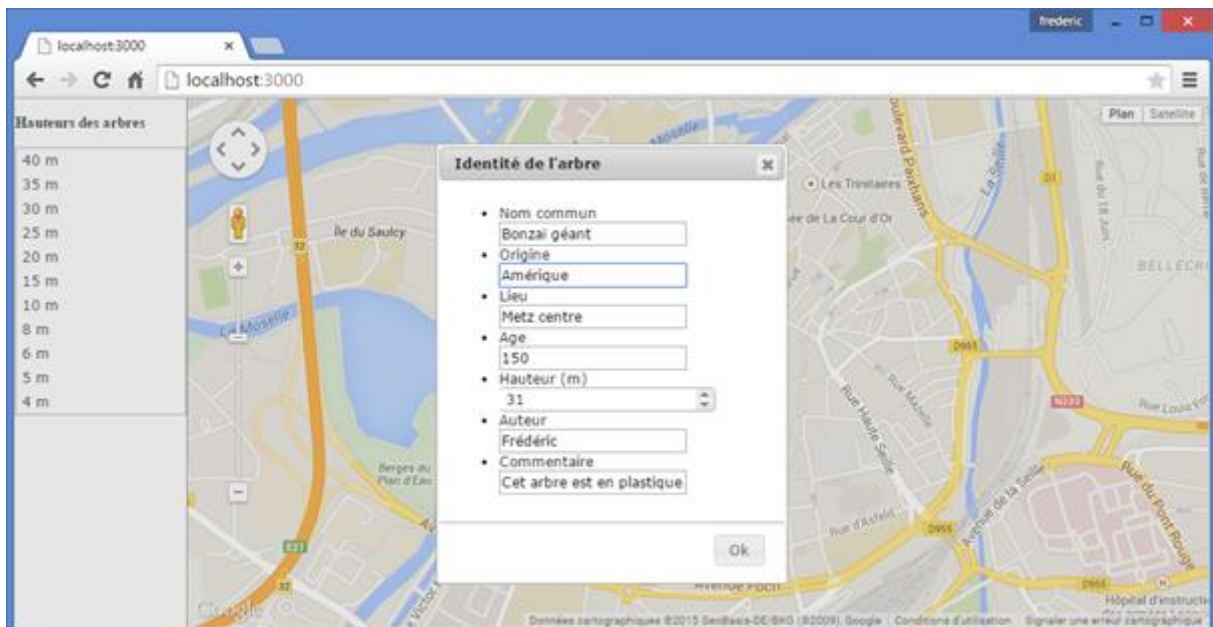
```
/* POST delete */
router.post('/delete', function(req, res) {
  // on récupère l'objet posté et on en extrait l'id correspondant
  var json_arbre = JSON.parse(req.body.arbre);
  var arbre_id = mongoose.Types.ObjectId(json_arbre._id);
  // ensuite, on recherche cet objet et on le supprime
  SchemaArbre.find({ id:arbre_id}).remove(function(err) {
    // on retourne ensuite un message de réussite ou d'échec
    if(err) console.log(err);
    res.end(err?"ko":"ok");
  });
});
```

```

/* POST save */
router.post('/save', function(req, res) {
  // on récupère l'objet posté et on en extrait l'id correspondant
  var json_arbre = JSON.parse(req.body.arbre);
  var arbre_id = mongoose.Types.ObjectId(json_arbre._id);
  // on commence par rechercher un document portant cet id
  SchemaArbre.findOne({_id: arbre_id}, function(e, doc){
    // s'il n'y a pas de correspondance, c'est qu'il faut insérer le document
    if (doc == null) {
      // on crée donc un nouveau document arbre avec un nouvel id
      var doc_arbre = new SchemaArbre(json_arbre);
      doc_arbre._id = mongoose.Types.ObjectId();
      // et on le sauvegarde
      doc_arbre.save(function(err){
        // on retourne ensuite un message de réussite ou d'échec
        if(err) console.log(err);
        res.end(err?"ko":"ok");
      });
    }
    // sinon, il s'agit d'une mise à jour d'un document existant
    else {
      // il faut alors faire une « recherche - mise à jour » en passant les
      // champs qu'il faut modifier
      SchemaArbre.findOneAndUpdate({_id: arbre_id}, json_arbre, function(err){
        // on retourne ensuite un message de réussite ou d'échec
        if(err) console.log(err);
        res.end(err?"ko":"ok");
      });
    }
  });
});
});

```

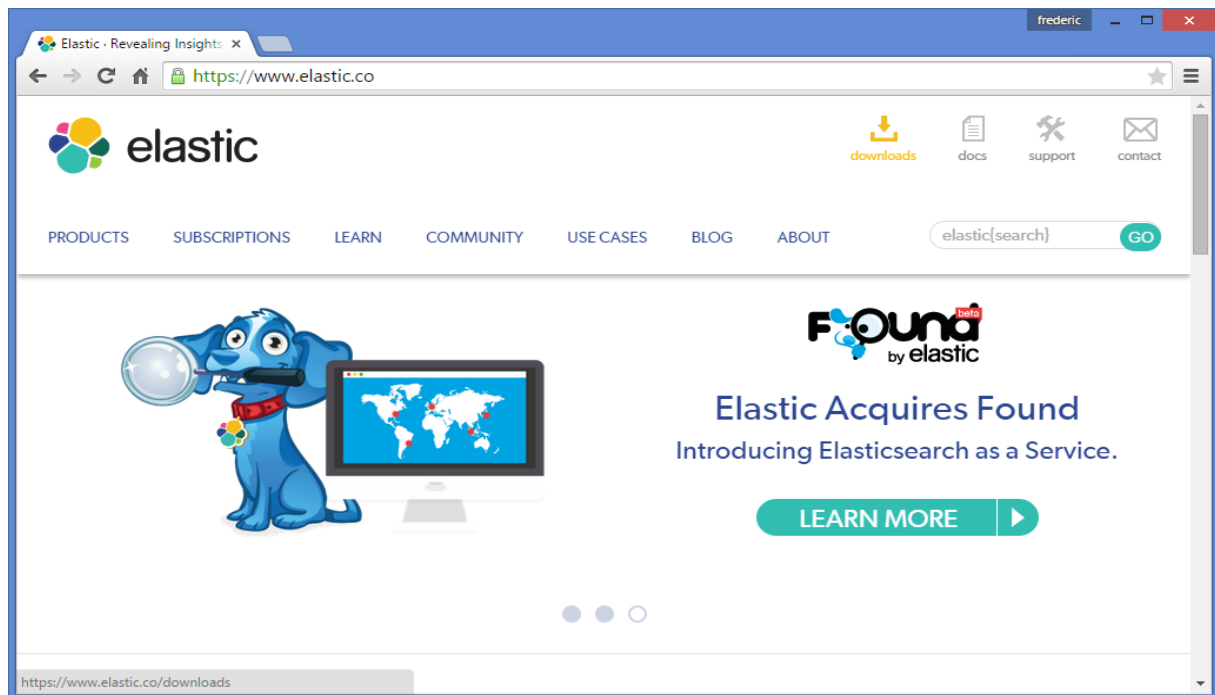
Avec ces deux nouvelles fonctions, on dispose des éléments nécessaires pour réaliser une application à même d'éditer les documents.



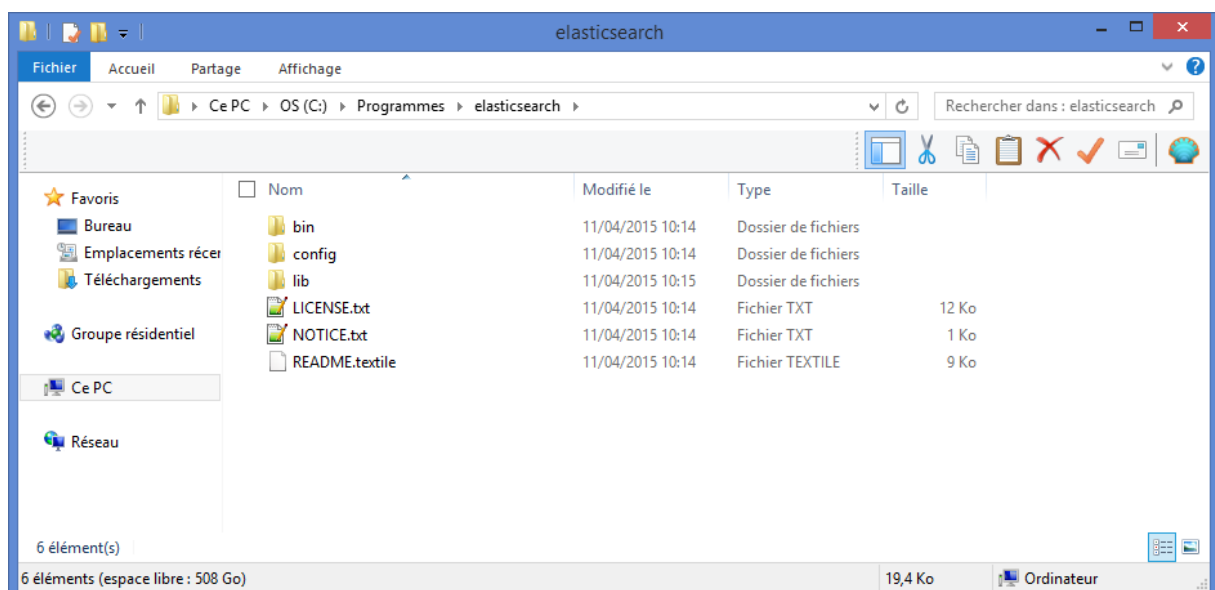
Et voici à quoi ressemble l'interface avec le nouveau menu et le formulaire de saisie.

Installation d'ElasticSearch

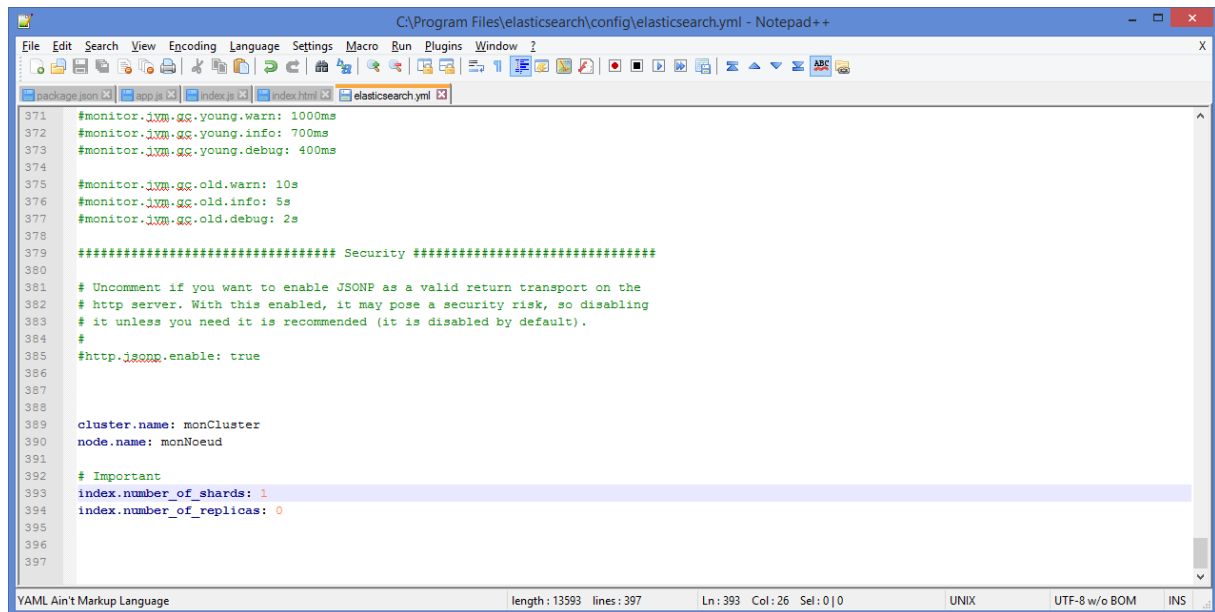
On passe maintenant à une partie qui sera proche de ce qui est décrit dans le cours, à savoir, l'installation d'ElasticSearch. Nous allons utiliser ElasticSearch pour faire des recherches plein texte sur nos documents. L'objectif sera toujours d'afficher sur la carte les arbres correspondant aux documents. On proposera ainsi un autre moyen de sélection à l'utilisateur : celui du moteur de recherche.



La première étape est de se rendre sur le site d'ElasticSearch : www.elastic.co



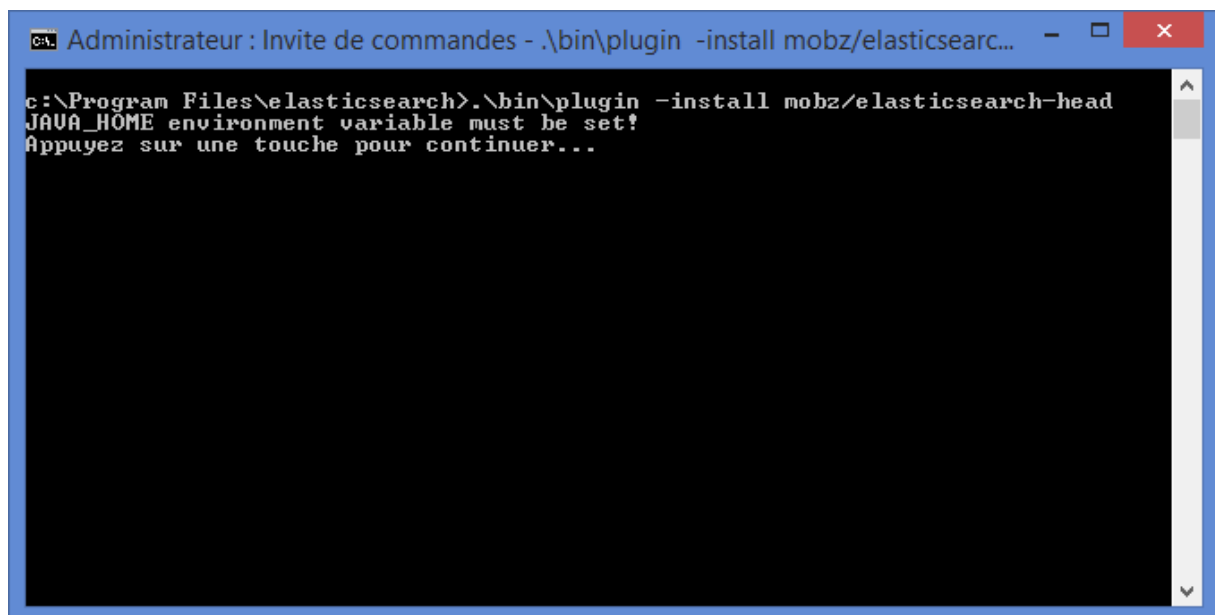
Ensuite, on télécharge l'archive que l'on décompresse dans Program Files.



The screenshot shows a Notepad++ window titled "C:\Program Files\elasticsearch\config\elasticsearch.yml - Notepad++". The file is a YAML configuration for Elasticsearch. The visible content includes monitoring settings for JVM GC, security warnings, and cluster/node names. The cluster name is set to "monCluster" and the node name is "monNoeud". The number of shards is set to 1 and the number of replicas is 0. The status bar at the bottom indicates the file is a YAML Ain't Markup Language, 13593 characters long, 397 lines, and is in UTF-8 w/o BOM encoding.

```
371 #monitor.jvm.gc.young.warn: 1000ms
372 #monitor.jvm.gc.young.info: 700ms
373 #monitor.jvm.gc.young.debug: 400ms
374
375 #monitor.jvm.gc.old.warn: 10s
376 #monitor.jvm.gc.old.info: 5s
377 #monitor.jvm.gc.old.debug: 2s
378
379 ##### Security #####
380
381 # Uncomment if you want to enable JSONP as a valid return transport on the
382 # http server. With this enabled, it may pose a security risk, so disabling
383 # it unless you need it is recommended (it is disabled by default).
384 #
385 #http.jsonp.enable: true
386
387
388
389 cluster.name: monCluster
390 node.name: monNoeud
391
392 # Important
393 index.number_of_shards: 1
394 index.number_of_replicas: 0
395
396
397
```

Comme vu en cours, on modifie le nom du cluster et du nœud. On définit également le nombre de fragments et de réplicas.

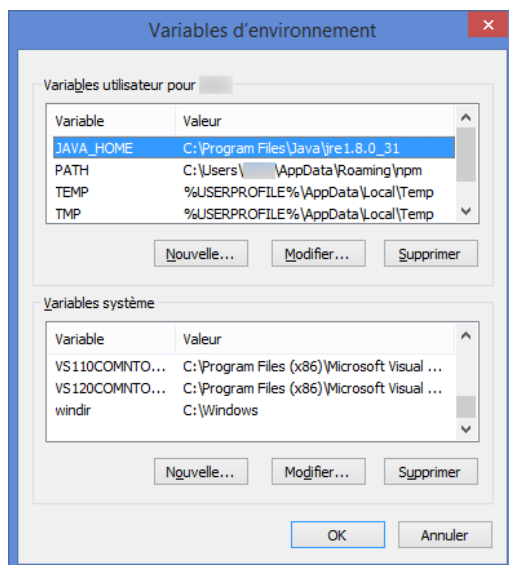
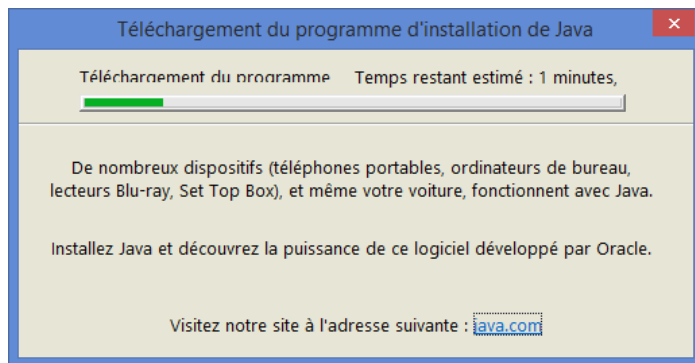


The screenshot shows a Windows command prompt window titled "C:\> Administrateur : Invite de commandes - \.bin\plugin -install mobz/elasticsearc...". The prompt is at "c:\Program Files\elasticsearch>". The user has entered the command ".bin\plugin -install mobz/elasticsearch-head". The output shows an error: "JAVA_HOME environment variable must be set!" followed by "Appuyez sur une touche pour continuer...".

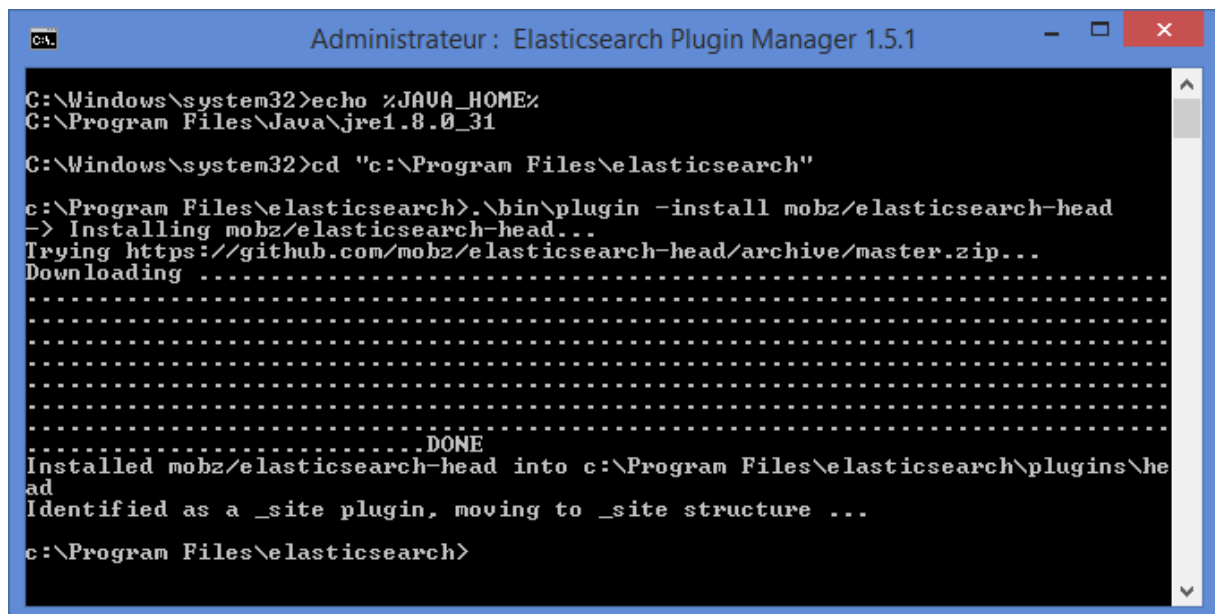
```
C:\> Administrateur : Invite de commandes - \.bin\plugin -install mobz/elasticsearc...
c:\Program Files\elasticsearch>.bin\plugin -install mobz/elasticsearch-head
JAVA_HOME environment variable must be set!
Appuyez sur une touche pour continuer...
```

On lance ensuite l'installation d'ElasticSearch Head qui est une interface web d'administration.

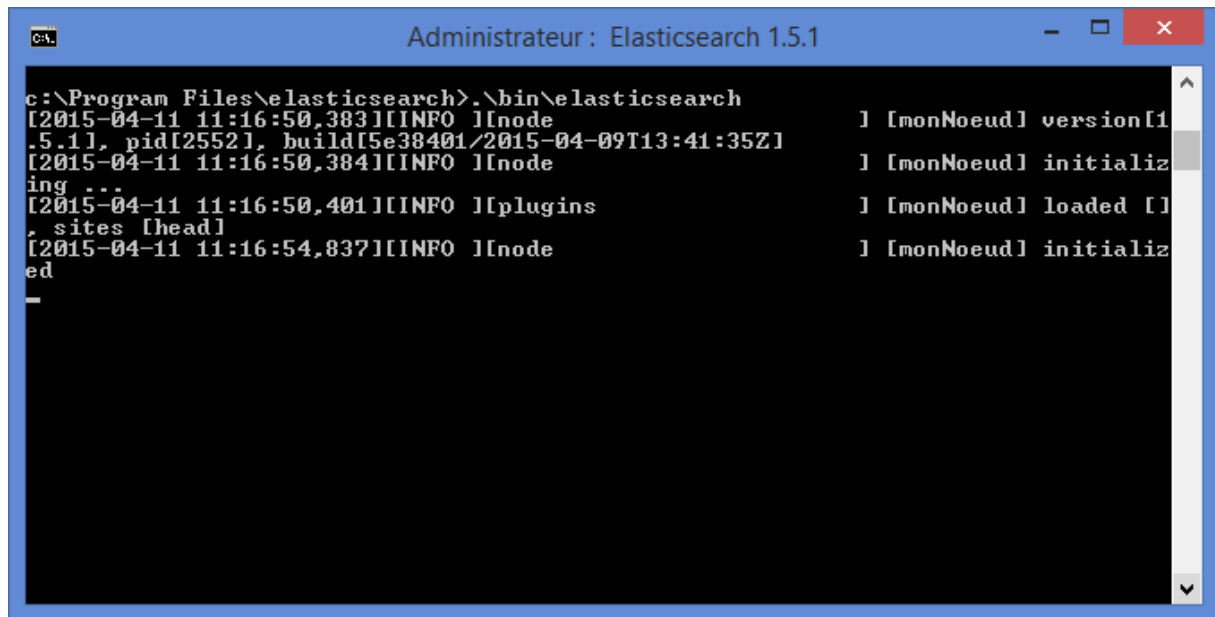
Mais il faut d'abord que je définisse la variable JAVA_HOME dans mon environnement.



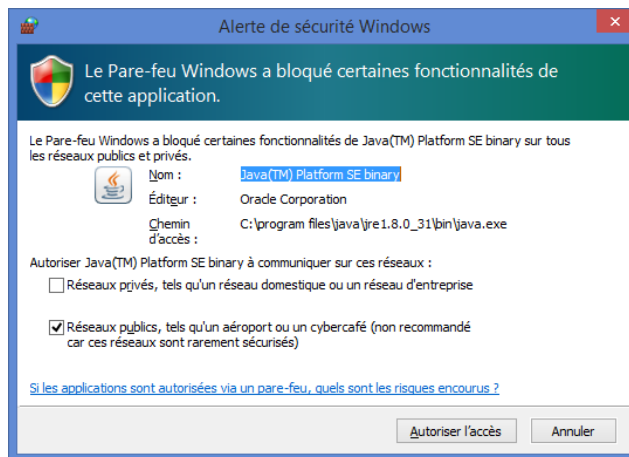
Dans le doute, je mets à jour ma JRE et j'ajoute la variable demandée dans les variables d'environnement de mon compte utilisateur.



Après avoir redémarré l'invite de commande pour que la variable soit prise en compte, je relance la même commande d'installation et, cette fois-ci, tout fonctionne normalement.

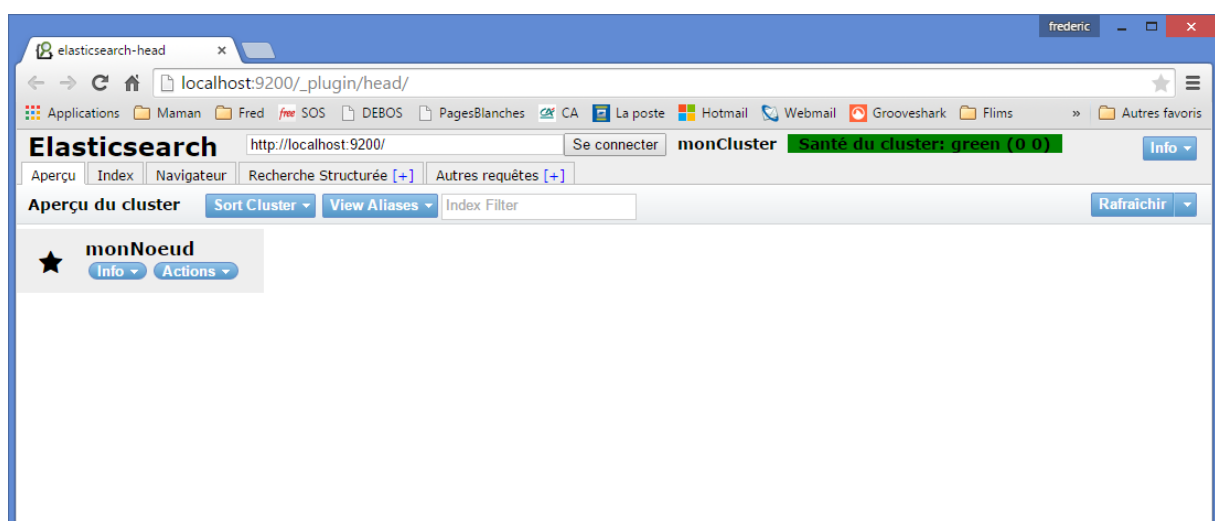


```
C:\Program Files\elasticsearch>.\\bin\\elasticsearch
[2015-04-11 11:16:50,383][INFO ][node
.5.1], pid[2552], build[5e38401/2015-04-09T13:41:35Z]
[2015-04-11 11:16:50,384][INFO ][node
ing ...
[2015-04-11 11:16:50,401][INFO ][plugins
, sites [head]
[2015-04-11 11:16:54,837][INFO ][node
ed
```



Je lance ensuite le serveur.

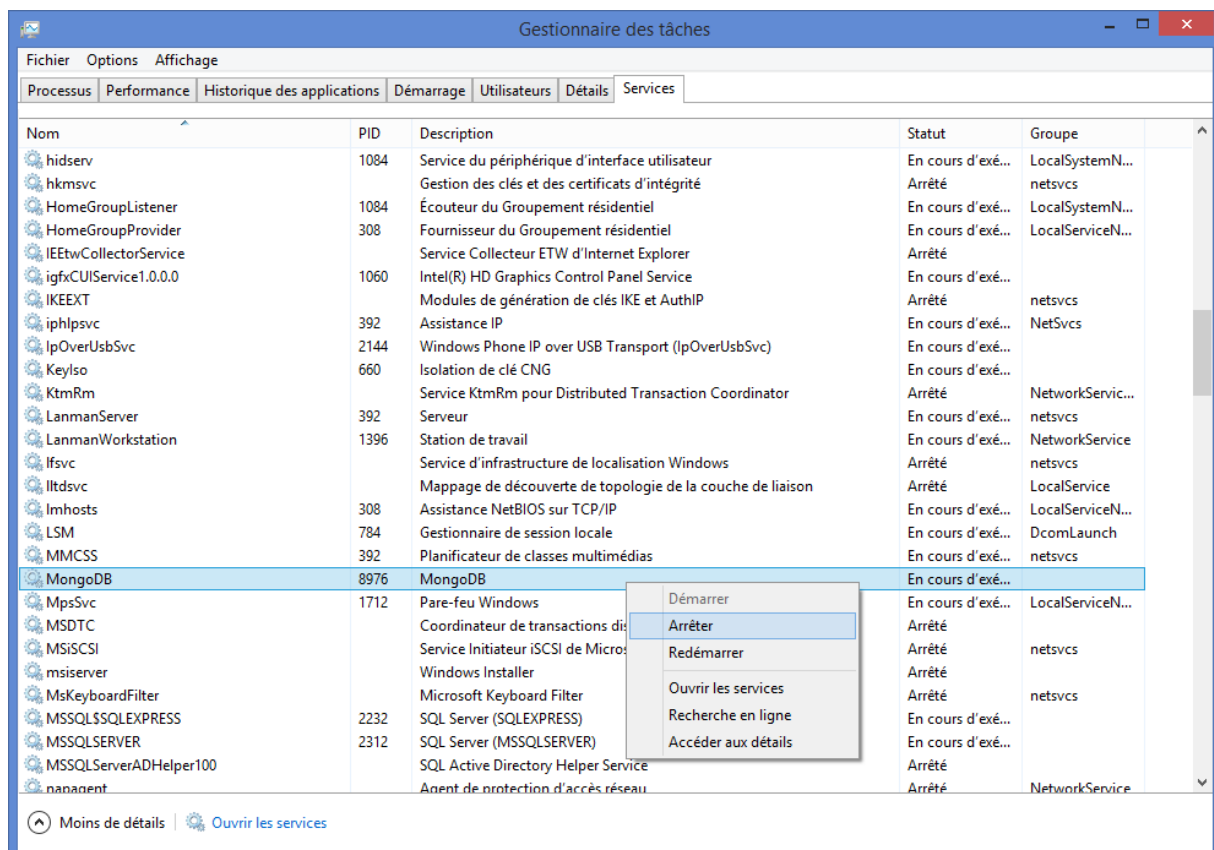
Celui-ci requière l'utilisation de Java et cela nécessite une autorisation dans le pare-feu.



Mais une fois que tout est bien lancé, on peut accéder à l'interface web que voici. Cela prouve que notre serveur fonctionne correctement.

[illegible]

On stoppe ensuite ce serveur pour installer le plugin River telle que cela a été décrit dans le cours. Cela permettra à Elasticsearch de se synchroniser automatiquement avec les modifications qui auront lieu sur la base de données MongoDB.



On arrête d'abord le serveur MongoDB car il va falloir ajouter la fonctionnalité de réplication au démarrage de celui-ci.


```

Administrateur : Elasticsearch Plugin Manager 1.5.1 - mongod --port 27017 --dbpath C:\MongoDB\Server\3.0\bdd\ --replSet rs0

c:\MongoDB\Server\3.0\bin>mongod --port 27017 --dbpath C:\MongoDB\Server\3.0\bdd\ --replSet rs0
2015-04-11T11:59:19.998+0200 I JOURNAL [initandlisten] journal dir=C:\MongoDB\Server\3.0\bdd\journal
2015-04-11T11:59:19.999+0200 I JOURNAL [initandlisten] recover: no journal files present, no recovery needed
2015-04-11T11:59:20.093+0200 I JOURNAL [durability] Durability thread started
2015-04-11T11:59:20.093+0200 I JOURNAL [journal writer] Journal writer thread started
2015-04-11T11:59:20.098+0200 I CONTROL [initandlisten] MongoDB starting : pid=4300 port=27017 dbpath=C:\MongoDB\Server\3.0\bdd\ 6
4-bit host=TDE_DEU01
2015-04-11T11:59:20.098+0200 I CONTROL [initandlisten] targetMinOS: Windows 7/Windows Server 2008 R2
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] db version v3.0.1
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] git version: 534b5a3f9d10f00cd27737fbc951032248b5952
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] OpenSSL version: OpenSSL 1.0.1j-fips 15 Oct 2014
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] build info: windows sys.getwindowsversion(major=6, minor=1, build=7601, pl
atform=2, service_pack='Service Pack 1') BOOST_LIB_VERSION=1_49
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] allocator: system
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] options: { net: { port: 27017 }, replication: { replSet: "rs0" }, storage:
{ dbPath: "C:\MongoDB\Server\3.0\bdd\" } }
2015-04-11T11:59:20.213+0200 I NETWORK [initandlisten] waiting for connections on port 27017
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] New replica set config in use: { _id: "rs0", version: 1, members: [
{ _id: 0, host: "TDE_DEU01:27017", arbiterOnly: false, buildIndexes: true, hidden: false, priority: 1.0, tags: {}, slaveDelay: 0,
votes: 1 } ], settings: { chainingAllowed: true, heartbeatTimeoutSecs: 10, getLastErrorModes: {}, getLastErrorDefaults: { w: 1, wt
imeout: 0 } } }
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] This node is TDE_DEU01:27017 in the config
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] transition to STARTUP2
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] Starting replication applier threads
2015-04-11T11:59:20.219+0200 I REPL [ReplicationExecutor] transition to RECOVERING
2015-04-11T11:59:20.220+0200 I REPL [ReplicationExecutor] transition to SECONDARY
2015-04-11T11:59:20.220+0200 I REPL [ReplicationExecutor] transition to PRIMARY
2015-04-11T11:59:21.221+0200 I REPL [rsSync] transition to primary complete; database writes are now permitted
2015-04-11T11:59:23.554+0200 I NETWORK [initandlisten] connection accepted from 127.0.0.1:60274 #1 (1 connection now open)
2015-04-11T11:59:23.556+0200 I NETWORK [conn1] end connection 127.0.0.1:60274 (0 connections now open)
2015-04-11T11:59:33.556+0200 I NETWORK [initandlisten] connection accepted from 127.0.0.1:60275 #2 (1 connection now open)

```

On initialise ce mode réplca avec la commande `--replSet rs0`. Pour être sûr que le serveur travaille bien sur notre base, on spécifie son chemin grâce au paramètre `--dbpath`.

```

Administrateur : Invite de commandes - mongo

c:\MongoDB\Server\3.0\bin>mongo
MongoDB shell version: 3.0.1
connecting to: test
rs0:PRIMARY> rs.initiate()
{
  "info2" : "no configuration explicitly specified -- making one",
  "ne" : "TDE_DEU01:27017",
  "info" : "try querying local.system.replset to see current configuration",
  "ok" : 0,
  "errmsg" : "already initialized",
  "code" : 23
}
rs0:PRIMARY>

```

Pour finir, dans une autre invite de commande, on ouvre un client MongoDB et on initialise le mode réplca.

```

Administrateur : Elasticsearch Plugin Manager 1.5.1 - mongod --port 27017 --dbpath C:\MongoDB\Server\3.0\bdd\ --replSet rs0

2015-04-11T11:59:20.093+0200 I JOURNAL [journal writer] Journal writer thread started
2015-04-11T11:59:20.098+0200 I CONTROL [initandlisten] MongoDB starting : pid=4300 port=27017 dbpath=C:\MongoDB\Server\3.0\bdd\ 6
4-bit host=TDE_DEU01
2015-04-11T11:59:20.098+0200 I CONTROL [initandlisten] targetMinOS: Windows 7/Windows Server 2008 R2
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] db version v3.0.1
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] git version: 534b5a3f9d10f00cd27737fbc951032248b5952
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] OpenSSL version: OpenSSL 1.0.1j-fips 15 Oct 2014
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] build info: windows sys.getwindowsversion(major=6, minor=1, build=7601, pl
atform=2, service_pack='Service Pack 1') BOOST_LIB_VERSION=1_49
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] allocator: system
2015-04-11T11:59:20.099+0200 I CONTROL [initandlisten] options: { net: { port: 27017 }, replication: { replSet: "rs0" }, storage:
{ dbPath: "C:\MongoDB\Server\3.0\bdd\" } }
2015-04-11T11:59:20.213+0200 I NETWORK [initandlisten] waiting for connections on port 27017
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] New replica set config in use: { _id: "rs0", version: 1, members: [
{ _id: 0, host: "TDE_DEU01:27017", arbiterOnly: false, buildIndexes: true, hidden: false, priority: 1.0, tags: {}, slaveDelay: 0,
votes: 1 } ], settings: { chainingAllowed: true, heartbeatTimeoutSecs: 10, getLastErrorModes: {}, getLastErrorDefaults: { w: 1, wt
imeout: 0 } } }
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] This node is TDE_DEU01:27017 in the config
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] transition to STARTUP2
2015-04-11T11:59:20.218+0200 I REPL [ReplicationExecutor] Starting replication applier threads
2015-04-11T11:59:20.219+0200 I REPL [ReplicationExecutor] transition to RECOVERING
2015-04-11T11:59:20.220+0200 I REPL [ReplicationExecutor] transition to SECONDARY
2015-04-11T11:59:20.220+0200 I REPL [ReplicationExecutor] transition to PRIMARY
2015-04-11T11:59:21.221+0200 I REPL [rsSync] transition to primary complete; database writes are now permitted
2015-04-11T11:59:23.554+0200 I NETWORK [initandlisten] connection accepted from 127.0.0.1:60274 #1 (1 connection now open)
2015-04-11T11:59:23.556+0200 I NETWORK [conn1] end connection 127.0.0.1:60274 (0 connections now open)
2015-04-11T11:59:33.556+0200 I NETWORK [initandlisten] connection accepted from 127.0.0.1:60275 #2 (1 connection now open)
2015-04-11T12:00:29.137+0200 I NETWORK [initandlisten] connection accepted from 127.0.0.1:60276 #3 (2 connections now open)
2015-04-11T12:00:40.942+0200 I COMMAND [conn3] replSet info initiate : no configuration specified. Using a default configuration
for the set
2015-04-11T12:00:40.943+0200 I COMMAND [conn3] replSet created this configuration for initiation : { _id: "rs0", version: 1, memb
ers: [ { _id: 0, host: "TDE_DEU01:27017" } ] }
2015-04-11T12:00:40.943+0200 I REPL [conn3] replSetInitiate admin command received from client

```

On peut voir sur la fenêtre du serveur qu'il a bien réagit à la demande d'initialisation.

```

Administrateur : Elasticsearch 1.5.1

c:\Program Files\elasticsearch\bin>elasticsearch
[2015-04-11 12:04:33.355][INFO] lnode
[2015-04-11 12:04:33.356][INFO] lnode
[2015-04-11 12:04:33.371][INFO] lplugins
[2015-04-11 12:04:36.418][INFO] lnode
[2015-04-11 12:04:37.847][INFO] lnode
[2015-04-11 12:04:38.359][INFO] ltransport
[2015-04-11 12:04:39.135][INFO] l[discovery]
[2015-04-11 12:04:42.905][INFO] lcluster.service
[2015-04-11 12:04:43.162][INFO] l[http]
[2015-04-11 12:04:43.162][INFO] lnode

[monNoeud] version[1.5.1], pid[10192], build[5e38401/2015-04-09T13:41:35Z]
[monNoeud] initializing ...
[monNoeud] loaded [mongodb-river], sites [head, river-mongodb]
[monNoeud] initialized
[monNoeud] starting ...
[monNoeud] bound_address <inet[/0:0:0:0:0:0:9300]/>, publish_address <inet[/192.168.0.13:9300]/>
[monNoeud] monCluster/zcEy0ceETp2NQvk5Kua5AA
[monNoeud] new_master [monNoeud]zcEy0ceETp2NQvk5Kua5AA1ITDE_DEV01[inet[/192.168.0.13:9300]], reason: zen-disco-join <elected_as_master>
[monNoeud] recovered [0] indices into cluster_state
[monNoeud] bound_address <inet[/0:0:0:0:0:0:9200]/>, publish_address <inet[/192.168.0.13:9200]/>
[monNoeud] started

```

On peut ensuite relancer le serveur ElasticSearch.

```

C:\Program Files\elasticsearch\config\mongo-river.json - Notepad++

{
  "type": "mongodb",
  "mongodb": {
    "db": "test",
    "collection": "arbres3"
  },
  "index": {
    "name": "idx_test",
    "type": "idx_arbres3"
  }
}

```

Toujours en suivant les étapes décrites dans le cours, on crée un fichier de paramétrage de la river.

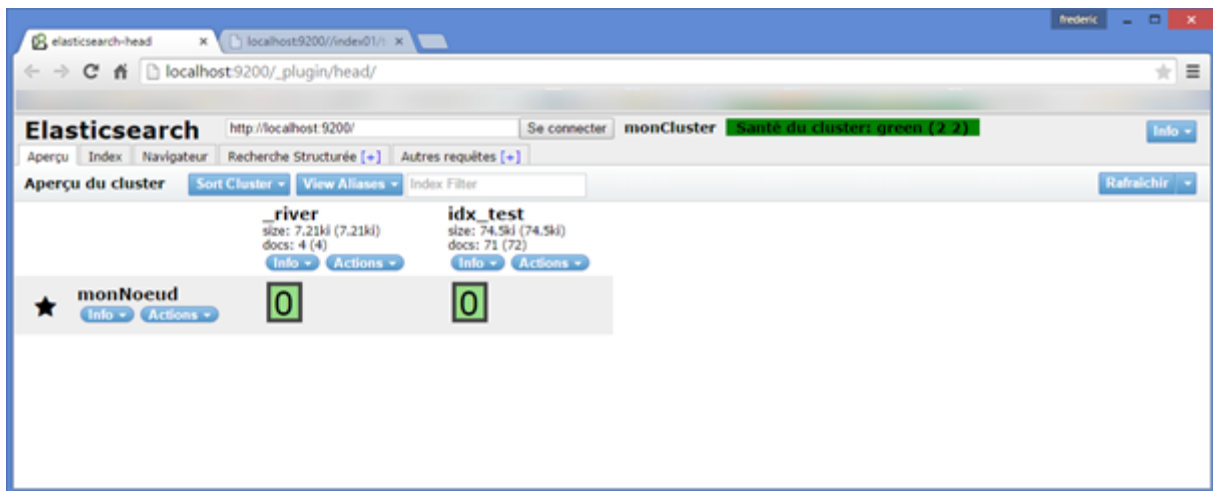
```

Administrateur : Invite de commandes

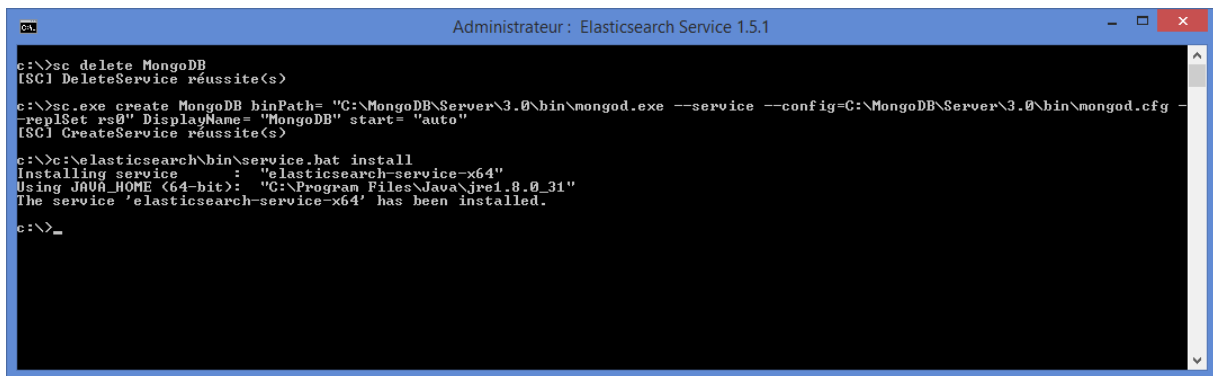
c:\Program Files>curl -X PUT http://localhost:9200/_river/mongodb/_meta -d @C:\Program Files\elasticsearch\config\mongo-river.json
{"_index":"_river","_type":"mongodb","_id":"_meta","_version":1,"created":true}
c:\Program Files>

```

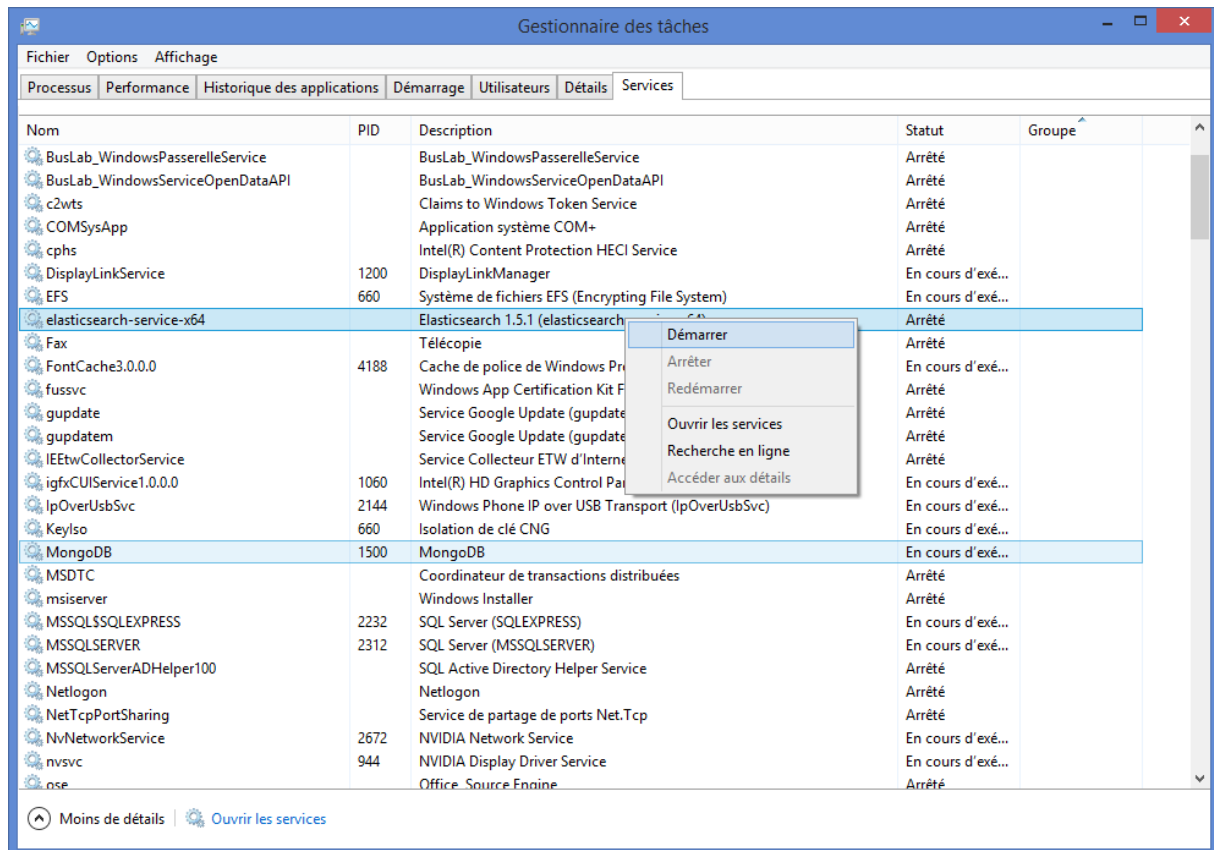
Il servira à la création de l'index `_river`. Pour cette commande, on utilise cURL. Dans le fichier de configuration, on a déclaré que l'on souhaitait faire un index nommé `idx_test` qui sera l'image de la base `test` de MongoDB.



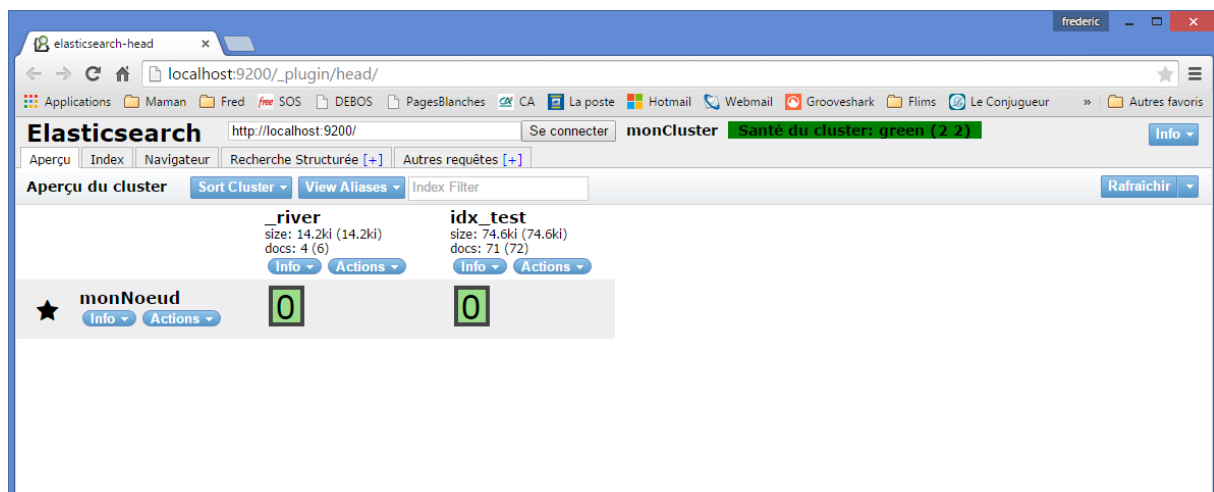
C'est ce que l'on retrouve en rafraichissant l'interface Head.



Le fonctionnement automatique en service était intéressant. On met donc à jour le service MongoDB pour ajouter la commande de réplica. On crée également un nouveau service pour lancer ElasticSearch.



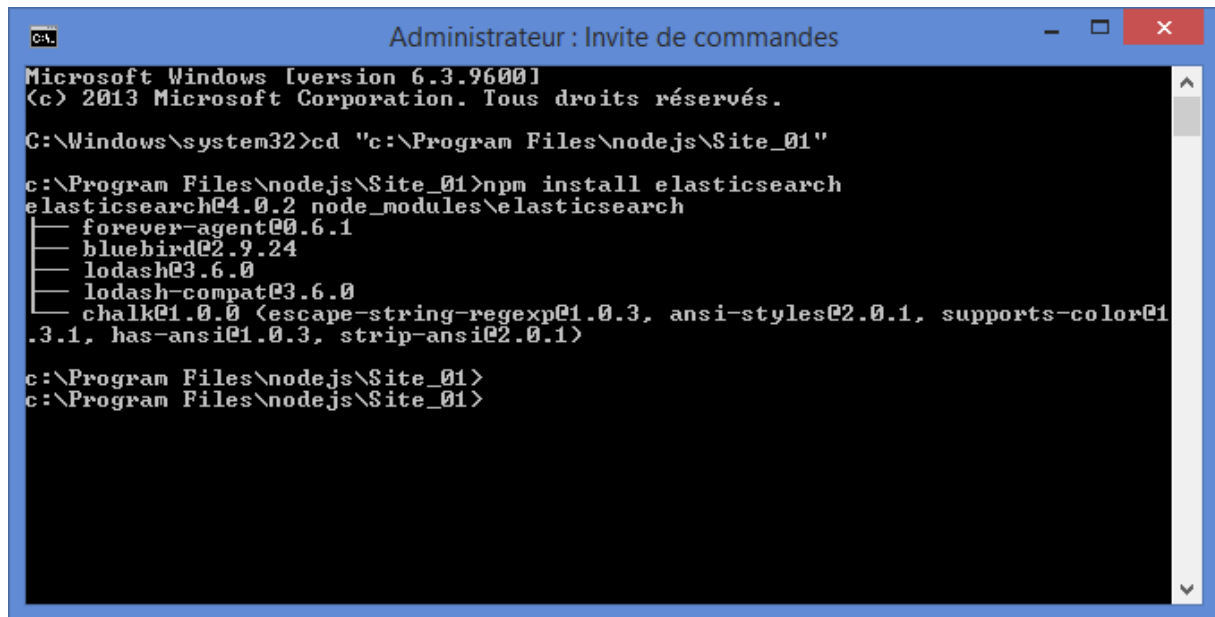
On démarre ces deux services en ayant pris soin d'arrêter les serveurs ouverts dans les invites de commande.



On vérifie que tout fonctionne en actualisant les interfaces. Et tout est bon.

ElasticSearch dans NodeJS

Maintenant que le serveur ElasticSearch est disponible dans notre environnement, nous allons voir comment l'interroger dans notre application.



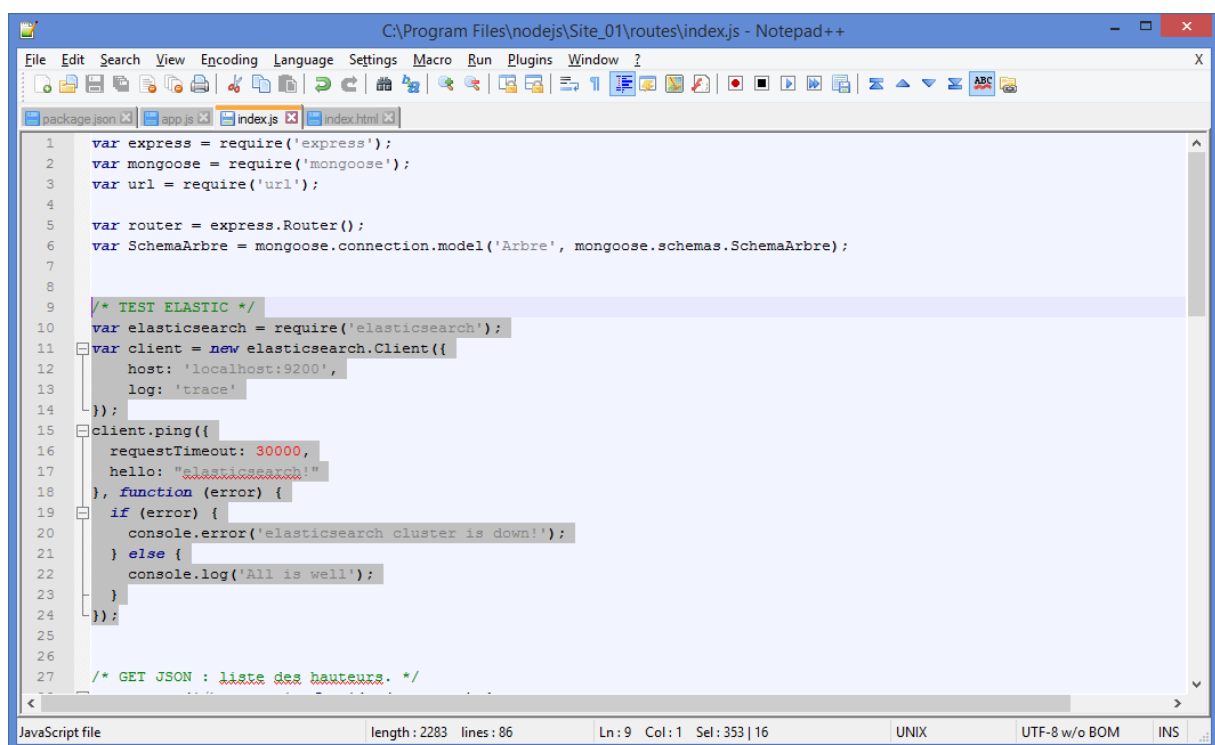
```
Administrateur : Invite de commandes

Microsoft Windows [version 6.3.9600]
(c) 2013 Microsoft Corporation. Tous droits réservés.

C:\Windows\system32>cd "c:\Program Files\nodejs\Site_01"

c:\Program Files\nodejs\Site_01>npm install elasticsearch
elasticsearch@4.0.2 node_modules\elasticsearch
├── forever-agent@0.6.1
├── bluebird@2.9.24
├── lodash@3.6.0
├── lodash-compat@3.6.0
├── chalk@1.0.0 (escape-string-regexp@1.0.3, ansi-styles@2.0.1, supports-color@1
├── .3.1, has-ansi@1.0.3, strip-ansi@2.0.1)
└──
c:\Program Files\nodejs\Site_01>
c:\Program Files\nodejs\Site_01>
```

On commence par installer le plugin d'ElasticSearch pour NodeJS.



```
C:\Program Files\nodejs\Site_01\routes\index.js - Notepad++

File Edit Search View Encoding Language Settings Macro Run Plugins Window ?

package.json app.js index.js index.html
1 var express = require('express');
2 var mongoose = require('mongoose');
3 var url = require('url');
4
5 var router = express.Router();
6 var SchemaArbre = mongoose.connection.model('Arbre', mongoose.schemas.SchemaArbre);
7
8
9 /* TEST ELASTIC */
10 var elasticsearch = require('elasticsearch');
11 var client = new elasticsearch.Client({
12   host: 'localhost:9200',
13   log: 'trace'
14 });
15 client.ping({
16   requestTimeout: 30000,
17   hello: 'elasticsearch!'
18 }, function (error) {
19   if (error) {
20     console.error('elasticsearch cluster is down!');
21   } else {
22     console.log('All is well!');
23   }
24 });
25
26
27 /* GET JSON : liste des hauteurs. */
```

Le code ci-dessus est fourni sur le site d'ElasticSearch et permet d'exécuter une requête dans le but de contrôler la présence du serveur.

```

c:\Program Files\nodejs\Site_01>npm start
> Site_01@0.0.0 start c:\Program Files\nodejs\Site_01
> node ./bin/www

Elasticsearch INFO: 2015-04-11T13:40:32Z
Adding connection to http://localhost:9200/

Elasticsearch DEBUG: 2015-04-11T13:40:32Z
starting request { method: 'HEAD',
  requestTimeout: 30000,
  castExists: true,
  path: '/',
  query: { hello: 'elasticsearch!' } }

Elasticsearch TRACE: 2015-04-11T13:40:32Z
-> HEAD http://localhost:9200/?hello=elasticsearch!

<- 200

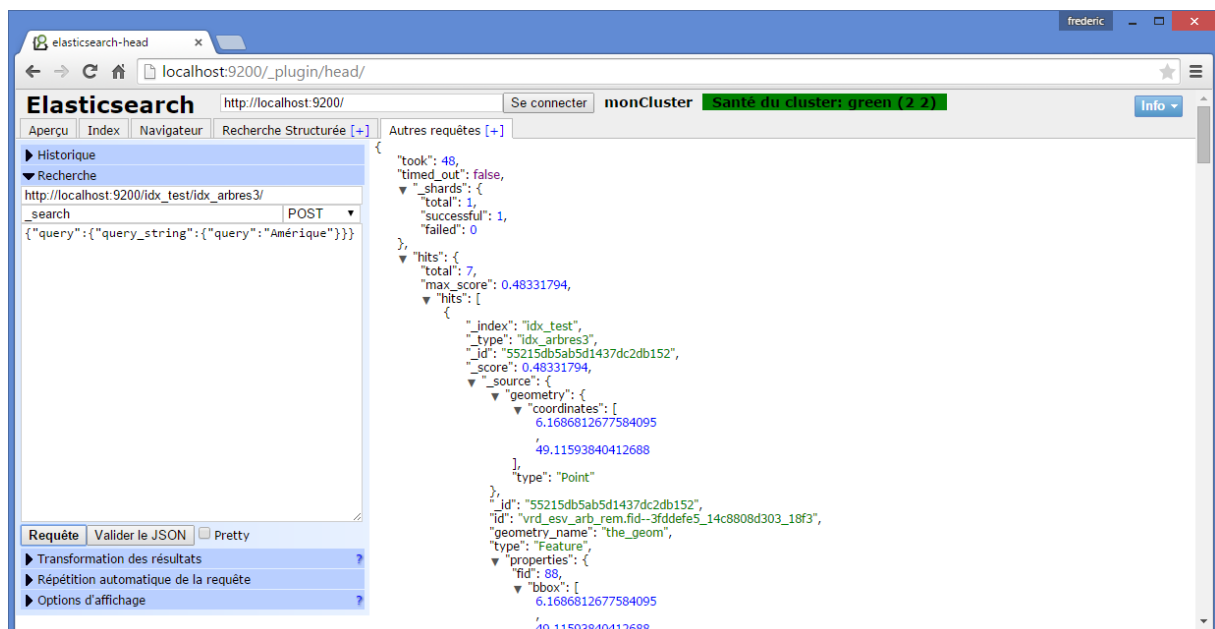
Elasticsearch DEBUG: 2015-04-11T13:40:32Z
Request complete

All is well

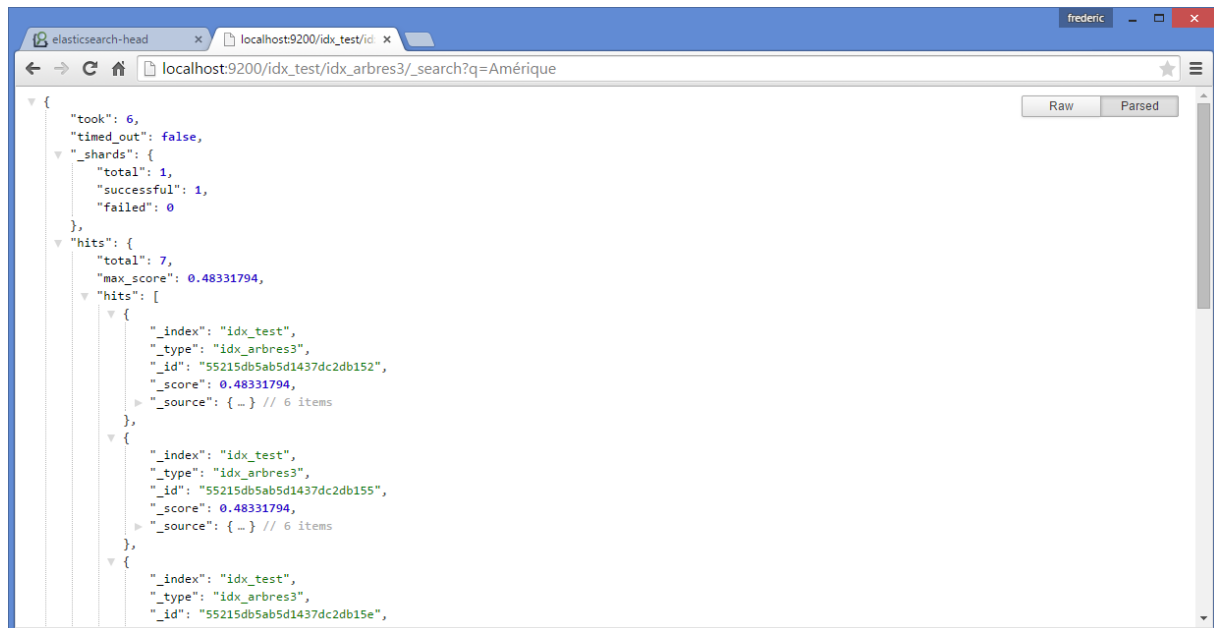
```

Lorsque l'on démarre le site avec ce bout de code, on peut voir que le serveur a retourné un code http 200. La connexion est donc OK.

Ici, on se rend compte que l'on peut interroger ElasticSearch comme un service web. Pour s'en convaincre, on peut tenter une requête à travers le navigateur.



Le résultat que l'on obtient avec cette interface peut également être obtenu avec l'URL suivante.



Il suffit de spécifier dans l'URL quel est l'index manipulé et de passer les paramètres de la requête.

Il aurait donc été possible de ne pas installer le plugin dans NodeJS et de l'interroger directement via le client de l'application web.

Maintenant que le plugin est installé, il serait dommage de ne pas s'en servir. On peut toujours justifier notre démarche en expliquant qu'il est plus propre de passer par NodeJS pour contrôler au mieux les données manipulées par l'utilisateur. Dans un mode MVC, il n'est pas acceptable qu'une vue puisse accéder directement aux données sans contrôle.

Les prochaines étapes consisteront donc à implémenter une nouvelle URL dans le routage de notre application et de transférer les paramètres dans une requête à destination d'ElasticSearch. On pourra ensuite enrichir notre interface pour que l'utilisateur puisse rentrer un texte correspondant à sa recherche. Pour finir, il faudra créer un conteneur pour lister le résumé de chaque document qui sera remonté.

```

1 var express = require('express');
2 var mongoose = require('mongoose');
3 var url = require('url');
4 var elasticsearch = require('elasticsearch');
5
6 var router = express.Router();
7 var SchemaArbre = mongoose.connection.model('Arbre', mongoose.schemas.SchemaArbre);
8
9
10 /* GET JSON : requête ElasticSearch */
11 router.get('/search', function(req, res) {
12   var client = new elasticsearch.Client({
13     host: 'localhost:9200'
14   });
15   var recherche = req.query.q;
16   console.log("recherche = " + recherche);
17   client.search({
18     q: recherche
19   }).then(
20     function (results) { writeJsonResponse(res, results); },
21     function (error) { console.trace(error.message); }
22   );
23 });
24
25 /* GET JSON : liste des auteurs */
26 router.get('/auteurs', function(req, res) {
27   SchemaArbre.find().distinct('properties.auteur', function (err, auteurs) {

```

On commence par créer l'action dans le routage du site. On choisit l'URL `/search` pour traiter les recherches plein texte. On ouvre un client vers ElasticSearch en précisant la chaîne de connexion. Ensuite, on récupère en ligne 15 le paramètre `q` passé dans l'URL. Ce paramètre contiendra le texte saisi par l'utilisateur. La valeur est simplement passée comme valeur pour la query qui sera adressée à ElasticSearch.

En réponse, on traite soit un succès (auquel cas on retourne la réponse au format Json sans transformation) soit un échec.

```

{
  "took": 3,
  "timed_out": false,
  "_shards": {
    "total": 2,
    "successful": 2,
    "failed": 0
  },
  "hits": {
    "total": 8,
    "max_score": 0.48116273,
    "hits": [
      > { }, // 5 items
      > { }, // 5 items
      > { }, // 5 items
      > { }, // 5 items
      > { }, // 5 items
      > { }, // 5 items
      > { }, // 5 items
      > { } // 5 items
    ]
  }
}

```

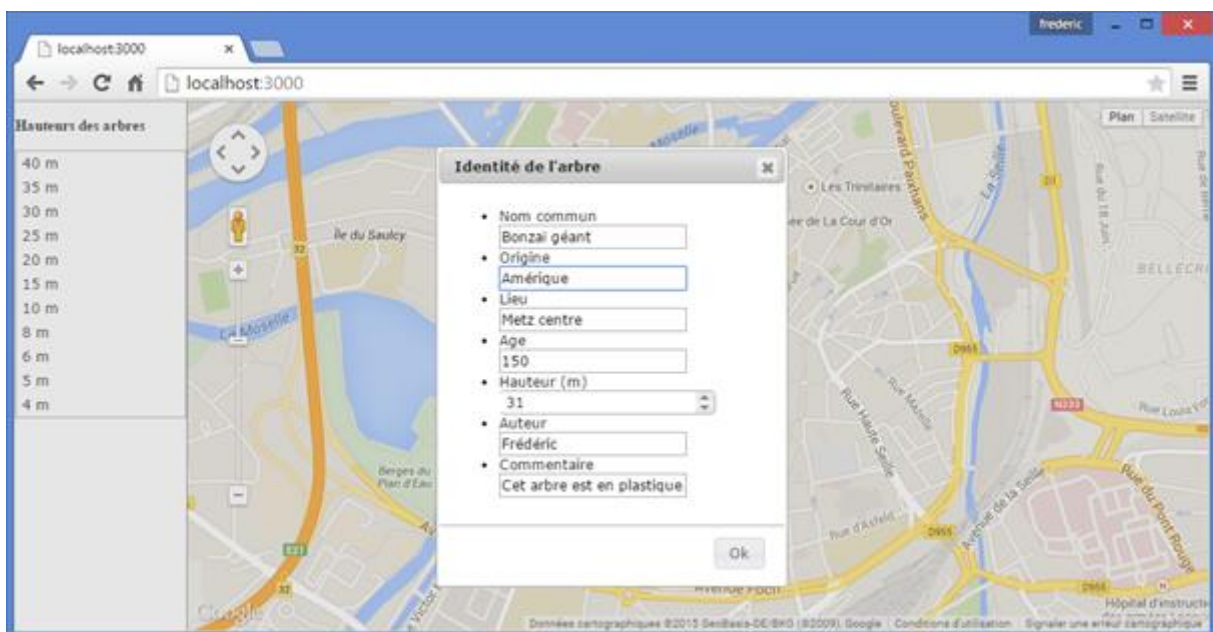
On tente un appel à cette ressource en passant le paramètre `q=Amérique`. Le résultat est conforme aux précédents.


```
Microsoft Windows [version 6.3.9600]
(c) 2013 Microsoft Corporation. Tous droits réservés.

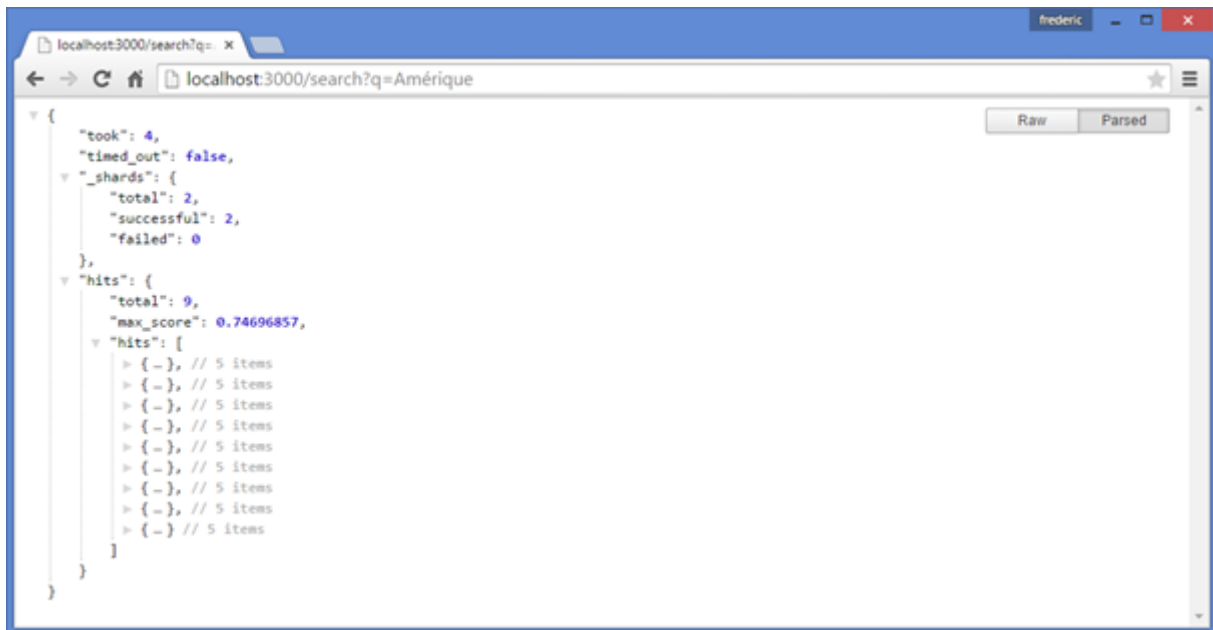
C:\Windows\system32>cd "c:\Program Files\nodejs\Site_01"
c:\Program Files\nodejs\Site_01>npm start
> Site_01@0.0.0 start c:\Program Files\nodejs\Site_01
> node ./bin/www

GET / 304 33.204 ms - -
GET /favicon.ico 404 1479.678 ms - 1777
GET /hauteurs 200 1496.146 ms - -
recherche = Amérique
GET /search?q=Am%C3%A9rique 200 38.388 ms - -
```

On retrouve une trace de cet appel dans la console (voir ligne 16 dans le code).

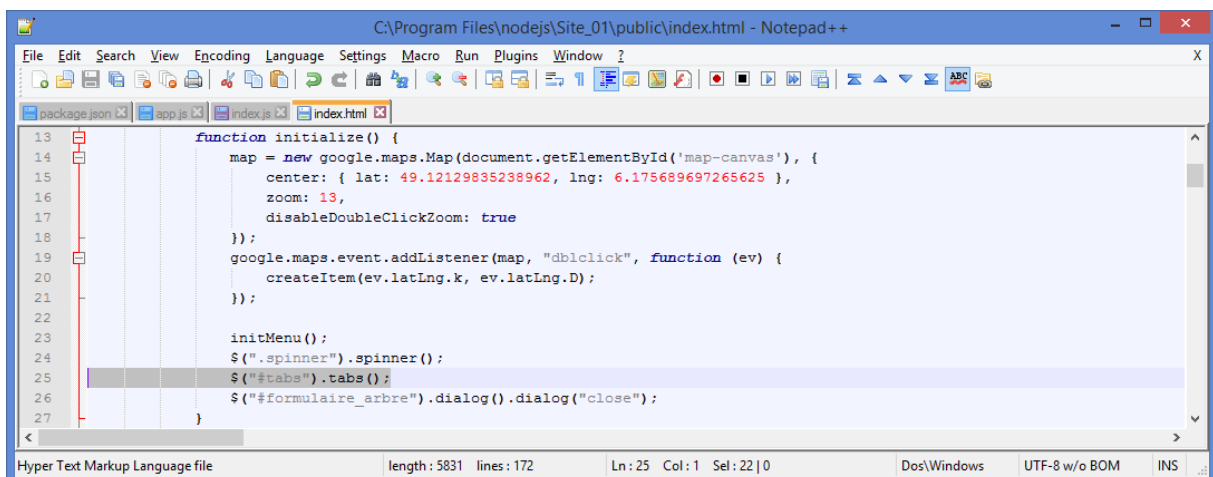


C'est le moment de voir si la river fonctionne efficacement. Je crée un nouvel arbre avec le formulaire de saisie.



Nous avons jusque-là 8 documents contenant le mot clé « Amérique ». On a maintenant un résultat remontant 9 documents. La mise à jour m'a semblée quasi instantanée. C'est une très bonne nouvelle et cela assure une synchronisation efficace entre la base de données de référence MongoDB et la base d'indexation ElasticSearch.

On prépare maintenant les menus de recherche.



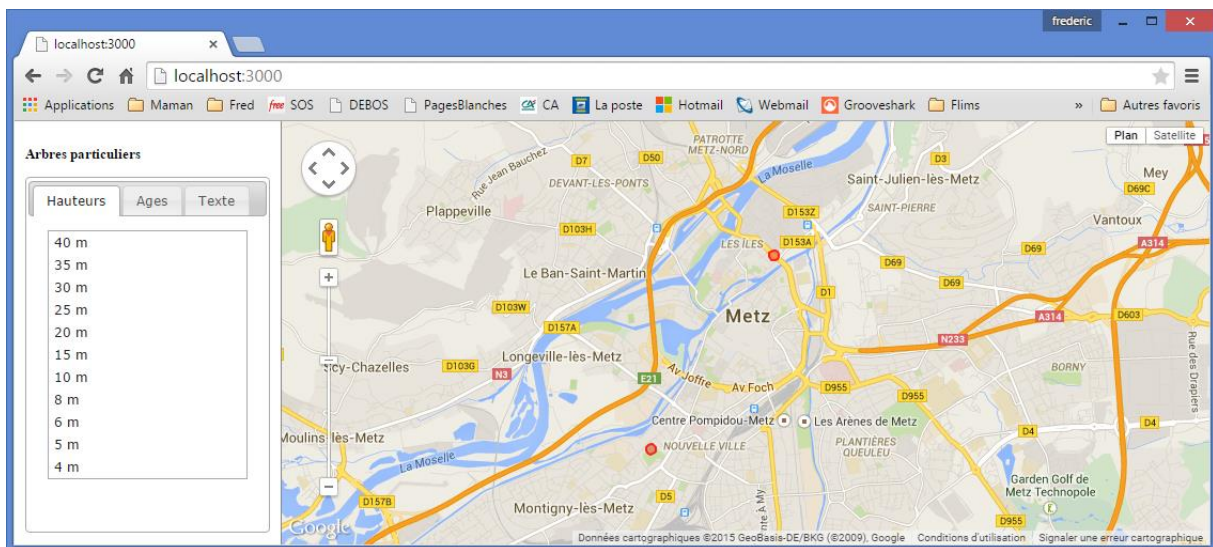
J'opte pour une disposition en onglets dans lesquels on pourra choisir soit une recherche par hauteur, soit par âge, soit libre (plein texte). Pour créer ces onglets, j'utilise JQueryUI avec son outil `tabs`.

```

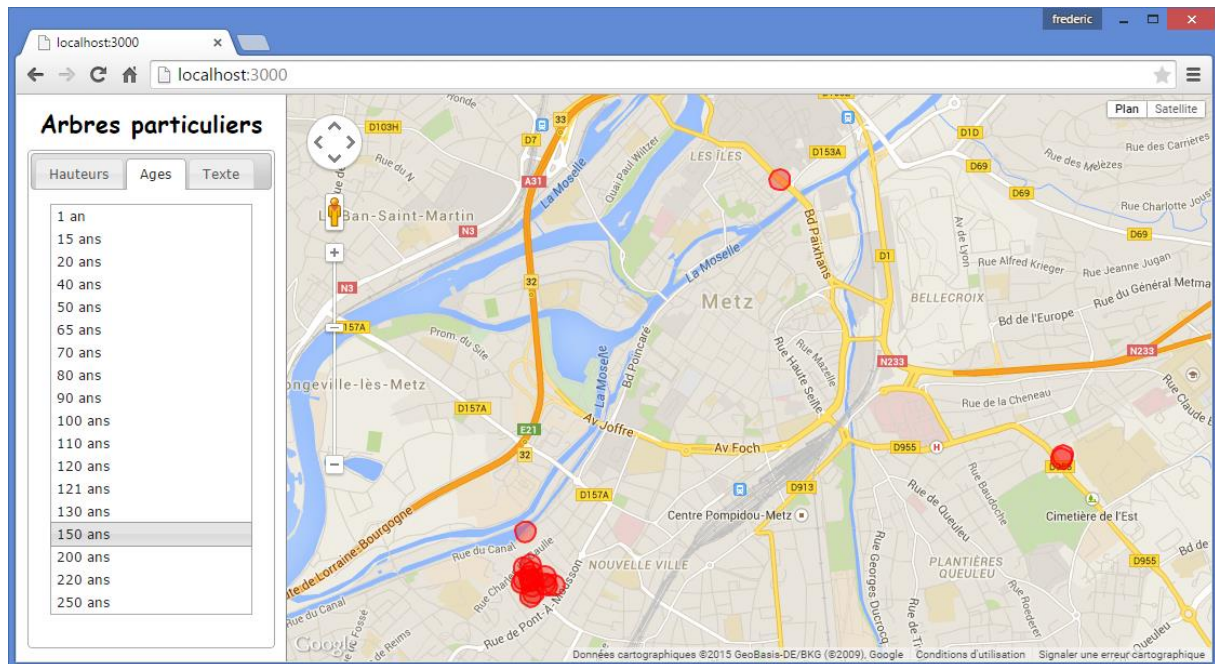
139 </head>
140 <body>
141   <div id="left_pan">
142     <h3>Arbres particuliers</h3>
143     <div id="tabs">
144       <ul>
145         <li><a href="#tabs-1">Hauteurs</a></li>
146         <li><a href="#tabs-2">Ages</a></li>
147         <li><a href="#tabs-3">Texte</a></li>
148       </ul>
149       <div id="tabs-1">
150         <div id="menu_hauteurs"><ul></ul></div>
151       </div>
152       <div id="tabs-2">
153         <div id="menu_ages"><ul></ul></div>
154       </div>
155       <div id="tabs-3"></div>
156     </div>
157   </div>
158   <div id="map-canvas"></div>
159   <div id="formulaire_arbre" title="Identité de l'arbre">
160     <h3></h3>
161     <ul>
162       <li><div>Nom commun</div><input name="nom_commun"/></li>
163       <li><div>Origine</div><input name="origine"/></li>

```

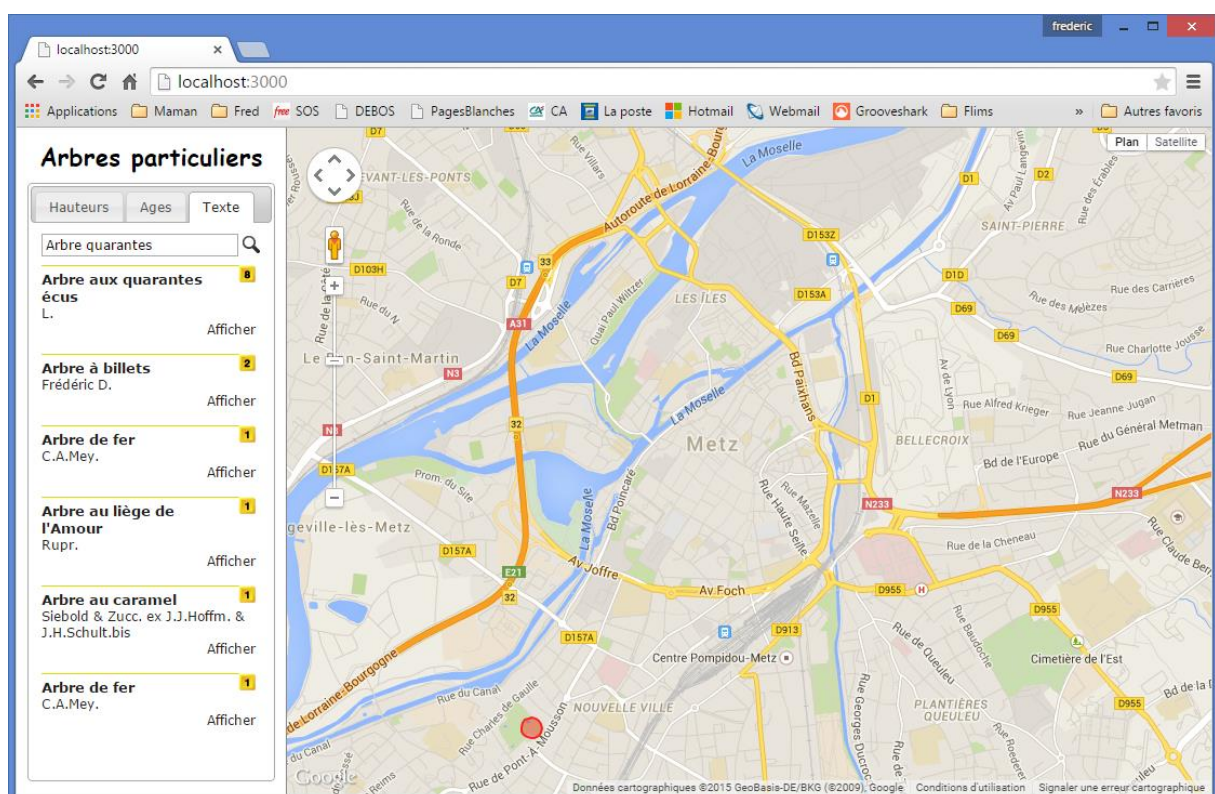
Cela passe forcément par quelques modifications dans la structure de la page.



Mais le résultat est très agréable à l'utilisation. On retrouve ici la liste des hauteurs.



Il nous faut rajouter la liste des âges que l'on avait codé précédemment.



Et pour finir, on ajoute un champ texte permettant la saisie de notre recherche plein texte. A la réception de la réponse, on remplit la liste se trouvant juste en dessous. Un clic sur l'un des liens « Afficher » permet de localiser l'arbre sur la carte.

Le détail complet du code final est disponible dans les annexes.

Conclusion

Impressions

J'ai eu grand plaisir à développer ce projet. Il m'a permis de réaliser certains objectifs personnels que je souhaitais atteindre depuis longtemps tels que le développement d'une application cartographique ou le déploiement et développement sur NodeJS. La seconde satisfaction que j'y ai éprouvée est liée à la découverte du monde des bases de données documentaires. Je me suis rendu compte que ce type de base de données pouvait offrir une alternative aux SGBD relationnels.

SGBD relationnel VS NoSQL

Il faut garder à l'esprit que les schémas ne sont pas rigides en base NoSQL. Heureusement, des plugins comme Mongoose permettent de se forcer à définir fortement ces schémas dans les applications, ce qui est une bonne chose pour l'intégrité des données persistées et pour le bon fonctionnement des systèmes qui auront à travailler avec.

C'est donc une alternative qui offre de nouvelles fonctionnalités : le MapReduce est une solution efficace pour le regroupement de données ; la recherche plein texte est un outil qui n'a pas son pareil dans les systèmes relationnels. Avec un tel constat, on pourrait facilement imaginer que les systèmes NoSQL seront les SGBD de demain. Cependant, les systèmes relationnels gardent encore une avance évidente sur la mise en place de contraintes d'intégrité (schémas, foreign keys ; unique keys...), sur les opérations de jointure, sur la mise en place de vues, de procédures stockées, de triggers...

Sur de petits projets, les deux systèmes peuvent se montrer à la hauteur. Lorsque le projet prendra de l'ampleur, il est possible qu'un mauvais choix fasse une grosse différence en termes de temps de développement et de maintenance.

Webographie

Les manuels de référence

Le cours NFE204 :

<http://b3d.bdpedia.fr/>

Manuels de référence de MongoDB :

<http://docs.mongodb.org/manual/>

Recommandations W3C sur JavaScript :

<http://www.w3schools.com/jsref/>

Manuel d'utilisation de NodeJS :

<https://nodejs.org/api/>

Manuel d'utilisation d'ExpressJS :

<http://expressjs.com/guide/routing.html>

Manuel d'utilisation de Mongoose :

<http://mongoosejs.com/docs/api.html>

Manuel d'utilisation de l'API Google Map :

<https://developers.google.com/maps/documentation/javascript/reference>

Manuel d'utilisation de l'API JQuery :

<https://api.jquery.com/>

Manuel d'utilisation de l'API JQuery UI :

<http://api.jqueryui.com/>

Manuel d'utilisation d'ElasticSearch :

<http://www.elastic.co/guide/>

Les tutoriaux :

Déploiement d'un serveur NodeJS :

<http://www.hongkiat.com/blog/node-js-server-side-javascript/>

Utilisation de base du plugin ExpressJS :

<http://shapedshed.com/creating-a-basic-site-with-node-and-express/>

Réalisation d'un projet avec le trio NodeJS – ExpressJS – MongoDB :

<http://cwbuecheler.com/web/tutorials/2013/node-express-mongo/>

Comment récupérer une clé d'utilisation pour l'API Google Map :

https://developers.google.com/maps/documentation/javascript/tutorial#api_key

ElasticSearch dans NodeJS :

<http://www.elastic.co/guide/en/elasticsearch/client/javascript-api/current/quick-start.html>

Autres sources :

Comment démarrer NodeJS sur une source :

<http://stackoverflow.com/questions/14154357/how-to-run-server-written-in-js-with-node-js>

Résolution du problème de lien relatif vers Bson :

<http://stackoverflow.com/questions/21656420/failed-to-load-c-bson-extension>

Répondre au format Json dans NodeJS :

<http://stackoverflow.com/questions/5892569/responding-with-a-json-object-in-nodejs-converting-object-array-to-json-string>

Cast d'une chaine en Int dans une requête MongoDB :

<http://stackoverflow.com/questions/3521601/is-it-possible-to-cast-in-a-mongodb-query>

Récupération des paramètres de l'URL dans la page de routage d'ExpressJS :

<http://stackoverflow.com/questions/6912584/how-to-get-get-query-string-variables-in-express-on-node-js>

Téléchargement de la base de données des arbres particuliers à Metz :

<https://www.data.gouv.fr/fr/datasets/arbres-remarquables-1/>

Liste des bibliothèques JavaScript publiques hébergées par Google :

<https://developers.google.com/speed/libraries/>

Suppression de formes dans la Google Map :

<http://stackoverflow.com/questions/6058904/removing-a-google-maps-circle-shape>

Contrôler la position et le zoom de Google Map dynamiquement :

<http://stackoverflow.com/questions/5054515/zoom-in-to-marker-google-maps>

Utilisation de distinct avec Mongoose :

<http://stackoverflow.com/questions/6043847/how-do-i-query-for-distinct-values-in-mongoose>

Tri et distinct dans Mongoose :

<http://stackoverflow.com/questions/27968930/sort-and-distinct-in-mongoose>

Ajout ou modification avec Mongoose :

<http://stackoverflow.com/questions/7267102/how-do-i-update-upsert-a-document-in-mongoose>

Problème d'_id : conversion de String vers ObjectId :

<http://www.jotform.org/javascript-codes/mongoose-convert-string-to-objectid/>

Outils :

Formatage Json en ligne :

<http://jsonformatter.curiousconcept.com/>

Formatage Json dans le navigateur :

<https://chrome.google.com/webstore/detail/json-formatter/bcjindcccaagfpajjjmafapmmgkkhgoa?hl=fr>

Plugin JQuery pour l'affichage de notifications :

<http://notifyjs.com/>

Installation de la machine virtuelle Java :

<https://www.java.com/fr/download/>

Commandes CURL pour Windows :

<http://curl.haxx.se/dlwiz/?type=bin&os=Win32&flav=-&ver=2000%2FXP>

Annexes

Code dans le fichier `/app.js`

```
var express = require('express');
var path = require('path');
var favicon = require('serve-favicon');
var logger = require('morgan');
var cookieParser = require('cookie-parser');
var bodyParser = require('body-parser');

// Cette page javascript est interprétée au démarrage du serveur.
// C'est à cette occasion que l'on crée une connexion à la base de données MongoDB
var mongoose = require('mongoose');
mongoose.connect('mongodb://localhost/test');

// Chaque document de MongoDB doit avoir un schéma pour être manipulé par mongoose.
// On donne une description de chaque champ du document en précisant son type.
var SchemaArbre = new mongoose.Schema({
  _id: mongoose.Schema.ObjectId, type: {type: String}, id: String,
  geometry: {coordinates: [Number], type: {type: String}},
  geometry_name: String,
  properties: {
    longitude: Number, diametre: String, age_estimation: String,
    bbox: [Number], fid: Number, lieu: String, hauteur: Number,
    auteur: String, parcelle: Number, type: {type: String}, nom_commun: String,
    nom_latin: String, latitude: Number, commentaire: String, id: Number,
    origine: String
  },
}, { collection: 'arbres3' }); // Ce schéma est associé à des documents provenant de
// la collection arbres3 que l'on retrouve dans la connexion.

// Comme un schéma ne peut être déclaré qu'une fois, on le crée dans cette page et
// on la transmet aux autres pages en l'incluant dans un nouvel objet qui sera affecté
// à mongoose.
mongoose.schemas = {
  SchemaArbre: SchemaArbre
};

var routes = require('./routes/index');
var users = require('./routes/users');

var app = express();

app.use(function(req, res, next) {
  next();
});

// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'jade');

// uncomment after placing your favicon in /public
//app.use(favicon(path.join(__dirname, 'public', 'favicon.ico')));
app.use(logger('dev'));
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/', routes);
app.use('/users', users);
```

```
// catch 404 and forward to error handler
app.use(function(req, res, next) {
  var err = new Error('Not Found');
  err.status = 404;
  next(err);
});

// error handlers

// development error handler
// will print stacktrace
if (app.get('env') === 'development') {
  app.use(function(err, req, res, next) {
    res.status(err.status || 500);
    res.render('error', {
      message: err.message,
      error: err
    });
  });
}

// production error handler
// no stacktraces leaked to user
app.use(function(err, req, res, next) {
  res.status(err.status || 500);
  res.render('error', {
    message: err.message,
    error: {}
  });
});

module.exports = app;
```

Code dans le fichier `/routes/index.js`

```
// On commence par importer toutes les librairies qui seront nécessaires au traitement des différentes requêtes.
var express = require('express');
var mongoose = require('mongoose');
var url = require('url');
var elasticsearch = require('elasticsearch');

var router = express.Router();
// On récupère le schéma qui a été décrit dans le fichier précédent et on en crée un model.
var SchemaArbre = mongoose.connection.model('Arbre', mongoose.schemas.SchemaArbre);

/* GET JSON : requête ElasticSearch */
// Les requêtes plein texte arriveront sur cette URL.
router.get('/search', function(req, res) {
  // On crée une nouvelle connexion sur notre serveur ElasticSearch
  var client = new elasticsearch.Client({
    host: 'localhost:9200'
  });
  // « q » est le paramètre de la requête qui sera transmis à ElasticSearch.
  var recherche = req.query.q;
  // Pour simplifier le débogage, on affiche le paramètre recherché dans la console.
  console.log("recherche = " + recherche);
  // Le client nous sert à envoyer la recherche au serveur.
  client.search({
    q: recherche
  }).then(
    function (results) {
      // Les résultats récupérés sont transmis dans la réponse au client web.
      writeJsonResponse(res, results.hits.hits);
    },
  ),
```

```

        // En cas d'erreur, on affiche le message dans la console.
        function (error) { console.trace(error.message); }
    });
});

/* GET JSON : regroupement par age (map_reduce) */
// On avait vu au cours du projet que la méthode du mapReduce n'était pas la plus adaptée
// pour les requêtes dont nous avons besoin car elle retrouvait la totalité de la base.
// J'ai tout de même gardé ce code pour avoir un exemple d'implémentation.
router.get('/ages', function(req, res) {
    // on prépare les opérations des différentes étapes du MapReduce.
    var operations = {
        map : function(){
            // Dans la fonction map, on qualifie les documents par l'âge de l'arbre.
            var age = this.properties.age_estimation.replace(" ans", "");
            // Conversion en int si possible pour un meilleur tri des clés
            if(age*1 == age) age = age*1;
            emit(age, this);
        },
        reduce : function(key, values){
            // Dans la fonction reduce, on retourne un objet contenant la clé et les valeurs.
            return {key:key, values:values};
        },
        finalize : function(key, reduced){
            // Dans la fonction finalize, on a la possibilité de traiter l'ensemble des
            // documents, à savoir, ceux qui ont été groupé, ainsi que ceux qui ne sont pas
            // passés dans la fonction reduce.
            var finalized = reduced;
            if(reduced._id){ // passage pour les documents non réduites
                finalized = [reduced];
            }
            else { // passage pour les documents « reduced »
                finalized = reduced.values;
            }
            return finalized;
        }
    };
    // Ces opérations sont transmises à mongoose à travers le schéma des documents arbres
    // dans le but de réaliser une action de MapReduce.
    SchemaArbre.mapReduce(operations, function(err, docs){
        // On obtient en réponse une liste de documents ou, éventuellement, une erreur.
        if(err) console.log(err);
        // Les documents récupérés sont transmis en réponse au navigateur
        writeJsonResponse(res, docs);
    });
});

/* GET JSON : liste des hauteurs */
// Cette fonction réalise simplement une liste de toutes les hauteurs d'arbres disponible
// dans la base.
router.get('/hauteurs', function(req, res) {
    // La liste que l'on souhaite obtenir ne doit pas contenir de doublons. On applique donc
    // un distinct.
    SchemaArbre.find().distinct('properties.hauteur', function (err, hauteurs) {
        if(err) console.log(err);
        writeJsonResponse(res, hauteurs);
    });
});

/* GET JSON : liste des arbres pour une hauteur */
// Ici, on traite une demande d'arbres. En paramètre, on reçoit une hauteur et MongoDB doit
// nous retourner tous les arbres dont la hauteur correspond.
router.get('/arbres', function(req, res) {
    // On commence par extraire le paramètre « hauteur » de l'URL.
    var url_parts = url.parse(req.url, true);
    var hauteur = parseInt(url_parts.query.hauteur);

```

```
// Ensuite, on construit la query qui sera transmise à MongoDB.
SchemaArbre.find({'properties.hauteur' : hauteur}, function (err, docs_arbre) {
  // Les documents retournés sont écrits dans la réponse.
  if(err) console.log(err);
  writeJsonResponse(res, {arbres:docs_arbre});
});

/* POST save */
// Cette page reçoit les extraits de documents qui devront être persistés. Comme le flux peut
// être conséquent, on passe par une méthode POST.
router.post('/save', function(req, res) {
  // On commence par extraire le document arbre candidat qui a été transmis par le
  // formulaire.
  var json_arbre = JSON.parse(req.body.arbre);
  var arbre_id = mongoose.Types.ObjectId(json_arbre._id);

  // On cherche ensuite ce document pour savoir s'il existe en base.
  SchemaArbre.findOne({_id: arbre_id}, function(e, doc){
    // Si on ne récupère aucun document, c'est que la base ne le contient pas. Il s'agit
    // d'un nouveau document. On va donc l'insérer en base.
    if (doc == null) {
      // On crée une nouvelle instance de document arbre à partir du schéma en
      // passant en paramètre le document candidat. Seuls les champs compatibles seront
      // conservés.
      var doc_arbre = new SchemaArbre(json_arbre);
      // On crée ensuite un nouvel identifiant
      doc_arbre._id = mongoose.Types.ObjectId();
      // Et on sauvegarde le document.
      doc_arbre.save(function(err){
        // Un acquittement est retourné pour que l'utilisateur sache si la tâche
        // s'est bien déroulée.
        if(err) console.log(err);
        res.end(err?"ko":"ok");
      });
    }
    // Si le document n'est pas nul, c'est qu'il existe déjà en base. On fait donc une
    // mise à jour de celui-ci
    else {
      // Le schéma nous permet d'effectuer une recherche par identifiant. La fonction
      // suivante combine la recherche et la mise à jour. Il faut donc passer aussi
      // le document candidat qui contient les champs mis à jour.
      SchemaArbre.findOneAndUpdate({_id: arbre_id}, json_arbre, function(err){
        // Un acquittement est retourné pour que l'utilisateur sache si la tâche
        // s'est bien déroulée.
        if(err) console.log(err);
        res.end(err?"ko":"ok");
      });
    }
  });
});

/* POST delete */
// L'URL suivante est dédiée à la suppression d'un document.
router.post('/delete', function(req, res) {
  // On extrait le document passé en POST
  var json_arbre = JSON.parse(req.body.arbre);
  // Duquel on peut récupérer l'identifiant.
  var arbre_id = mongoose.Types.ObjectId(json_arbre._id);
  // A partir de cet identifiant, on recherche le document et on le supprime.
  SchemaArbre.findOne({_id:arbre_id}).remove(function(err){
    // Un acquittement est retourné pour que l'utilisateur sache si la tâche s'est bien
    // déroulée.
    if(err) console.log(err);
    res.end(err?"ko":"ok");
  });
});
```

```

/* Retourne un objet json dans le flux http */
// Cette fonction permet de sérialiser un objet JSON et de le transmettre dans le flux
// http de la réponse.
function writeJsonResponse(res, jsonObj){
    res.writeHead(200, {"Content-Type": "application/json; charset=utf-8"});
    var jsonStr = JSON.stringify(jsonObj);
    res.end(jsonStr);
}

module.exports = router;

```

Code dans le fichier /public/index.html

```

<html>
  <head>
    <!-- JQuery pour la manipulation du DOM et les requêtes Ajax -->
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js">
    </script>
    <!-- JQuery UI pour les composants graphiques « onglet » et « popup » -->
    <script src="https://ajax.googleapis.com/ajax/libs/jqueryui/1.11.3/jquery-ui.min.js">
    </script>
    <!-- NotifyJs pour les messages d'alerte -->
    <script src="http://notifyjs.com/dist/notify-combined.min.js"></script>
    <!-- Google Map API pour l'affichage et la manipulation de la cartographie -->
    <script src="https://maps.googleapis.com/maps/api/js?key=AIzaSyADaXClIhDpUYTcYs33SNC
vp0IpYE17F4"></script>
    <!-- Script de notre application -->
    <script>
      // déclaration des variables globales
      var map = null;
      var cercles = new Array();
      var arbre_affiche = null;
      var arbres_par_age = {};
      var arbres_recherche = {};
      var tplt_resultItem;

      // fonction appelée au démarrage de la page
      function initialize() {
        // création de la carte
        map = new google.maps.Map(document.getElementById('map-canvas'), {
          center: { lat: 49.12129835238962, lng: 6.175689697265625 },
          zoom: 13,
          // on empêche le zoom au double clic car on utilisera cette action pour
          // la création d'un nouvel arbre
          disableDoubleClickZoom: true
        });
        // le double clic sur la carte est utilisé pour créer un nouvel arbre
        // cela nous permettra de définir facilement une position GPS
        google.maps.event.addListener(map, "dblclick", function (ev) {
          createItem(ev.latLng.k, ev.latLng.D);
        });

        // on initialise les listes des menus dans le panneau de gauche
        initMenus();
        // on crée les spinners (champs text en champs number)
        $(".spinner").spinner();
        // on crée les onglets du menu à gauche
        $("#tabs").tabs();
        // on crée la popup que l'on ferme par défaut
        $("#formulaire_arbre").dialog().dialog("close");
        // on attrape la soumission du formulaire de recherche plein texte
        $("#tabs-3>form").submit(function(event) {
          // lorsque la recherche est validée, on passe le champ cherché à la
          // fonction rechercheTexte
          rechercheTexte($("#tabs-3 input[type=search]").val());
        });
      }
    </script>
  </head>
  <body>
    <div id="map-canvas">
    </div>
  </body>
</html>

```

```

        // l'action par défaut (changement url) est annulé
        event.preventDefault();
    });
    // on met en variable globale le DOM correspondant au template des balises
    // LI dans lesquelles on affichera les documents retournés par la recherche
    // plein texte
    tplt_resultItem = document.querySelector('#tplt_resultItem').content;
}

// lorsque la recherche plein texte est validée, on déclenche cette fonction
// en paramètre, on reçoit la valeur recherchée
function rechercheTexte(texte){
    // on appelle la page « /search » en passant le paramètre « q »
    $.get("/search?q=" + texte, function(es_docs){
        // on garde la liste des documents retournés en variable globale
        arbres_recherche = es_docs;
        // on recherche la balise UL où seront insérés les réponses
        var $listDoc = $("#tabs-3>ul");
        // on vide cette liste
        $listDoc.empty();
        for(var i in es_docs){
            var doc = es_docs[i];
            // on crée un nouveau nœud LI qui sera une copie du template
            $item = $(document.importNode(tpl_resultItem, true));
            // on modifie le titre en y mettant le nom de l'arbre
            $item.find("h3").text(doc._source.properties.nom_commun);
            // on modifie le paragraphe en y écrivant le nom de l'auteur
            $item.find("p").eq(0).text(doc._source.properties.auteur);
            // le lien appellera la fonction « selectTree »
            $item.find("a").attr("href", "javascript:selectTree(" + i + ")");
            // le score donné par ElasticSearch est converti en note sur 10
            var note = Math.floor(doc._score*10);
            if(note>9) note=9;
            if(note==0) note='o'; // l'image 0 n'existe pas
            // et on affiche cette note à travers une image
            $item.find("img").attr("src", "http://www.r2djeux.com/design/lettre_"
+note+".gif");

            // la balise LI est maintenant remplie, on l'ajoute à la liste
            $listDoc.append($item);
        }
    });
}

// l'utilisateur vient de cliquer sur un lien permettant de voir un arbre
function selectTree(docIdx){
    // l'arbre est inséré dans une liste pour afficher sur la carte
    setTreesOnMap([arbres_recherche[docIdx]._source]);
}

// l'utilisateur vient de cliquer sur une hauteur
function selectHeight(hauteur){
    // on appelle la page « /arbres » en passant le paramètre hauteur
    $.get("/arbres?hauteur="+hauteur, function(docs){
        // on passe le résultat à « setTreesOnMap » pour les afficher sur
        // la carte
        setTreesOnMap(docs.arbres);
    });
}

// l'utilisateur vient de cliquer sur un âge
function selectAge(ageIdx) {
    // on sélectionne les arbres correspondant à l'âge dans la liste enregistrée
    // en variable globale
    setTreesOnMap(arbres_par_age[ageIdx].value);
}

// cette fonction permet de supprimer tous les arbres représentés sur la carte
function cleanMap(){

```

```

        for(var i in cercles){
            cercles[i].setMap(null);
        }
        cercles = new Array();
    }

    // l'utilisateur vient de demander l'affichage d'une liste d'arbre
    function setTreesOnMap(arbres){
        // on commence par nettoyer la carte
        cleanMap();
        for(var idx in arbres){
            var doc_arbre = arbres[idx];
            // pour chaque arbre, on crée un cercle à la position correspondante
            var cercle=new google.maps.Circle({
                strokeColor: '#FF0000', strokeOpacity: 0.8, strokeWeight: 2,
                fillColor: '#FF0000', fillOpacity: 0.35, radius: 60,
                center: new google.maps.LatLng(
                    doc_arbre.geometry.coordinates[1],
                    doc_arbre.geometry.coordinates[0]),
                map: map, clickable:true,
                doc_arbre:doc_arbre
            });
            // si on clique sur le cercle
            google.maps.event.addListener(cercle, 'click', function(ev){
                // on lance l'édition de l'arbre
                editItem(this.doc_arbre);
            });
            // on garde ce cercle en variable globale pour les retrouver au moment
            // où on doit les retirer de la carte
            cercles.push(cercle);
        }
    }

    // l'utilisateur vient de double cliquer sur la carte pour créer un nouvel arbre
    function createItem(latitude, longitude){
        // on recherche tous les champs textes du formulaire et on les vide
        $("#formulaire_arbre input").val("");
        // on crée un nouvel arbre en variable globale dans lequel on garde la
        // position choisie par l'utilisateur
        arbre_affiche = {
            geometry : { coordinates : [longitude, latitude] },
            properties : {}
        };
        // on affiche ensuite la popup en modal (la page en arrière plan n'est pas
        // accessible quand la popup est ouverte) et on ajoute un bouton OK qui
        // lancera la fonction de sauvegarde
        $("#formulaire_arbre").dialog({
            modal: true,
            buttons: {
                Ok: saveItem
            }
        });
    }

    // l'utilisateur vient de cliquer sur un cercle rouge pour éditer un arbre
    function editItem(arbre) {
        // on parcourt tous les champs textes du formulaire
        $("#formulaire_arbre input").each(function(){
            var $this = $(this);
            // chaque champ texte a un attribut « name » correspondant à un champ
            // du document arbre
            var property = $this.attr("name");
            // on écrit le champ correspondant dans ce champ texte
            $this.val(arbre.properties[property]);
        });
        // on garde l'arbre en cours d'édition en variable globale
        arbre_affiche = arbre;
    }

```



```

// on ouvre la popup en modal avec 2 boutons (supprimer et ok)
$("#formulaire_arbre").dialog({
    modal: true,
    buttons: {
        Supprimer: removeItem,
        Ok: saveItem
    }
});

}

// l'utilisateur vient de cliquer sur le bouton supprimer de la popup d'édition
function removeItem() {
    // on demande d'abord une confirmation
    if(confirm("Supprimer cet arbre?")){
        // si l'utilisateur confirme, on appelle la page « /delete » en passant
        // l'arbre en contenu POST : la réponse est envoyée à la fonction
        // « updateCallback »
        $.post("/delete",{arbre:JSON.stringify(arbre_affiche)}, updateCallback);
        // sans attendre la réponse, on ferme la popup
        $( this ).dialog( "close" );
    }
}

// l'utilisateur vient de valider le formulaire de saisie ou d'édition
function saveItem(){
    // on parcourt les champs texte du formulaire
    $("#formulaire_arbre input").each(function(){
        var $this = $(this);
        var property = $this.attr("name");
        // pour chaque champ texte, on met à jour le paramètre correspondant
        // dans le document arbre en cours d'édition
        arbre_affiche.properties[property] = $this.val();
    });
    // pour éviter les problèmes d'enregistrement en base, on met la hauteur
    // à 0 si la valeur n'est pas renseignée
    if(arbre_affiche.properties.hauteur=="") arbre_affiche.properties.hauteur=0;
    // on appelle ensuite la page « /save » en plaçant en contenu le document à
    // sauvegarder
    $.post("/save",{arbre:JSON.stringify(arbre_affiche)}, updateCallback);
    // sans attendre la réponse, on ferme la popup
    $("#formulaire_arbre").dialog( "close" );
}

// la page vient de se charger ou une mise à jour de la base vient d'être lancée,
// on met donc à jour les menus
function initMenus(){
    // pour le menu des hauteurs, on fait appel à la page « /hauteurs »
    $.get("/hauteurs",function(data){
        // cette page nous retourne simplement une liste de nombre que l'on tri
        data = data.sort(function(a, b){ return b - a });
        // on recherche le nœud de la liste où seront insérée les hauteurs
        var $ulHauteurs = $("#menu_hauteurs>ul");
        // comme cette liste est mise en forme par JQuery UI, on commence par la
        // nettoyer
        $ulHauteurs.menu().menu("destroy");
        // ensuite, on la vide
        $ulHauteurs.empty();
        for(var idx in data){
            var hauteur = data[idx];
            // pour chaque hauteur, on ajoute un lien de sélection
            $ulHauteurs.append (
                "<li onclick='javascript:selectHeight(" +
                hauteur + ")'>" + hauteur + " m</li>");
        }
        // ensuite on recrée la mise en forme avec JQuery UI
        $ulHauteurs.menu();
    });
}

```

```

    // pour le menu des âges, on appelle la page « /ages »
    $.get("/ages",function(data){
        // elle retourne un regroupement par âge par mapReduce que l'on place
        // en variable globale
        arbres_par_age = data;
        // on recherche ensuite la liste des âges dans le DOM de la page
        var $ulAges = $("#menu_ages>ul");
        // comme cette liste est mise en page par JQuery UI, on détruit cette
        // mise en page
        $ulAges.menu().menu("destroy");
        // on vide la liste
        $ulAges.empty();
        for(var idx in data){
            var age = data[idx]._id;
            // pour chaque âge, on met en forme l'affichage de l'âge
            var ans = age == age*1 ? age > 1 ? " ans" : " an" : "";
            // et on ajoute un lien de sélection
            $ulAges.append(
                "<li onclick='javascript:selectAge(" + idx + ")'>" +
                age + ans + "</li>");
        }
        // pour finir, on crée la mise en page du menu avec JQuery UI
        $ulAges.menu();
    });
}

// cette fonction est appelée en réponse à des requêtes Ajax
function updateCallback(content) {
    // pour le débogage, on affiche la réponse dans la console du navigateur
    console.log(content);
    // si la réponse est OK
    if (content == "ok") {
        // on crée un message pour l'utilisateur au dessus du titre de la page
        $.notify("Opération effectuée avec succès", {
            className: 'success',
            position:"top left"
        });
        // et on met à jour les menus qui ont peut être changé
        initMenus();
    } else {
        // sinon on affiche un message d'erreur
        $.notify("Une erreur a interrompu l'opération. Voir les logs.", {
            position: "top left"
        });
    }
}

// on attrape l'événement de chargement de la page pour lancer la fonction
// « initialize »
google.maps.event.addDomListener(window, 'load', initialize);
</script>
<!-- CSS de JQuery UI -->
<link rel="stylesheet" href="https://ajax.googleapis.com/ajax/libs/jqueryui/1.11.3/themes/smoothness/jquery-ui.css">
<!-- CSS de notre application -->
<style type="text/css">
    /* Affichage pleine hauteur de la carte */
    html, body, #map-canvas { height: 100%; margin: 0; padding: 0; }
    /* Panneau de gauche */
    #left_pan { padding:10px; position: absolute; width:229px;
                border-right:1px solid #888; height:calc(100% - 20px); }
    #map-canvas { width:calc(100% - 250px); left:250px; }
    /* Titre */
    body{ font-size: 0.7em; font-family: cursive; }
    h1 { margin: 0 0 10px 0; text-align: center;}
    /* Mise en page des onglets */
    #tabs { height: calc(100% - 50px);}
    #tabs>div {overflow-y: auto; height: calc(100% - 58px);}

```

```

#tabs-3 { padding:10px;}
/* Formulaire de recherche plein texte */
#tabs-3 form { margin-bottom: 5px;}
#tabs-3 h3{ margin: 0; font-size: 1.05em; border-top: 1px solid #DD0;
padding-top: 4px;}
/* Documents affichés dans la liste de réponses de la recherche plein texte */
#tabs-3 ul{ padding : 3px 5px 0 0; overflow-y: auto; height: calc(100% - 30px);
margin: 0;}
#tabs-3 img{ float: right;}
#tabs-3 p{ margin: 0;}
#tabs-3 p+p{ text-align: right; margin-bottom: 15px;}
#tabs-3 a{ text-decoration:none;}
#tabs-3 a:hover{ text-decoration:underline;}
</style>
</head>
<body>
<!-- Panneau de gauche -->
<div id="left_pan">
<h1>Arbres particuliers</h1>
<!-- Conteneur à onglets -->
<div id="tabs">
<ul>
<!-- Les 3 onglets -->
<li><a href="#tabs-1">Hauteurs</a></li>
<li><a href="#tabs-2">Ages</a></li>
<li><a href="#tabs-3">Texte</a></li>
</ul>
<!-- Page correspondant à l'onglet des hauteurs -->
<div id="tabs-1">
<div id="menu_hauteurs"><ul></ul></div>
</div>
<!-- Page correspondant à l'onglet des âges -->
<div id="tabs-2">
<div id="menu_ages"><ul></ul></div>
</div>
<!-- Page correspondant à l'onglet de recherche plein texte -->
<div id="tabs-3">
<!-- Formulaire de recherche plein texte -->
<form>
<input type="search" style="width: calc(100% - 20px)"/>
<input src="http://environment.yale.edu/common/icons/symbolize/16/sea
rch.png" type="image" style="vertical-align: text-bottom;"/>
</form>
<!-- Liste d'affichage des résultats de recherche plein texte -->
<ul> <!-- Modèle d'élément de liste pour un document -->
<template id="tplt_resultItem">
<img/><h3></h3><p></p><p><a href="">Afficher</a></p>
</template>
</ul>
</div>
</div>
</div>
<div id="map-canvas"></div> <!-- Zone réservée à la carte -->
<!-- Zone qui sera transformée en popup par JQuery UI -->
<div id="formulaire_arbre" title="Identité de l'arbre">
<ul> <!-- Liste des champs pour l'édition d'un document -->
<li><div>Nom commun</div><input name="nom_commun"/></li>
<li><div>Origine</div><input name="origine"/></li>
<li><div>Lieu</div><input name="lieu"/></li>
<li><div>Age (en années)</div><input name="age_estimation"/></li>
<!-- Spinner permettra de transformer le champs text en champ number -->
<li><div>Hauteur (m)</div><input name="hauteur" class='spinner'/></li>
<li><div>Auteur</div><input name="auteur"/></li>
<li><div>Commentaire</div><input name="commentaire"/></li>
</ul>
</div>
</body>
</html>

```